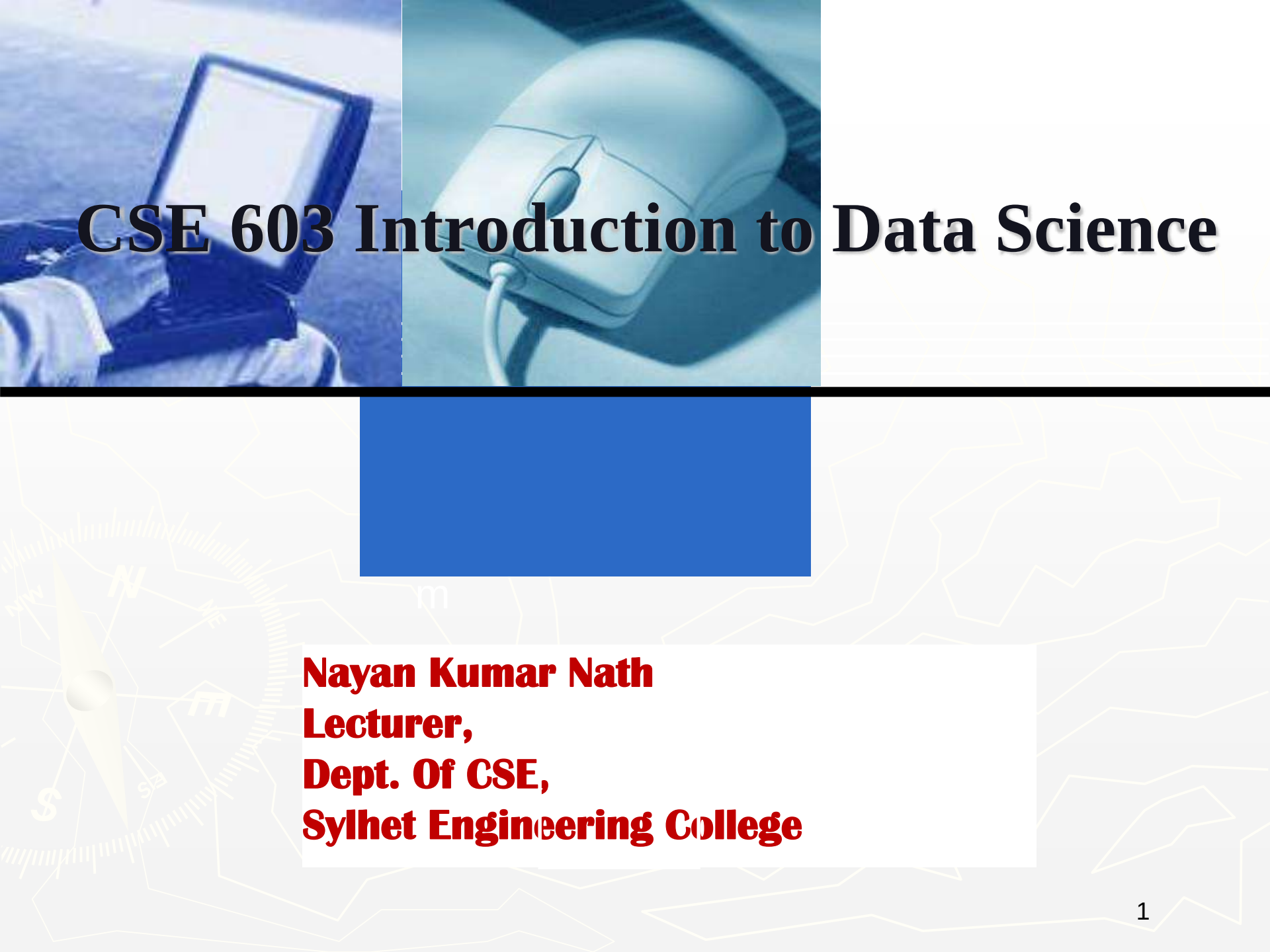




CSE 603 Introduction to Data Science

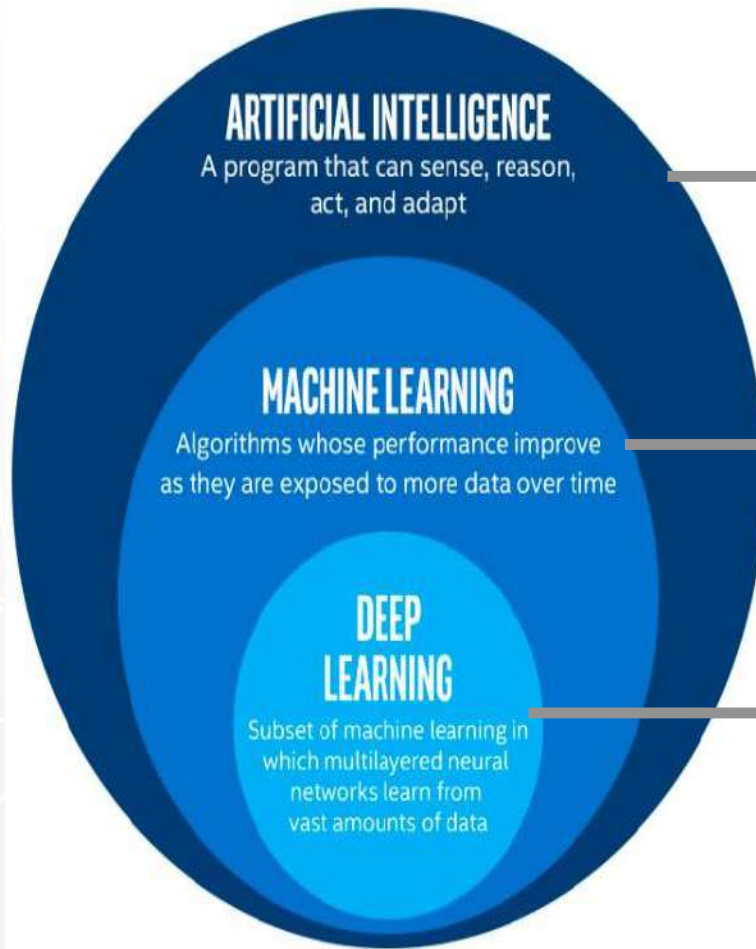
Nayan Kumar Nath
Lecturer,
Dept. Of CSE,
Sylhet Engineering College

What is machine learning?



Blue faces seem to be important...

What is machine learning really?



1950's: creation of first “intelligent” algorithms and programs. Alan Turing published “Computer Machinery and Intelligence” which proposed a test of machine intelligence called The Imitation Game

1980's: statistical models and algorithms that can learn from data

2010's: statistical models and algorithms inspired by neurones that can learn from data

Main Components of a Machine Learning System

▪ A Machine Learning system is an integrated pipeline that transforms raw data into intelligent predictions or decisions. It consists of multiple interconnected components working together systematically.

1. Problem Definition

Before building an ML system, the problem must be clearly defined.

- What type of problem? (Classification, Regression, Clustering, etc.)
- What is the objective?
- What is the expected output?
- What constraints exist (time, cost, interpretability)?

Example:

- Spam detection → Classification
- House price prediction → Regression
- Customer segmentation → Clustering

Clear problem definition ensures correct model and metric selection.

2. Data Collection

Data is the backbone of ML systems.

Types of Data:

- Structured (tables, CSV files, databases)
- Semi-structured (JSON, XML)
- Unstructured (text, images, audio, video)

Sources:

Sensors and IoT devices, Company databases, Public datasets, APIs, Web scraping, Logs

Main Components of a Machine Learning System

3. Data Preprocessing

Raw data is often incomplete, inconsistent, and noisy.

Key steps:

1.Data Cleaning

- Handling missing values
- Removing duplicates
- Outlier detection

2.Data Transformation

Normalization,Standardization,Encoding categorical variables (One-hot encoding, Label encoding)

3.Data Integration

- Combining multiple data sources

4.Data Splitting

Training set,Validation set,Test set

Without proper preprocessing, even advanced models fail.

4. Exploratory Data Analysis (EDA)

EDA helps understand data patterns before modeling.

- Statistical summaries (mean, variance, correlation)
- Data visualization
- Distribution analysis
- Identifying skewness and imbalance

EDA helps in:Selecting features,Detecting anomalies,Understanding relationships

Main Components of a Machine Learning System

5. Feature Engineering

Feature engineering significantly impacts model performance.

Includes:

- Feature selection (removing irrelevant features)
- Feature extraction (e.g., PCA)
- Creating new features
- Dimensionality reduction

Well-engineered features often outperform complex models with poor features.

6. Model Selection

Choosing the right algorithm depends on:

- Problem type
- Dataset size
- Interpretability requirement
- Computational resources

Common categories:

1. Supervised Learning

Linear Regression, Logistic Regression, Decision Trees, Support Vector Machines, Neural Networks

2. Unsupervised Learning

K-Means Clustering, Hierarchical Clustering, PCA

3. Reinforcement Learning

Q-learning, Policy-based methods

Main Components of a Machine Learning System

7. Model Training

Training involves:

- Feeding training data
- Calculating predictions
- Computing loss function
- Updating model parameters

Optimization techniques:

- Gradient Descent
- Stochastic Gradient Descent (SGD)
- Adam Optimizer

The goal is to minimize error and generalize well.

8. Model Evaluation

Evaluation determines model effectiveness.

Common metrics:

For Classification:

Accuracy, Precision, Recall, F1-score, ROC-AUC

For Regression:

Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R^2 score

Cross-validation improves reliability of evaluation.

9. Hyperparameter Tuning

Hyperparameters are not learned automatically.

Examples: Learning rate, Number of hidden layers, Tree depth, Number of clusters

Techniques: Grid Search, Random Search, Bayesian Optimization

Main Components of a Machine Learning System

10. Model Deployment

After validation, the model is deployed into production.

Deployment methods:

- Web APIs
- Cloud platforms
- Edge devices
- Mobile applications
- Tools commonly used:
- Flask / FastAPI
- Docker
- AWS / Azure / GCP

11. Monitoring and Maintenance

ML systems require continuous monitoring.

Key issues:

- Data drift
- Concept drift
- Performance degradation
- Bias issues
- Solutions:
- Retraining
- Updating features
- Monitoring dashboards

Main Components of a Machine Learning System

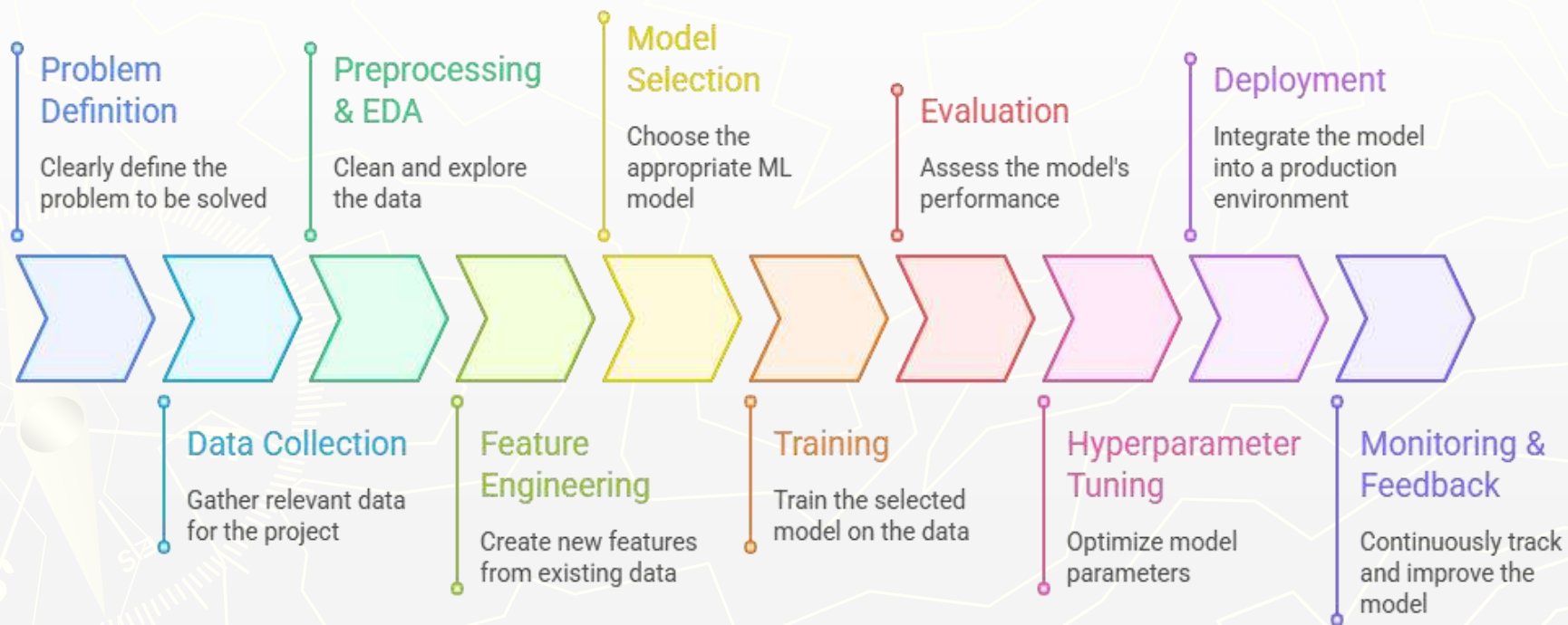
12. Feedback Loop

A mature ML system includes a feedback mechanism:

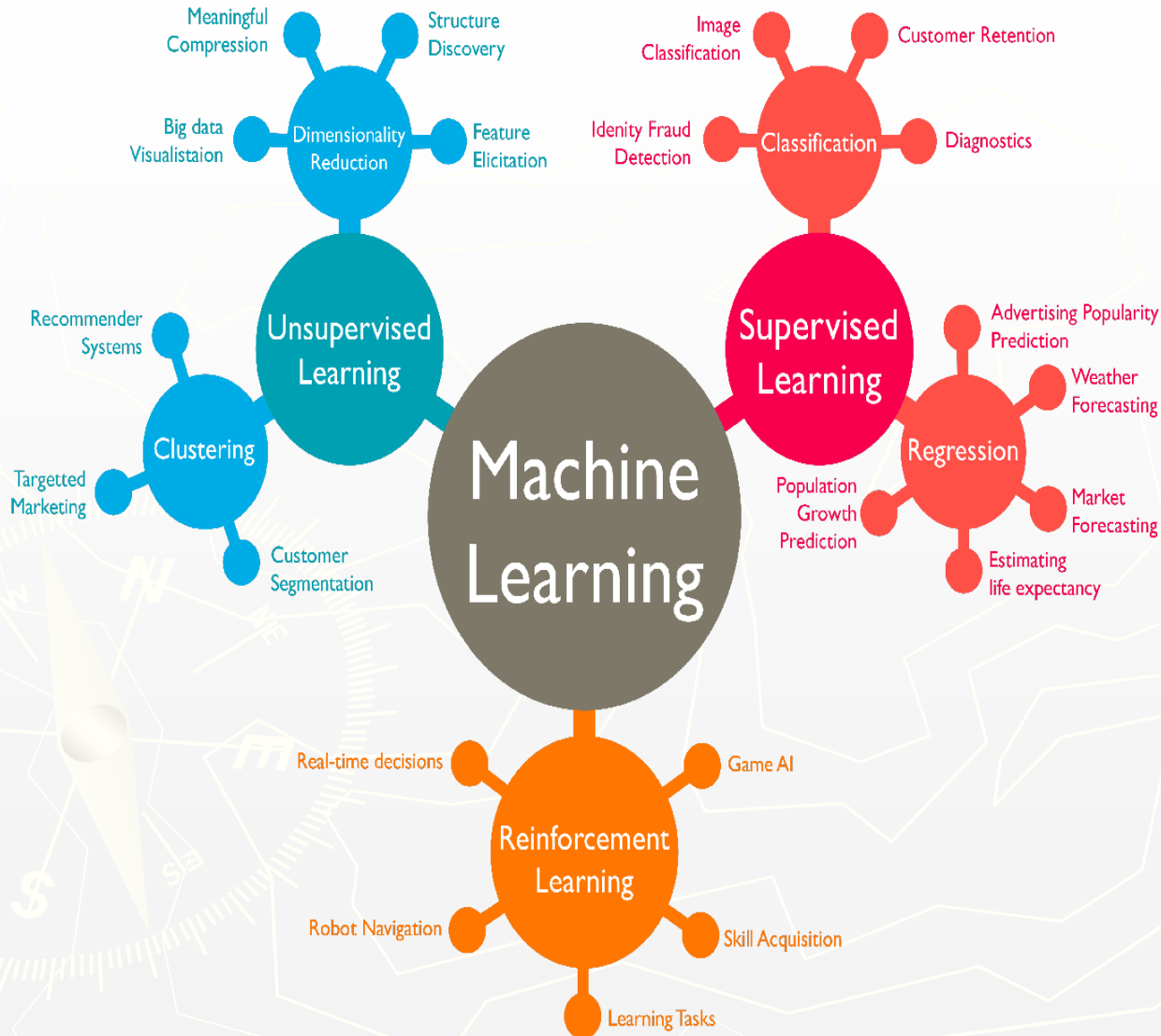
Prediction → User Interaction → New Data → Retraining → Improved Model

This creates a continuous learning cycle.

Building a Machine Learning System: A Step-by-Step Journey



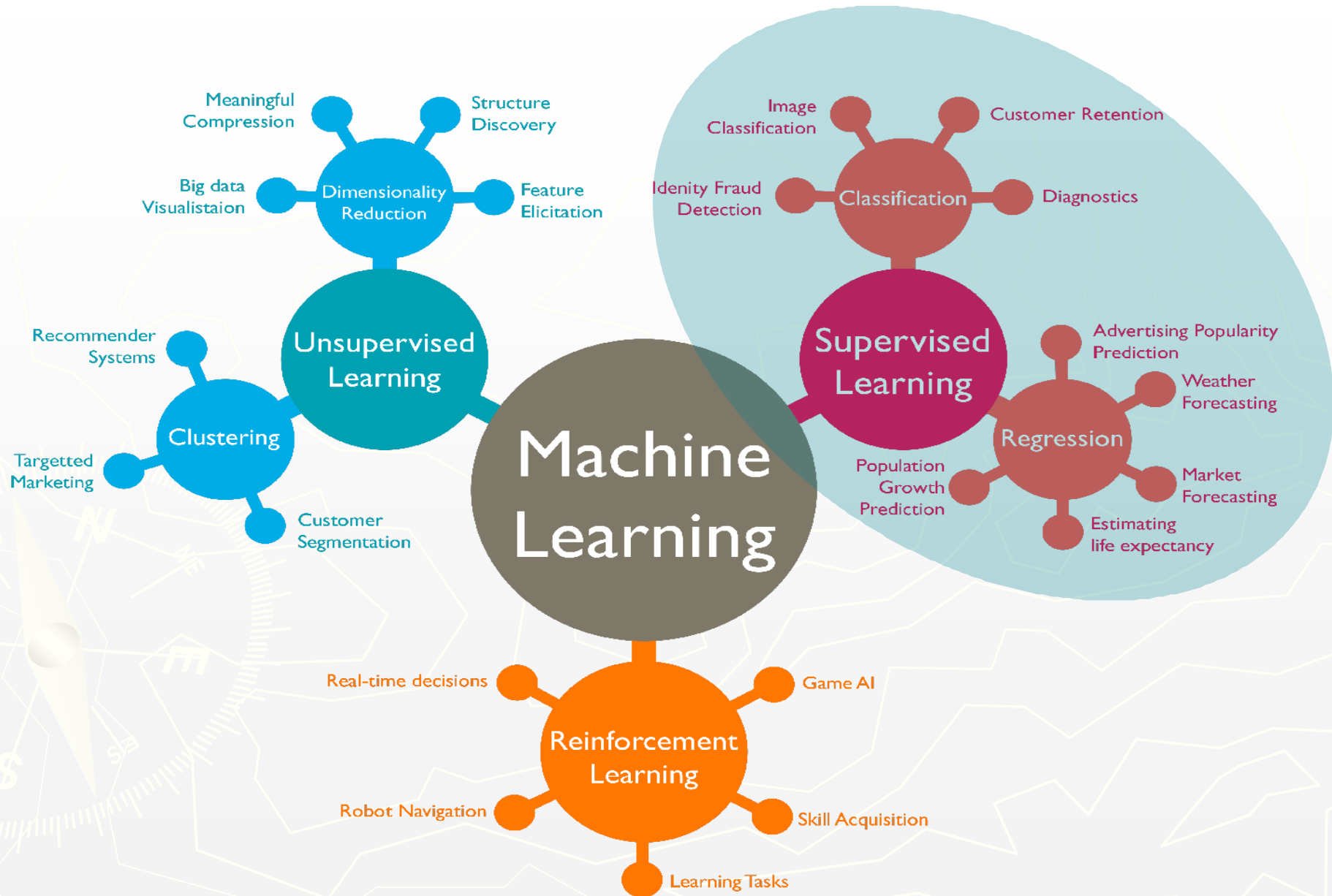
Machine Learning Branches



3 Main Branches:

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning

Machine Learning Branches



Machine Learning Branches: Supervised Learning

In **supervised learning** the training data consists of **input/output pairs** and we train a function to map the inputs to the outputs.

Values, Vectors,
Words, Images etc.

Input

Classification

Regression

Categorical Variable
eg. Image Classification

A, B, C
Dogs/Cats

Continuous Variables
eg. Market Prediction

Prize/Cost,
Weight, Lifetime

Supervised Learning: Classification

Classification: Classification algorithms are used to solve the classification problems in which the output variable is categorical, such as **"Yes" or No, Male or Female, Red or Blue, etc.** The classification algorithms predict the categories present in the dataset. Assign categorical labels from a fixed set of labels to data samples. Some popular classification algorithms are given below:

Random Forest Algorithm, Decision Tree Algorithm, Logistic Regression Algorithm, Support Vector Machine Algorithm, Naive Bayes Algorithm, Gradient Boosting Machines (GBM), AdaBoost (Adaptive Boosting)

Input
Data:

Output/Label:



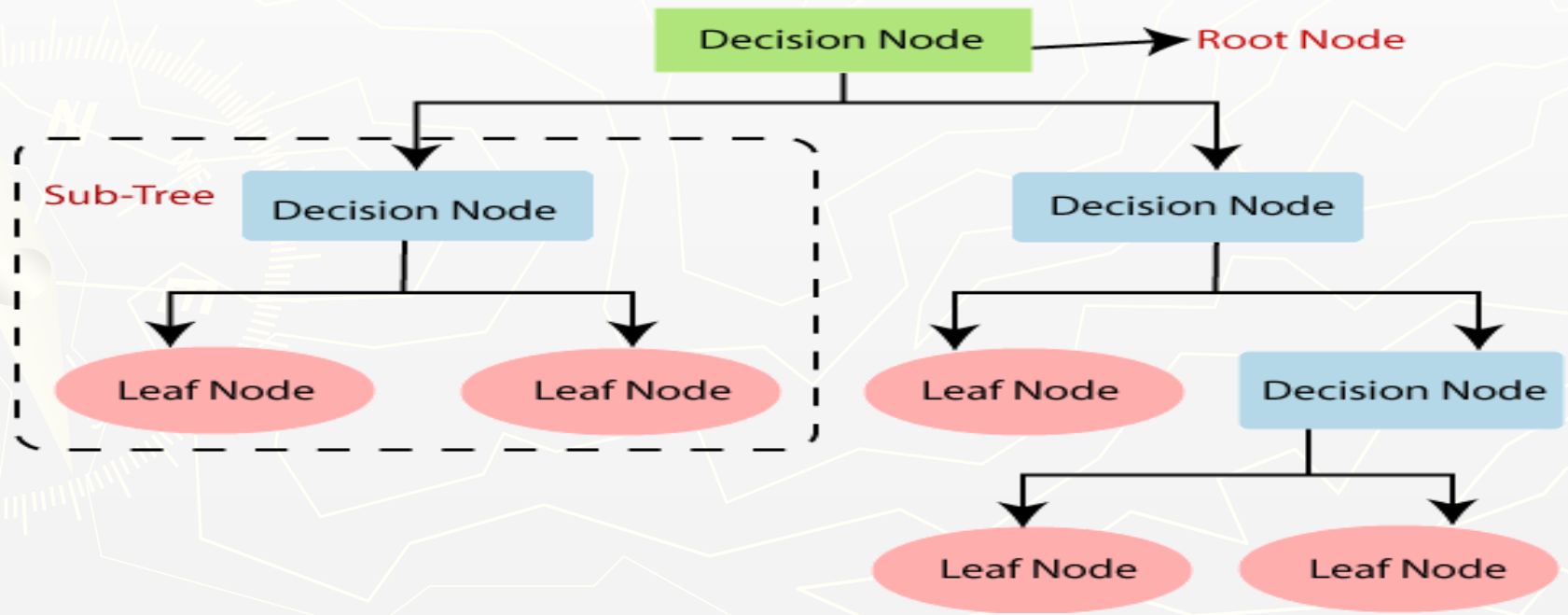
"Normal
Bike"



"Broken
Bike"

Decision Tree Classification Algorithm

- Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome**.
- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node**
- It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.**
- In order to build a tree, we use the **CART algorithm**, which stands for **Classification and Regression Tree algorithm**.
- A decision tree simply asks a question, and based on the answer (Yes/No), it further splits the tree into subtrees.



How does the Decision Tree algorithm Work?

- In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.
- For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

Step-1: Begin the tree with the root node, says S, which contains the complete dataset.

Step-2: Find the best attribute in the dataset using **Attribute Selection Measure (ASM)**.

Step-3: Divide the S into subsets that contains possible values for the best attributes.

Step-4: Generate the decision tree node, which contains the best attribute.

Step-5: Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Classification and regression glossary

Classification and regression involve many specialized terms. You've come across some of them in earlier examples, and you'll see more of them in future chapters. They have precise, machine learning-specific definitions, and you should be familiar with them:

- **Sample or input** — One data point that goes into your model.
- **Prediction or output** — What comes out of your model.
- **Target** — The truth. What your model should ideally have predicted, according to an external source of data.
- **Prediction error or loss value** — A measure of the distance between your model's prediction and the target.
- **Classes** — A set of possible labels to choose from in a classification problem. For example, when classifying cat and dog pictures, "dog" and "cat" are the two classes.
- **Label** — A specific instance of a class annotation in a classification problem. For instance, if picture #1234 is annotated as containing the class "dog," then "dog" is a label of picture #1234.
- **Ground-truth or annotations** — All targets for a dataset, typically collected by humans.
- **Binary classification** — A classification task where each input sample should be categorized into two exclusive categories.
- **Categorical classification or multiclass classification** — A classification task where each input sample should be categorized into more than two categories: for instance, classifying handwritten digits.
- **Multilabel classification** — A classification task where each input sample can be assigned multiple labels. For instance, a given image may contain both a cat and a dog and should be annotated with both the "cat" label and the "dog" label. The number of labels per image is usually variable.
- **Scalar regression** — A task where the target is a continuous scalar value. Predicting house prices is a good example: the different target prices form a continuous space.
- **Vector regression** — A task where the target is a set of continuous values: for example, a continuous vector. If you're doing regression against multiple values (such as the coordinates of a bounding box in an image), then you're doing vector regression.
- **Mini-batch or just batch** — A small set of samples (typically between 8 and 128) that are processed simultaneously by the model. The number of samples is often a power of 2, to facilitate memory allocation on GPU. When training, a mini-batch is used to compute a single gradient-descent update applied to the weights of the model.

How does the Decision Tree algorithm Work?

- **Types of Decision Tree algorithm**

- **ID3 (Iterative Dichotomiser 3):** It is a simple decision tree algorithm that uses the entropy measure to choose the best attribute for splitting the data.

- **C4.5:** It is an extension of the ID3 algorithm that can handle both categorical and numerical data, and can handle missing values in the dataset.

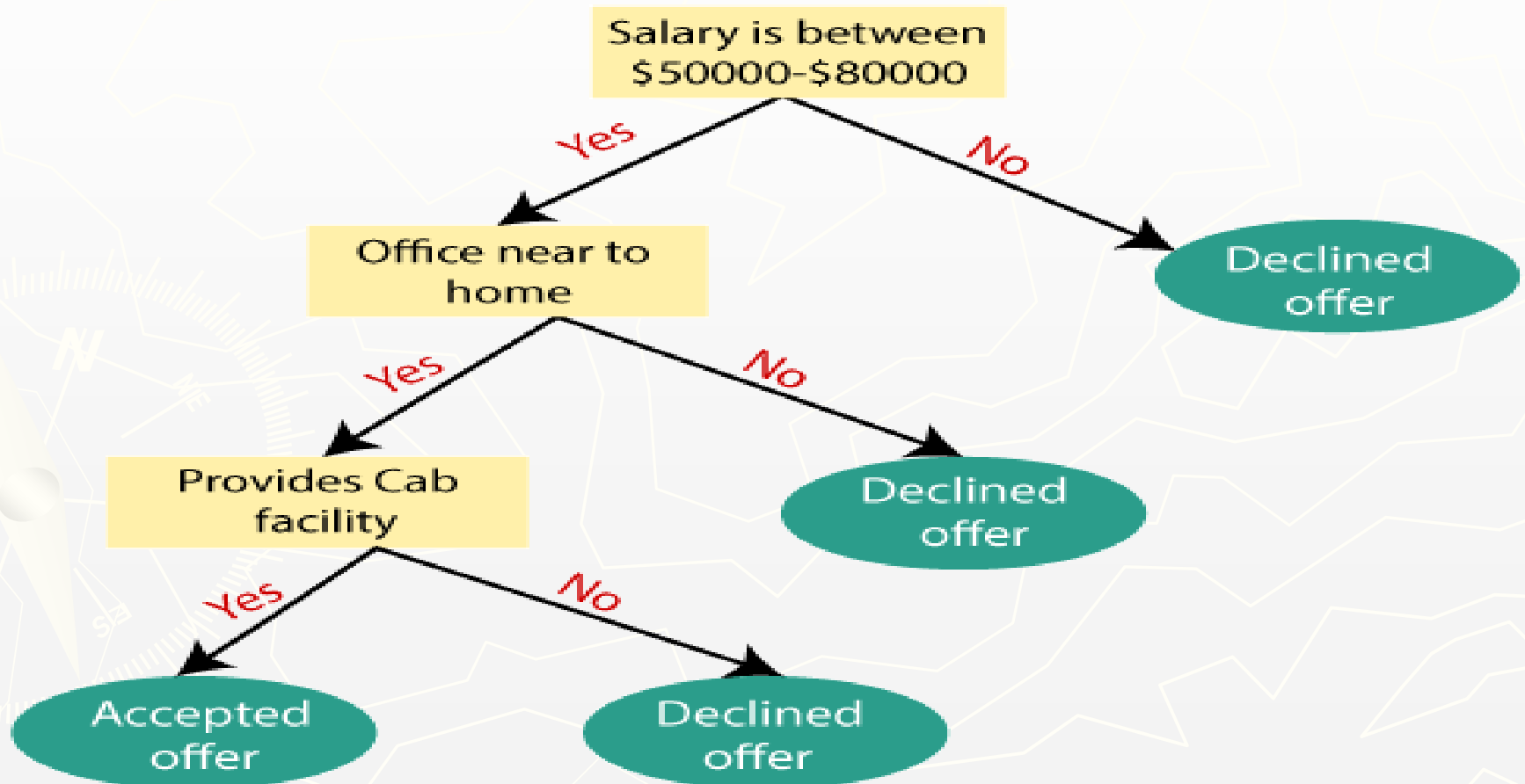
- **CART (Classification and Regression Tree):** It is a decision tree algorithm that can be used for both classification and regression tasks. It uses the Gini index as a measure of impurity to choose the best attribute for splitting the data.

- **CHAID (Chi-squared Automatic Interaction Detection):** It is a decision tree algorithm that uses the chi-squared statistic to find the best attribute for splitting the data.

- **MARS (Multivariate Adaptive Regression Splines):** It is a decision tree algorithm that uses a combination of linear regression and decision trees to model complex relationships between variables.

Decision Tree algorithm: Example

Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not. So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM). The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into one decision node (Cab facility) and one leaf node. Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer). Consider the below diagram:



Attribute Selection Measures(ASM):

- While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes. So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM**. The goal of ASM is to identify the attribute that will create the most homogeneous subsets of data after the split, thereby maximizing the information gain. This process is repeated on each derived subset in a recursive manner called **recursive partitioning**
- By this measurement, we can easily select the best attribute for the nodes of the tree. There are two popular techniques for ASM, which are:
 - **Information Gain**
 - **Gini Impurity or Index**

1. Information Gain:

Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute. It calculates how much information a feature provides us about a class. According to the value of information gain, we split the node and build the decision tree. A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first. It can be calculated using the below formula:

Information Gain = Entropy(S) - [(Weighted Avg) * Entropy(each feature)]

The information gain of an attribute A, with respect to a dataset S, is calculated as follows:

$$IG(S, A) = Entropy(S) - \sum \frac{|S_v|}{|S|} Entropy(S_v)$$

Where:

S = Total dataset, **A** = Attribute, **S_v** = Subset of S for value v, $|S_v|/|S|$ = Weight of subset

Attribute Selection Measures(ASM):

Entropy: Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:

$$\text{Entropy}(s) = -P(\text{yes})\log_2 P(\text{yes}) - P(\text{no})\log_2 P(\text{no})$$

Interpretation:

- **Entropy = 0:** The dataset is pure (only one class exists, e.g., all "Yes" or all "No").
- **Entropy > 0:** There is some impurity in the dataset.
- **Entropy = 1:** Maximum impurity (for a binary classification, it's a 50-50 split between the classes)

Where,

- **S= Total number of samples, P(yes)= probability of yes, P(no)= probability of no**
- **Important points related to Entropy:**

2. Gini Impurity or Index

Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm. An attribute with the low Gini index should be preferred as compared to the high Gini index. It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits. Gini index can be calculated using the below formula:

$$\text{Gini Index} = 1 - \sum_j P_j^2$$

p_j is the proportion of elements in the set that belongs to the j th category.

ID3 Algorithm

The ID3 (Iterative Dichotomiser 3) algorithm is one of the earliest and most widely used algorithms to create Decision Trees from a given dataset. It uses the concept of entropy and information gain to select the best attribute for splitting the data at each node.

Understanding the ID3 Algorithm:

The ID3 algorithm uses the concept of entropy and information gain to construct a decision tree. Entropy measures the amount of uncertainty or randomness in a dataset, while information gain quantifies the reduction in entropy achieved by splitting the data on a specific attribute. The attribute with the highest information gain is selected as the decision node for the tree.

Steps to Create a Decision Tree using the ID3 Algorithm:

Step 1: Data Preprocessing: Clean and preprocess the data. Handle missing values and convert categorical variables into numerical representations if needed.

Step 2: Selecting the Root Node: Calculate the entropy of the target variable (class labels) based on the dataset. The formula for entropy is:

$$\text{Entropy}(S) = -\sum (p_i * \log_e p_i) = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, \quad -\infty < x < \infty$$

where p_i is the proportion of instances belonging to class i .

Step 3: Calculating Information Gain: For each attribute in the dataset, calculate the information gain when the dataset is split on that attribute. The formula for information gain is:

$$\text{Information Gain}(S, A) = \text{Entropy}(S) - \sum \left(\frac{|S_v|}{|S|} * \text{Entropy}(S_v) \right)$$

where S_v is the subset of instances for each possible value of attribute A , and $|S_v|$ is the number of instances in that subset.

Step 4: Selecting the Best Attribute: Choose the attribute with the highest information gain as the decision node for the tree.

Step 5: Splitting the Dataset: Split the dataset based on the values of the selected attribute.

Step 6: Repeat the Process: Recursively repeat steps 2 to 5 for each subset until a stopping criterion is met (e.g., the tree depth reaches a maximum limit or all instances in a subset belong to the same class).

ID3 Algorithm

compute the entropy for data-set

for every attribute/feature:

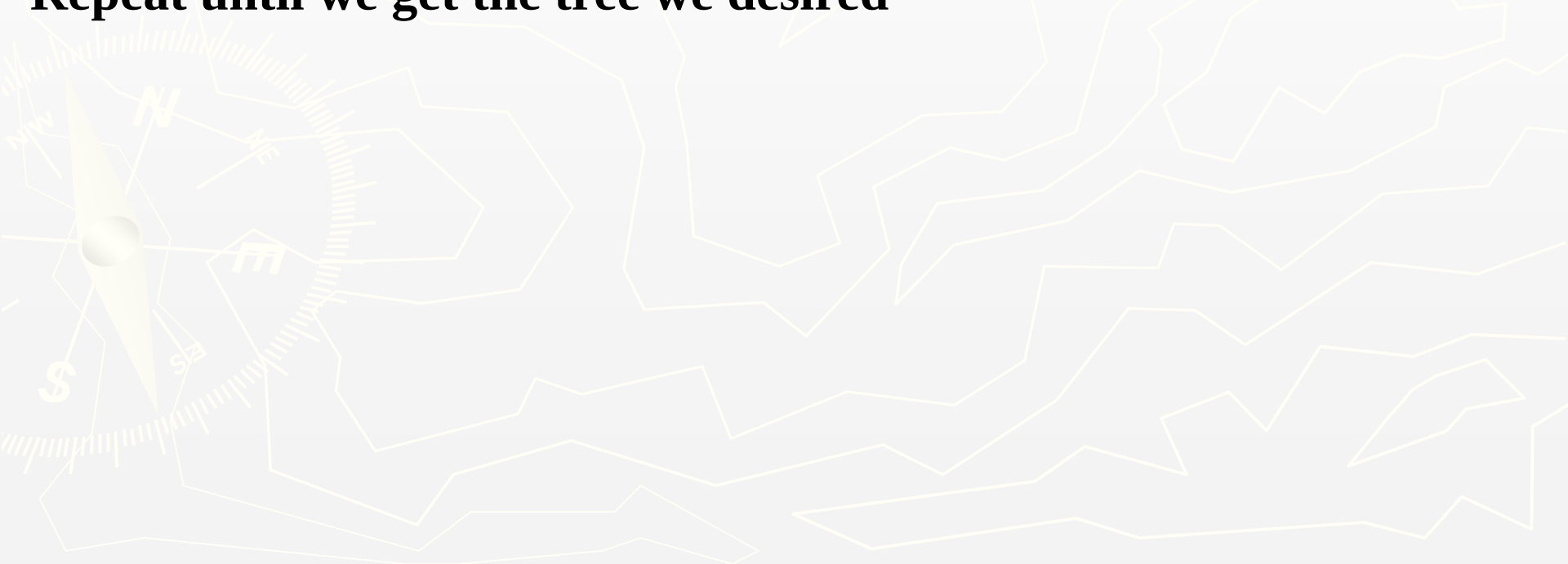
1.calculate entropy for all categorical values

2.take average information entropy for the current attribute

3.calculate gain for the current attribute

pick the highest gain attribute.

Repeat until we get the tree we desired



Example

- What is the entropy of this collection of training examples with respect to target function classification.
- What is the information gain of a1 and a2 relative to these training examples?
- Draw a decision tree for the given dataset.

Instance	Classification	a1	a2
1.	+	T	T
2.	+	T	T
3.	-	T	F
4.	+	F	F
5.	-	F	T
6.	-	F	T

Attribute(a1): values(T,F)

- $S=[3+,3-]$, Entropy(S)=1.0 [Equal no of -ve,+ve $E=1$ / If only +ve/-ve , $E=0$]
- $S_T = [2+,1-]$, Entropy(S_T)= $-p_+ \log_2(p_+) - p_- \log_2(p_-) = -\frac{2}{3} \log_2\left(\frac{2}{3}\right) - \frac{1}{3} \log_2\left(\frac{1}{3}\right)$
 ≈ 0.9183
- $S_F = [1+,2-]$, Entropy(S_F)= $-p_+ \log_2(p_+) - p_- \log_2(p_-) = -\frac{1}{3} \log_2\left(\frac{1}{3}\right) - \frac{2}{3} \log_2\left(\frac{2}{3}\right)$
 ≈ 0.9183

ID3 Algorithm: Examples

$$\square \text{Gain}(S, a) = \text{Entropy}(S) - \sum_{v \in \text{values}(S)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

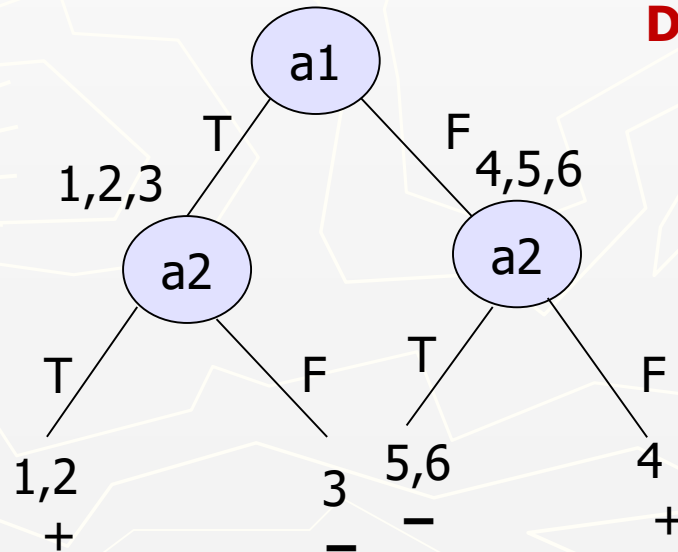
$$\begin{aligned} \square \text{Gain}(S, a1) &= \text{Entropy}(S) - \frac{3}{6} * \text{Entropy}(S_T) - \frac{3}{6} * \text{Entropy}(S_F) \\ &= 1 - \frac{3}{6} * 0.9183 - \frac{3}{6} * 0.9183 = 0.087 \rightarrow \text{maximum gain} \end{aligned}$$

Attribute(a2): values(a2) = (T,F)

- $S = [3+, 3-]$, Entropy(S)=1.0 [Equal no of -ve,+ve E=1 / If only +ve/-ve ,E=0]
- $S_T = [2+, 2-]$, Entropy(S_T)= 1
- $S_F = [1+, 1-]$, Entropy(S_F)= 1

$$\begin{aligned} \square \text{Gain}(S, a2) &= \text{Entropy}(S) - \frac{4}{6} * \text{Entropy}(S_T) - \frac{2}{6} * \text{Entropy}(S_F) \\ &= 1 - \frac{4}{6} * 1 - \frac{2}{6} * 1 = 0 \rightarrow \text{minimum gain} \end{aligned}$$

Decision Tree



Example

▪ Example: Play Tennis Dataset (14 Records)

Outlook	Temperature	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Total:
Yes = 9
No = 5

Example

◆ Step 1: Calculate Entropy of Entire Dataset

$$Entropy(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14}$$

$$Entropy(S) = 0.94$$

◆ Step 2: Calculate Information Gain for Each Attribute

1 Information Gain for Outlook

Outlook = Sunny (5)

•Yes = 2

•No = 3

Entropy = 0.971

$$p_{\text{Yes}} = \frac{2}{5} = 0.4 \quad p_{\text{No}} = \frac{3}{5} = 0.6$$

$$Entropy(S) = -(0.4 \log_2 0.4 + 0.6 \log_2 0.6)$$

$$Entropy(S) = 0.971$$

Outlook = Overcast (4)

•Yes = 4

•No = 0

Entropy = 0

Outlook = Rain (5)

•Yes = 3

•No = 2

Entropy = 0.971

Weighted Entropy:

$$\begin{aligned} &= \frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) \\ &= 0.693 \end{aligned}$$

Example

Information Gain: $IG(Outlook) = 0.94 - 0.693 = 0.247$

2 Information Gain for Humidity

After calculation:

$$IG(Humidity) = 0.151$$

3 Information Gain for Wind

$$IG(Wind) = 0.048$$

4 Information Gain for Temperature

$$IG(Temperature) = 0.029$$

◆ Step 3: Select Best Attribute

Highest IG = **Outlook (0.247)**

So, **Root Node = Outlook**

□ Key Idea of ID3

At **every node**:

☞ Choose the attribute with **highest Information Gain for that subset only, Not for the whole dataset.**

So after splitting on **Outlook**, we treat:

- **Sunny data**

- **Rain data**

- **Overcast data** as **separate mini-datasets.**

Example

Step 4: Split and Repeat

❑ Case 1: Outlook = Overcast

All Yes → Leaf Node = Yes

❑ Case 2 — Outlook = Sunny subset

Humidity	Wind	Play
High	Weak	No
High	Strong	No
High	Weak	No
Normal	Weak	Yes
Normal	Strong	Yes

Counts:

•Yes = 2, No = 3, **Entropy = 0.971**

Try each attribute

◆ Humidity

Split: High → 3 No → 3, Entropy = 0

Normal → 2 Yes → 2, Entropy = 0, Weighted entropy = 0

Gain: $0.971 - 0 = 0.971$, Perfect split

Example

◆ Wind

Split:

Weak → 2 No, 1 Yes

Strong → 1 No, 1 Yes

Entropy still mixed → small gain

◆ Temperature

Also mixed → small gain

✓ Best = Humidity

So ID3 chooses **Humidity**

Case 3 — Outlook = Rain subset

Wind	Play
Weak	Yes
Weak	Yes
Weak	Yes
Strong	No
Strong	No

Counts:

•Yes = 3

•No = 2

Entropy = 0.971

Example

Try each attribute

◆ Wind

Split:

Weak → 3 Yes → Entropy 0

Strong → 2 No → Entropy 0

Gain = 0.971 (perfect)

◆ Humidity / Temperature

Mixed → lower gain

✓ **Best = Wind**

So ID3 chooses **Wind**

ID3 selects the attribute with maximum Information Gain at each node using only that subset of data.

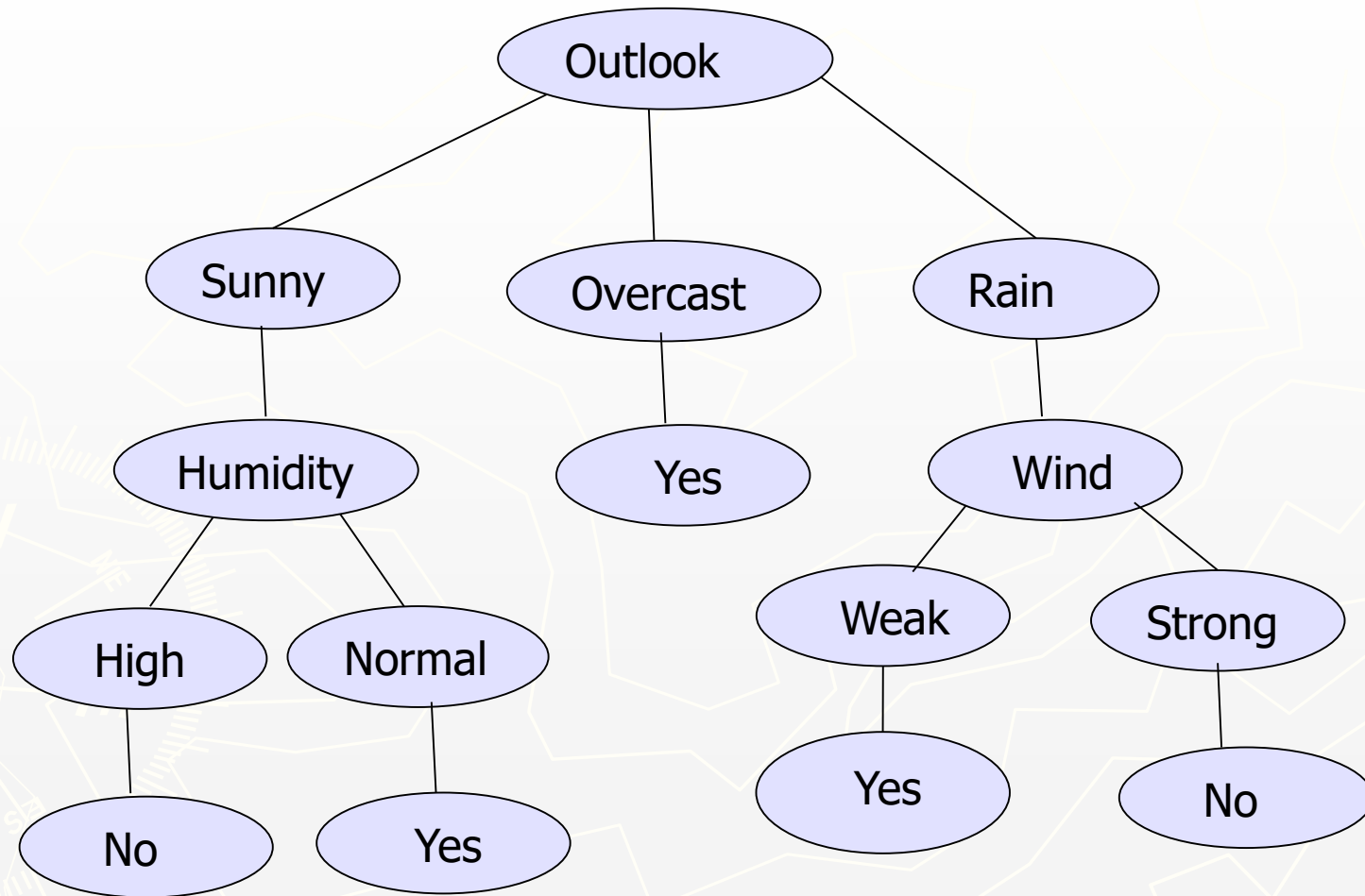
Therefore:

Sunny → Humidity gives highest gain

Rain → Wind gives highest gain

Example

◆ Final Decision Tree



Example

◆ Final Classification Rules

- If Outlook = Overcast $\rightarrow$ Play = Yes
- If Outlook = Sunny AND Humidity = High $\rightarrow$ No
- If Outlook = Sunny AND Humidity = Normal $\rightarrow$ Yes
- If Outlook = Rain AND Wind = Weak $\rightarrow$ Yes
- If Outlook = Rain AND Wind = Strong $\rightarrow$ No

CART (Classification And Regression Tree)

To break a dataset into smaller, meaningful groups, CART (Classification and Regression Tree) is used which builds a decision tree that predicts outcomes for both classification and regression tasks. It works by splitting data based on rules that reduce error at each step. CART (Classification and Regression Trees) constructs a decision tree by recursively splitting the dataset based on the feature and threshold that produce the highest reduction in Gini Impurity (for classification) or Mean Squared Error (MSE) or Residual Sum of Squares (RSS)(for regression)

CART for Classification

CART is used for classification tasks when the output variable is categorical.

How it Works

- Recursively splits the dataset to increase class purity at each step.
- Uses Gini Impurity to select the best feature and threshold for splitting.
- Continues splitting until a stopping rule is met such as maximum tree depth or minimum required samples.
- CART chooses splits that produce the purest possible child nodes.

CART (Classification And Regression Tree)

To break a dataset into smaller, meaningful groups, CART (Classification and Regression Tree) is used which builds a decision tree that predicts outcomes for both classification and regression tasks. It works by splitting data based on rules that reduce error at each step. CART (Classification and Regression Trees) constructs a decision tree by recursively splitting the dataset based on the feature and threshold that produce the highest reduction in **Gini Impurity** (for classification) or **Mean Squared Error (MSE)** or **Residual Sum of Squares (RSS)**(for regression)

CART for Classification

CART is used for classification tasks when the output variable is categorical.

How it Works

- Recursively splits the dataset to increase class purity at each step.
- Uses Gini Impurity to select the best feature and threshold for splitting.
- Continues splitting until a stopping rule is met such as maximum tree depth or minimum required samples.
- CART chooses splits that produce the purest possible child nodes.

CART (Classification And Regression Tree)

Suppose we want to decide whether to play tennis based on Weather and Wind.

Day	Weather	Wind	PlayTennis
1	Sunny	Weak	Yes
2	Sunny	Strong	No
3	Rain	Weak	Yes
4	Rain	Strong	No

Target = **PlayTennis (Yes/No)**

Features = **Weather, Wind**

Step 1 — Calculate Root Node

Impurity (Gini)

CART uses **Gini impurity**:

$$Gini(S) = 1 - \sum p_i^2$$

Where p_i = fraction of each class (Yes/No).

Root counts:

- Yes = 2
- No = 2
- Total = 4

CART (Classification And Regression Tree)

$$p_{Yes} = 2/4 = 0.5, \quad p_{No} = 2/4 = 0.5$$

$$Gini_{root} = 1 - (0.5^2 + 0.5^2) = 1 - 0.25 - 0.25 = 0.5$$

Interpretation: maximum impurity for 2 classes, data is evenly mixed.

Step 2 — Try Splits for Each Feature

CART does **binary splits**. Let's compute **weighted Gini** for each split.

2A — Split on Weather

Values: Sunny, Rain → binary split: {Sunny} vs {Rain}

Sunny group:

Day	Weather	Wind	Play
1	Sunny	Weak	Yes
2	Sunny	Strong	No

Yes = 1, No = 1

$$Gini = 1 - (0.5^2 + 0.5^2) = 0.5$$

CART (Classification And Regression Tree)

Rain group:

Day	Weather	Wind	Play
3	Rain	Weak	Yes
4	Rain	Strong	No

Yes = 1, No = 1

$$Gini = 0.5$$

Weighted Gini for Weather: $(2/4) * 0.5 + (2/4) * 0.5 = 0.5$

✗ No improvement. Weather alone doesn't reduce impurity.

2B — Split on Wind

Values: Weak, Strong → binary split: {Weak} vs {Strong}

Weak group:

Day	Weather	Wind	Play
1	Sunny	Weak	Yes
3	Rain	Weak	Yes

Yes = 2, No = 0

$$Gini = 1 - 1^2 = 0$$

CART (Classification And Regression Tree)

Strong group

Day	Weather	Wind	Play
2	Sunny	Strong	No
4	Rain	Strong	No

Yes = 0, No = 2

$$Gini = 0$$

Weighted Gini for Wind:

$$(2/4) * 0 + (2/4) * 0 = 0$$

✓ **Perfect split!**

Step 3 — Choose the Best Split

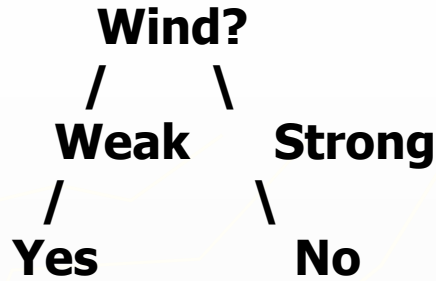
Compare weighted Gini:

Feature	Weighted Gini
Weather	0.5
Wind	0

☞ Best split = **Wind**

CART (Classification And Regression Tree)

Step 4 — Split the Root Node on Wind



✓ Both leaves are pure. Tree is complete.

Step 5 — Rules from the Tree

- 1.If Wind = Weak → PlayTennis = Yes
- 2.If Wind = Strong → PlayTennis = No

Key Points of CART

1.Gini Impurity measures “mixedness”

1. 0 = pure, 0.5 = fully mixed

2.Binary splits only → every node splits into 2 branches

3.Choose feature with **lowest weighted Gini** after split

4.Repeat recursively until leaves are pure or stopping condition

Step 6 — Optional Notes

- For bigger datasets, CART computes Gini for all possible splits, then picks the best.
- Can handle numeric features (splits like Temperature ≤ 75)
- Can also build regression trees (use variance instead of Gini)

Decision Tree algorithm

Advantages of the Decision Tree

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the **Random Forest algorithm**.
- For more class labels, the computational complexity of the decision tree may increase.

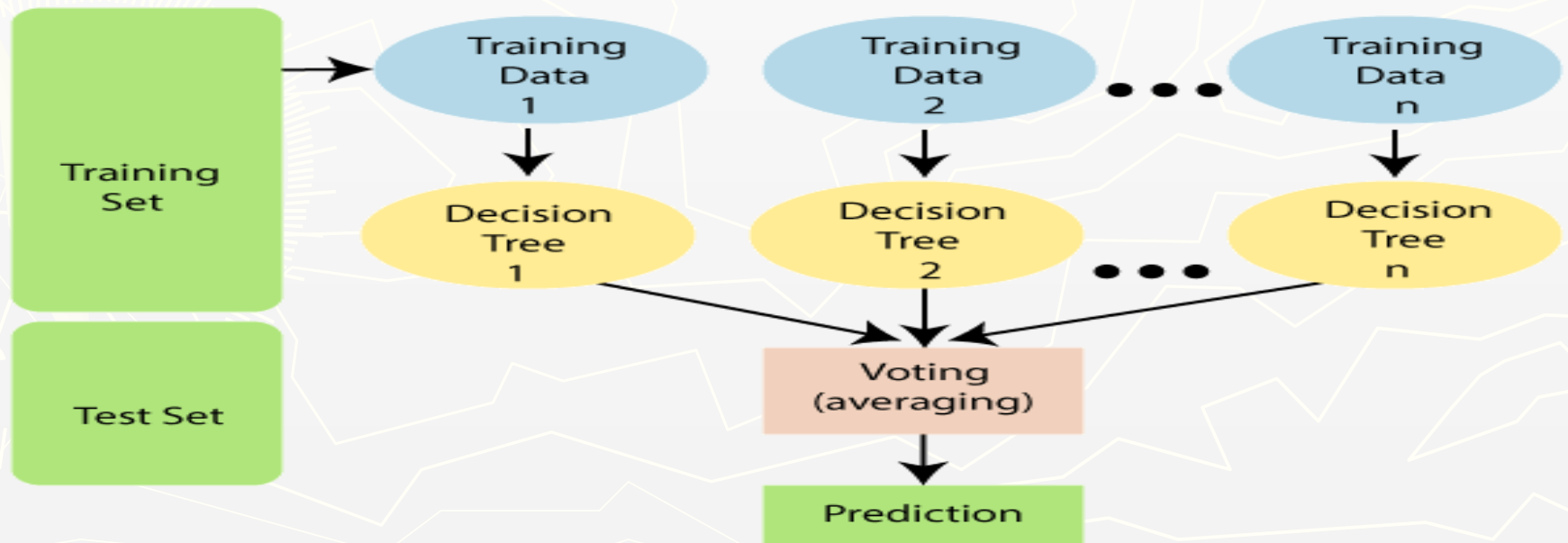
Random Forest Algorithm

▪ **Random Forest** is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset." Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output. It can be used for both Classification and Regression problems in ML. It is based on the concept of **ensemble learning**, which is a process of combining multiple classifiers to solve a complex problem and to improve the performance of the model.

▪ The individual decision trees tend to overfit to the training data but random forest can mitigate that issue by averaging the prediction results from different trees. The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

Therefore, below are two assumptions for a better Random forest classifier:

- There should be some actual values in the feature variable of the dataset so that the classifier can predict accurate results rather than a guessed result.
- The predictions from each tree must have very low correlations.



Random Forest Algorithm

Underfitting: Underfitting means that the model is too simple and does not cover all real patterns in the data. Underfitting happens when the model fails to learn important patterns. It performs poorly on both **training** and **testing** data.

Intuition

Imagine fitting a straight line to curved data.

Causes: Underfitting happens due to:

- Model is too simple
- Very high regularization
- Features are weak or missing
- Not enough training
- High bias

***Bias:** systematic error from wrong assumptions. It is like assuming all birds can only be small and fly, so the model fails to recognize big birds like ostriches or penguins that can't fly and get biased with predictions.*

Signs

- High training error
- High validation/test error
- Learning curves converge early at poor performance

Fixes

- Increase model complexity (deeper nets, more trees)
- Add better features
- Reduce regularization
- Train longer (Train for more epochs)
- Scale features properly

Random Forest Algorithm

Overfitting: Overfitting means that the model learns not just the underlying pattern, but also noise or random quirks in the training data. model memorizes training data. Overfitting happens when the model learns too much from the **training data**, including noise and outliers. It performs very well on **training data** but poorly on **test data**.

Intuition

Imagine a very wiggly curve passing through every training point exactly.

Causes:

- Model too complex
- Too many features
- Very little data
- No regularization, Data leakage
- High variance

Variance: *sensitivity to small changes in data. Error that happens when a machine learning model learns too much from the data, including random noise.*

Signs

- High training error
- High validation/test error
- Learning curves converge early at poor performance

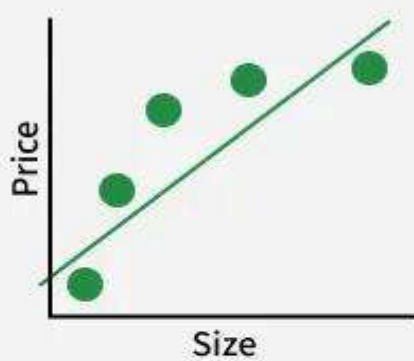
Fixes

- More data
- Regularization (L1/L2, dropout)
- Simpler model
- Use early stopping
- Cross-validation
- Techniques like bagging (e.g., Random Forest)

Random Forest Algorithm

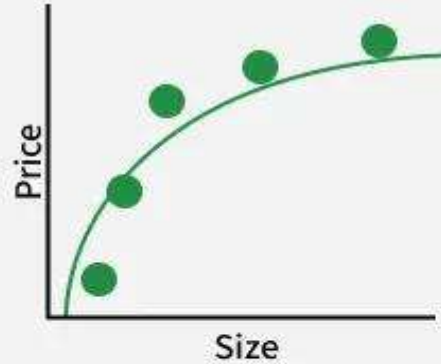
Underfitting = High Bias + Low Variance

Overfitting = Low Bias + High Variance



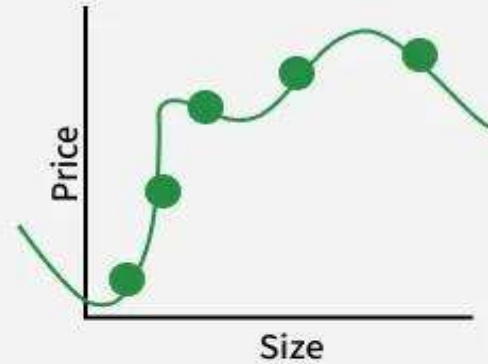
$$\theta_0 + \theta_1 x$$

High Bias
(Underfitting)



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

Low Bias, Low Variance
(Goodfitting)



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

High Variance
(Overfitting)

Bias-Variance Tradeoff

The relationship between bias and variance is often referred to as the bias-variance tradeoff, which highlights the need for balance:

- Increasing model complexity reduces bias but increases variance (risk of overfitting).
- Simplifying the model reduces variance but increases bias (risk of underfitting).

The goal is to find an optimal balance where both bias and variance are minimized, resulting in good generalization performance.

Crowdsourcing and Ensemble Learning

Crowdsourcing and **Ensemble Learning** are two strategies for improving decision quality by combining multiple inputs instead of relying on a single source. One comes from human intelligence, the other from machine models—but they share the same core principle: **aggregate many weak/independent predictors to get a stronger one.**

Crowdsourcing

Crowdsourcing collects answers, labels, or solutions from many people and combines them.

Core idea

“The wisdom of the crowd” — many independent judgments often outperform a single expert.

Examples

- Labeling images on platforms like Amazon Mechanical Turk
- Community knowledge on Wikipedia
- Bug finding via Kaggle competitions

Aggregation methods

- Majority voting
- Averaging
- Weighted voting (trust better contributors more)
- EM-style truth inference (Dawid–Skene model)

Ensemble Learning

The combination of multiple models is called Ensemble. Ensemble learning combines predictions from multiple machine learning models.

Core idea

Many weak models → strong combined model

Common methods

- **Bagging** (parallel, variance reduction): Example: Random Forest
- **Boosting** (sequential, bias reduction): Examples: AdaBoost, XGBoost
- **Stacking** (meta-model learns how to combine)

How does Random Forest algorithm work?

- Random Forest works in two-phase first is to create the random forest by combining N decision tree, and second is to make predictions for each tree created in the first phase.
- The Working process can be explained in the below steps and diagram:
- **Step-1:** Select random K data points from the training set.
- **Step-2:** Build the decision trees associated with the selected data points (Subsets).
- **Step-3:** Choose the number N for decision trees that you want to build.
- **Step-4:** Repeat Step 1 & 2.
- **Step-5:** For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

Ensemble uses two methods:

Bagging: Creating a different training subset from sample training data with replacement is called Bagging. The final output is based on majority voting.

Boosting: Combining weak learners into strong learners by creating sequential models such that the final model has the highest accuracy is called Boosting. Example: ADA BOOST, XG BOOST.

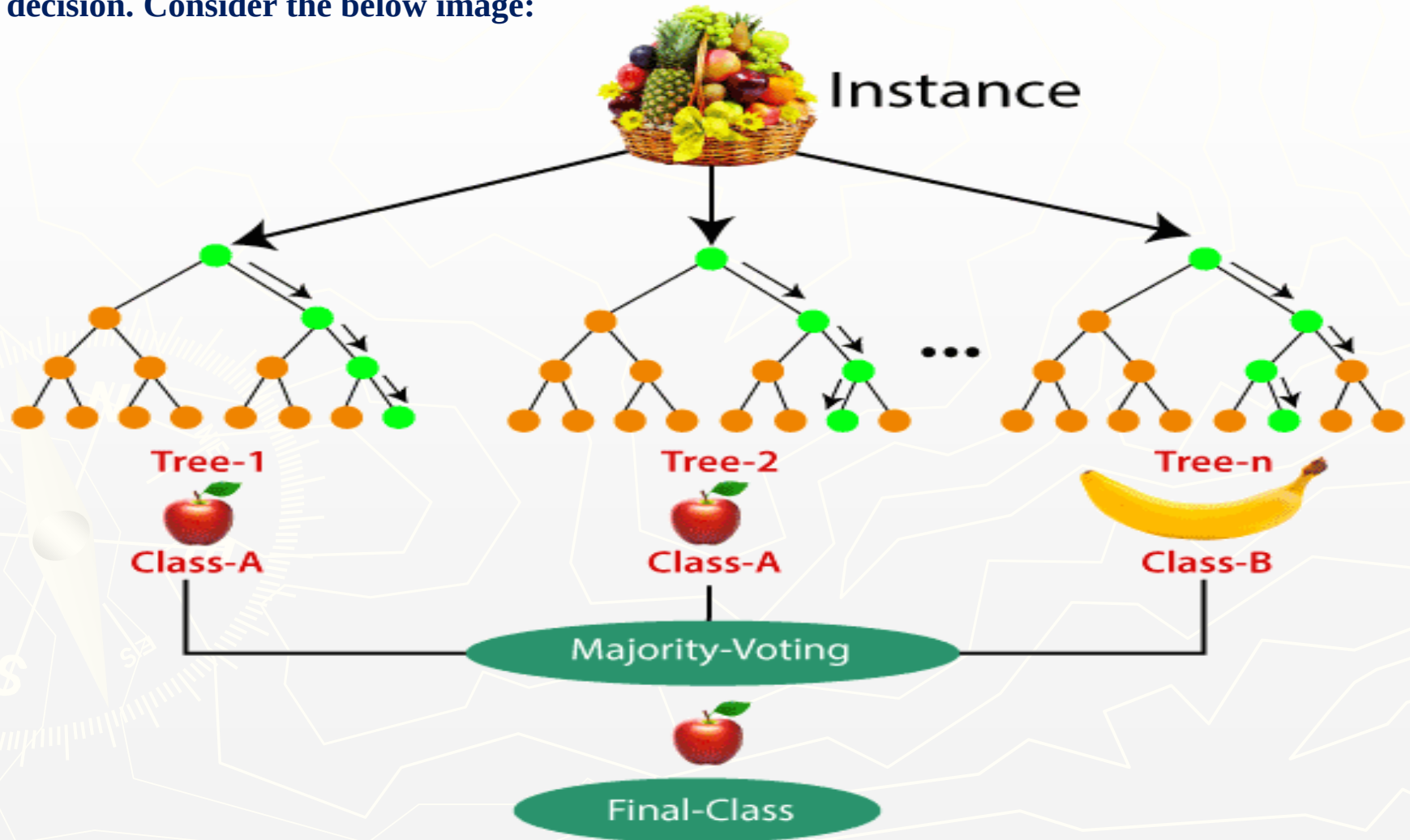
Applications of Random Forest

There are mainly four sectors where Random forest mostly used:

- 1.Banking:** Banking sector mostly uses this algorithm for the identification of loan risk.
- 2.Medicine:** With the help of this algorithm, disease trends and risks of the disease can be identified.
- 3.Land Use:** We can identify the areas of similar land use by this algorithm.
- 4.Marketing:** Marketing trends can be identified using this algorithm.

Random forest algorithm: Example

Suppose there is a dataset that contains multiple fruit images. So, this dataset is given to the Random forest classifier. The dataset is divided into subsets and given to each decision tree. During the training phase, each decision tree produces a prediction result, and when a new data point occurs, then based on the majority of results, the Random Forest classifier predicts the final decision. Consider the below image:



Example:Penguin classification

Species	Island	Body_mass_g	Flipper_length_mm
Adelie	Togersen	3750.0	181.0
Adelie	Togersen	3800.0	186.0
Chinstrap	Dream	3250.0	187.0
Chinstrap	Dream	3675.00	198.0
Gentoo	Biscoe	6000.0	220.0
Gentoo	Biscoe	4750.0	215.0

Working of random forest algorithm:

1. Random sampling with replacement
2. **Ensemble techniques(bootstrap aggregation or bagging):** Aggregating the result of decision trees and taking **the majority votes(classification) or mean(regression)**
3. How feature selection is done in classification and regression problem?
 - **Classification:** By default feature selection is taken as the square root of total number of all the features.
 - **Regression:** By default the feature are selected by total number of all features divided by 3.
- 4.The best split is chosen on Gini impurity or information gain method

A bootstrap sample is a dataset created by randomly selecting rows from the original dataset with replacement.

Example: Penguin classification

1. Random sampling with replacement/Bootstrap

Species	Island	Body_mass_g	Flipper_length_mm
Adelie	Togersen	3750.0	181.0
Adelie	Togersen	3800.0	186.0
Chinstrap	Dream	3250.0	187.0
Chinstrap	Dream	3675.00	198.0
Gentoo	Biscoe	6000.0	220.0
Gentoo	Biscoe	4750.0	215.0

Species	Island	Body_mass_g
Adelie	Togersen	3750.0
Chinstrap	Dream	3250.0
Gentoo	Biscoe	6000.0
Gentoo	Biscoe	4750.0

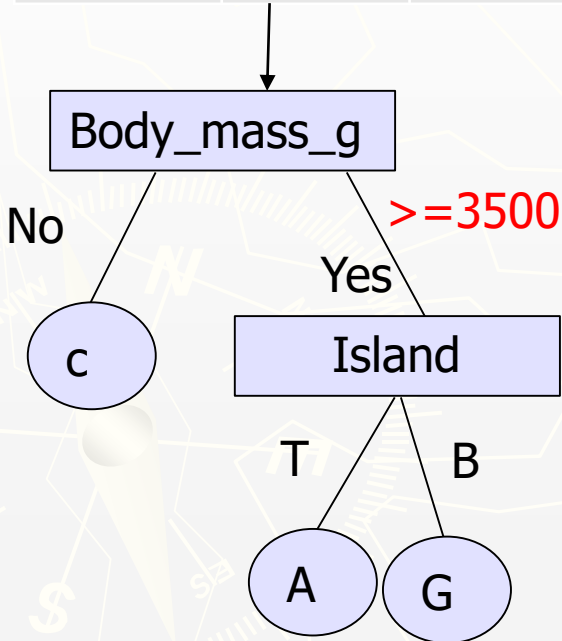
Species	Island	Flipper_length_mm
Adelie	Togersen	186.0
Chinstrap	Dream	187.0
Gentoo	Biscoe	215.0
Gentoo	Biscoe	215.0

Species	Body_mass_g	Flipper_length_mm
Adelie	3750	181.0
Adelie	3800	186.0
Chinstrap	3675	198.0
Gentoo	6000	220.0

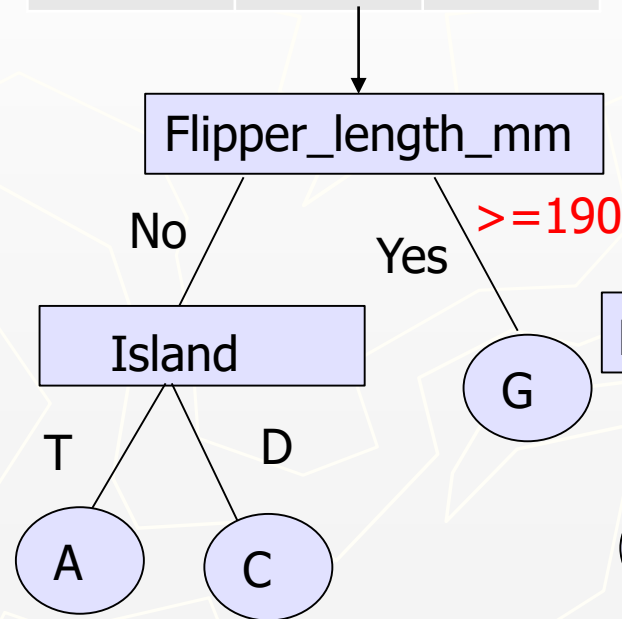
Example: Penguin classification

Creating decision trees from the subsamples

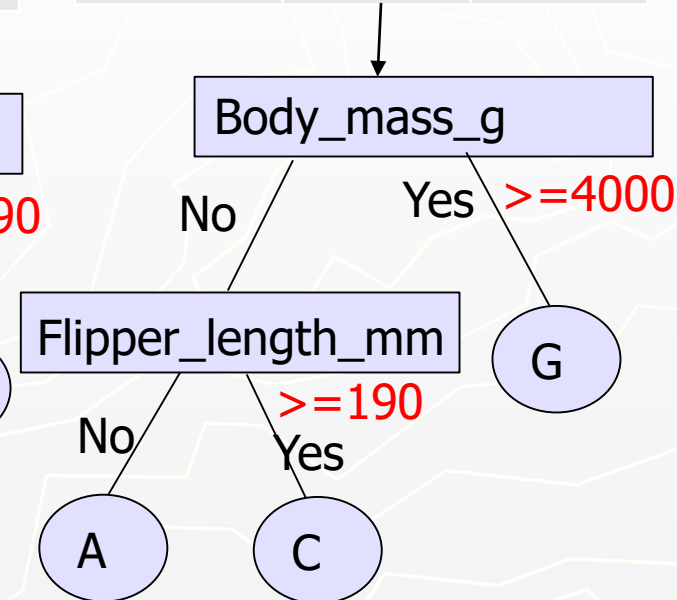
Species	Island	Body_mass_g
Adelie	Togersen	3750.0
Chinstrap	Dream	3250.0
Gentoo	Biscoe	6000.0
Gentoo	Biscoe	4750.0



Species	Island	Flipper_length_mm
Adelie	Togersen	186.0
Chinstrap	Dream	187.0
Gentoo	Biscoe	215.0
Gentoo	Biscoe	215.0



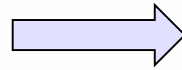
Species	Body_mass_g	Flipper_length_mm
Adelie	3750	181.0
Adelie	3800	186.0
Chinstrap	3675	198.0
Gentoo	6000	220.0



Example: Penguin classification

2. Ensemble techniques: bootstrap aggregation or bagging:

Dream 325.5 186.3



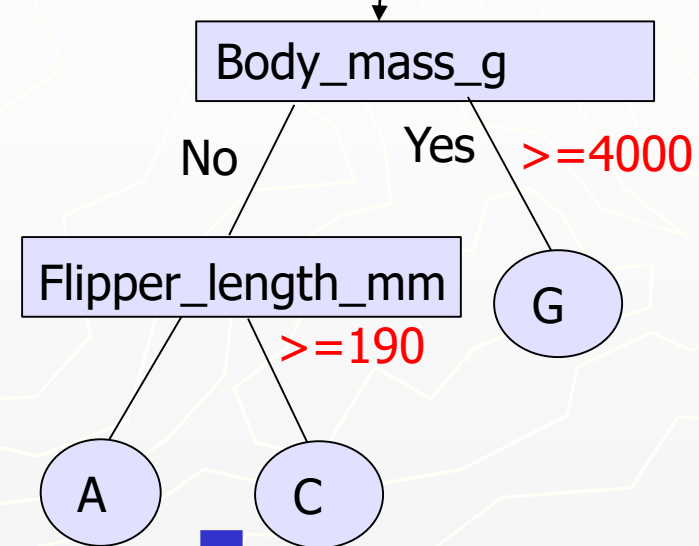
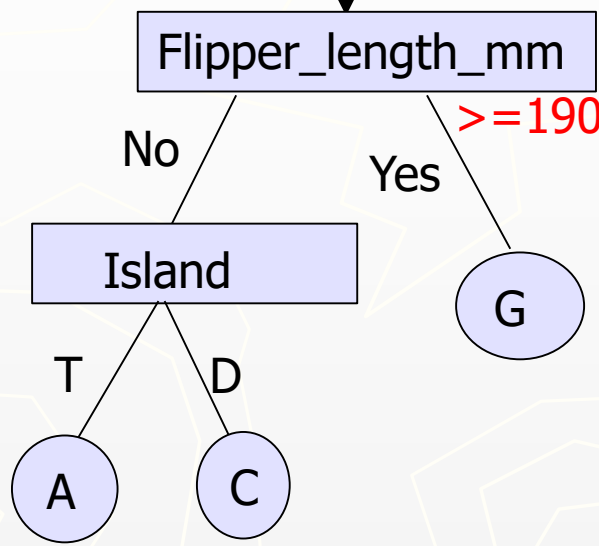
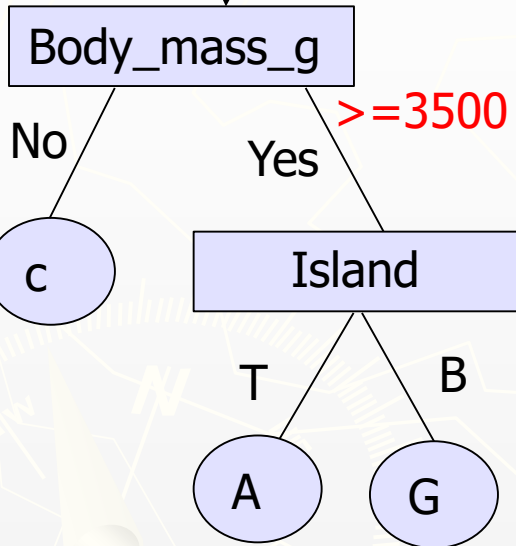
Species?



DT 1

DT 2

DT 3



Chinstrap

Chinstrap

Adelie

Ensemble Result

Unknown Species = max votes

Dream 325.5 186.3



Species?



Chinstrap	Adelie
2	1

Random forest algorithm

Advantages of Random Forest

- Random Forest is capable of performing both Classification and Regression tasks.
- It is capable of handling large datasets with high dimensionality.
- It enhances the accuracy of the model and prevents the overfitting issue.

Disadvantages of Random Forest

- Although random forest can be used for both classification and regression tasks, it is not more suitable for Regression tasks.



Example: Buy a Laptop classification

Problem

Predict whether a person will **Buy a Laptop (Yes/No)**.

ID	Age	Income	Student	Buy
1	Young	High	No	No
2	Young	Low	Yes	Yes
3	Middle	Low	Yes	Yes
4	Old	Medium	No	No
5	Old	High	Yes	Yes
6	Middle	Medium	No	No

Step 1 — Choose number of trees

Assume: Number of trees = 3

We will build **3 decision trees**.

Step 2 — Bootstrap Sampling (Random rows with replacement)

Each tree gets different training data.

Tree 1 sample: 1, 2, 3, 3, 5, 6

Tree 2 sample: 2, 4, 4, 5, 6, 1

Tree 3 sample: 1, 2, 2, 3, 4, 5

Some rows repeat, some missing → trees become different.

Example: Buy a Laptop classification

Step 3 — Random Feature Selection

Total features = 3

We randomly pick **2 features** per split.

Tree	Features used
T1	Age, Income
T2	Income, Student
T3	Age, Student

Trees don't look at the same features → less correlation.

Step 4 — Train each Decision Tree

Each tree learns rules from its data.

Tree 1 rules

If Income = Low → Yes

Else → No

Tree 2 rules

If Student = Yes → Yes

Else → No

Tree 3 rules

If Age = Old → No

Else → Yes

Example: Buy a Laptop classification

Step 3 — Random Feature Selection

Total features = 3

We randomly pick **2 features** per split.

Tree	Features used
T1	Age, Income
T2	Income, Student
T3	Age, Student

Trees don't look at the same features → less correlation.

Step 4 — Train each Decision Tree

Each tree learns rules from its data.

Tree 1 rules

If Income = Low → Yes

Else → No

Tree 2 rules

If Student = Yes → Yes

Else → No

Tree 3 rules

If Age = Old → No

Else → Yes

Each tree gives **different logic**.

Example: Buy a Laptop classification

Step 5 — Predict new person

New customer:

Age = Young

Income = Medium

Student = Yes

◆ Step 6 — Get predictions from all trees

Tree	Prediction
T1	No
T2	Yes
T3	Yes

Step 7 — Majority Voting

Yes = 2

No = 1

Final prediction:

Buy = Yes

Random Forest Steps

1. Choose number of trees (n)
2. For each tree:
 1. Take bootstrap sample
 2. Select random subset of features
 3. Train decision tree
3. For new input:
 1. Get predictions from all trees
 2. Use majority vote

Naïve Bayes Classifier Algorithm

- Naive Bayes is a statistical classification technique based on Bayes Theorem. It is one of the simplest supervised learning algorithms. Naive Bayes classifier is the fast, accurate and reliable algorithm. Naive Bayes classifiers have high accuracy and speed on large datasets. It **is a probabilistic classifier, which means it predicts on the basis of the probability of an object**. Some popular examples of Naïve Bayes Algorithm are spam filtration, Sentimental analysis, and classifying articles.
- The Naïve Bayes algorithm is comprised of two words Naïve and Bayes, Which can be described as:
- Naïve:** It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.
- Bayes:** It is called Bayes because it depends on the principle of Bayes' Theorem.

Bayes' Theorem: Bayes' theorem is also known as **Bayes' Rule** or **Bayes' law**, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability. The formula for Bayes' theorem is given as:

$$P(A|B) \propto P(B|A) \cdot P(A)$$

P(A|B) is Posterior probability: Probability of hypothesis A on the observed event B.

P(B|A) is Likelihood probability : Probability of the evidence given that the probability of a hypothesis is true. (chance of something happening)

P(A) is Prior Probability(Class): Probability of hypothesis before observing the evidence.

P(B) is Marginal Probability(Feature vector): Probability of Evidence.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

How does Naïve Bayes' Classifier work?

Working of Naïve Bayes' Classifier:

Working of Naïve Bayes' Classifier can be understood with the help of the below example:

Suppose we have a dataset of **weather conditions** and corresponding target variable "**Play**". So using this dataset we need to decide that whether we should play or not on a particular day according to the weather conditions. So to solve this problem, we need to follow the below steps: If multiple features exist: $P(C|X1,X2) \propto P(C) \cdot P(X1|C) \cdot P(X2|C)$

1. Convert the given dataset into frequency tables.
2. Generate Likelihood table by finding the probabilities of given features.
3. Now, use Bayes theorem to calculate the posterior probability.

Problem: If the weather is sunny, then the Player should play or not?

Solution: To solve this, first consider the defined dataset:

Weather	Yes	No
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	10	4

Frequency table for the Weather Conditions

No	Weather	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

Datasets

How does Naïve Bayes' Classifier work?

Weather	No	Yes	
Overcast	0	5	5/14= 0.35
Rainy	2	2	4/14=0.29
Sunny	2	3	5/14=0.35
All	4/14=0.29	10/14=0.71	

Likelihood table weather condition

Applying Bayes 'theorem:

- **$P(\text{Yes}|\text{Sunny}) = P(\text{Sunny}|\text{Yes}) * P(\text{Yes}) / P(\text{Sunny})$**

$$P(\text{Sunny}|\text{Yes}) = 3/10 = 0.3$$

$$P(\text{Sunny}) = 0.35$$

$$P(\text{Yes}) = 0.71$$

$$\text{So } P(\text{Yes}|\text{Sunny}) = 0.3 * 0.71 / 0.35 = \mathbf{0.60}$$

- **$P(\text{No}|\text{Sunny}) = P(\text{Sunny}|\text{No}) * P(\text{No}) / P(\text{Sunny})$**

$$P(\text{Sunny}|\text{NO}) = 2/4 = 0.5$$

$$P(\text{No}) = 0.29$$

$$P(\text{Sunny}) = 0.35$$

$$\text{So } P(\text{No}|\text{Sunny}) = 0.5 * 0.29 / 0.35 = \mathbf{0.41}$$

So as we can see from the above calculation that **$P(\text{Yes}|\text{Sunny}) > P(\text{No}|\text{Sunny})$**

Hence on a Sunny day, Player can play the game.

How does Naïve Bayes' Classifier work?

Problem

Predict whether a person will **Buy Computer (Yes/No)** based on:

- Age (Young / Middle / Senior)
- Income (High / Medium / Low)
- Student (Yes / No)

Task

Predict for:

Age = Young

Income = Low

Student = Yes

Step 1: Compute Prior Probabilities

Count class labels:

Yes = 6

No = 4

$$P(\text{Yes}) = 6/10 = 0.6$$

$$P(\text{No}) = 4/10 = 0.4$$

Step 2: Compute Likelihoods

We compute probabilities **conditioned on each class**.

- **For Class = Yes** $P(\text{Age=Young} \mid \text{Yes})$

Total Yes = 6

From Yes rows: Young appears 1 time

$$P(\text{Young} \mid \text{Yes}) = 1/6$$

Datasets

Age	Income	Student	Buy
Young	High	No	No
Young	High	No	No
Middle	High	No	Yes
Senior	Medium	No	Yes
Senior	Low	Yes	Yes
Senior	Low	Yes	No
Middle	Low	Yes	Yes
Young	Medium	No	No
Young	Low	Yes	Yes
Senior	Medium	Yes	Yes

Total samples = 10

How does Naïve Bayes' Classifier work?

2)P(Income=Low | Yes)

Low appears 3 times

$$P(\text{Low}|\text{Yes})=3/6=0.5$$

3)P(Student=Yes | Yes)

Yes appears 4 times

$$P(\text{Student}=\text{Yes}|\text{Yes})=4/6$$

• For Class = No

Total No = 4

1)P(Young | No)

Young appears 3 times

$$P(\text{Young}|\text{No})=3/4$$

2)P(Low | No)

Low appears 1 time

$$P(\text{Low}|\text{No})=1/4$$

3)P(Student=Yes | No)

Appears 1 time

$$P(\text{Student}=\text{Yes}|\text{No})=1/4$$

Step 3: Apply Naive Bayes Formula

$$P(\text{Yes}|X) \propto P(\text{Yes}) \cdot P(\text{Young}|\text{Yes}) \cdot P(\text{Low}|\text{Yes}) \cdot P(\text{Student} = \text{Yes}|\text{Yes})$$

$$= 0.6 \times (1/6) \times (3/6) \times (4/6)$$

$$= 0.6 \times 0.167 \times 0.5 \times 0.667$$

$$\approx 0.033$$

$$P(\text{No}|X) \propto 0.4 \times (3/4) \times (1/4) \times (1/4)$$

$$= 0.4 \times 0.75 \times 0.25 \times 0.25$$

$$\approx 0.0187$$

Step 4: Compare

$$0.033 > 0.0187$$

Final Prediction = **Yes**

Naïve Bayes Classifier

Advantages of Naïve Bayes Classifier:

- Naïve Bayes is one of the fast and easy ML algorithms to predict a class of datasets.
- It can be used for Binary as well as Multi-class Classifications.
- It performs well in Multi-class predictions as compared to the other Algorithms.
- It is the most popular choice for **text classification problems**.

Disadvantages of Naïve Bayes Classifier:

- Naive Bayes assumes that all features are independent or unrelated, so it cannot learn the relationship between features.

Applications of Naïve Bayes Classifier:

- It is used for **Credit Scoring**.
- It is used in **medical data classification**.
- It can be used in **real-time predictions** because Naïve Bayes Classifier is an eager learner.
- It is used in Text classification such as **Spam filtering** and **Sentiment analysis**.

Types of Naïve Bayes Model:

There are three types of Naive Bayes Model, which are given below:

Gaussian(For continuous data): The Gaussian model assumes that features follow a normal distribution. This means if predictors take continuous values instead of discrete, then the model assumes that these values are sampled from the Gaussian distribution.

Multinomial(For text data): The Multinomial Naïve Bayes classifier is used when the data is multinomial distributed. It is primarily used for document classification problems, it means a particular document belongs to which category such as Sports, Politics, education, etc. The classifier uses the frequency of words for the predictors.

Bernoulli(For binary features): The Bernoulli classifier works similar to the Multinomial classifier, but the predictor variables are the independent Booleans variables. Such as if a particular word is present or not in a document. This model is also famous for document classification tasks.

K-Nearest Neighbor(KNN) Algorithm

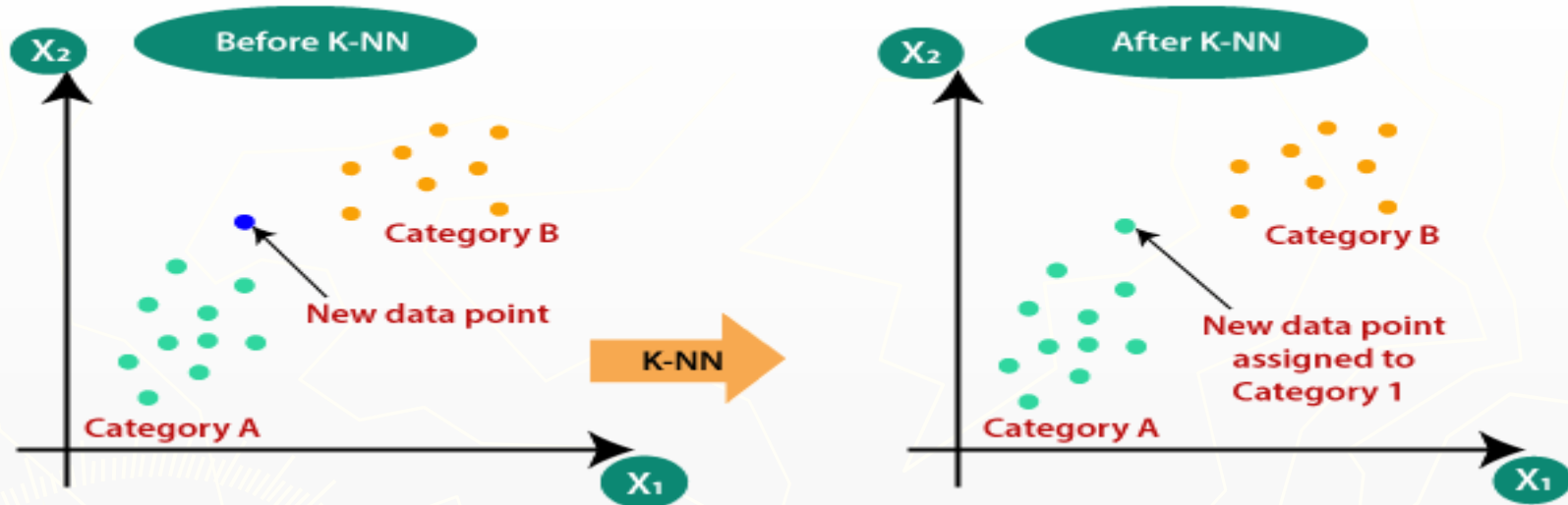
- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique. K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.KNN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data. It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset. KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.



Why do we need a K-NN Algorithm?

- Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x_1 , so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram.



How does K-NN work?

The K-NN working can be explained on the basis of the below algorithm:

Step-1: Select the number K of the neighbors

Step-2: Calculate the Euclidean distance of **K number of neighbors**

Step-3: Take the K nearest neighbors as per the calculated Euclidean distance.

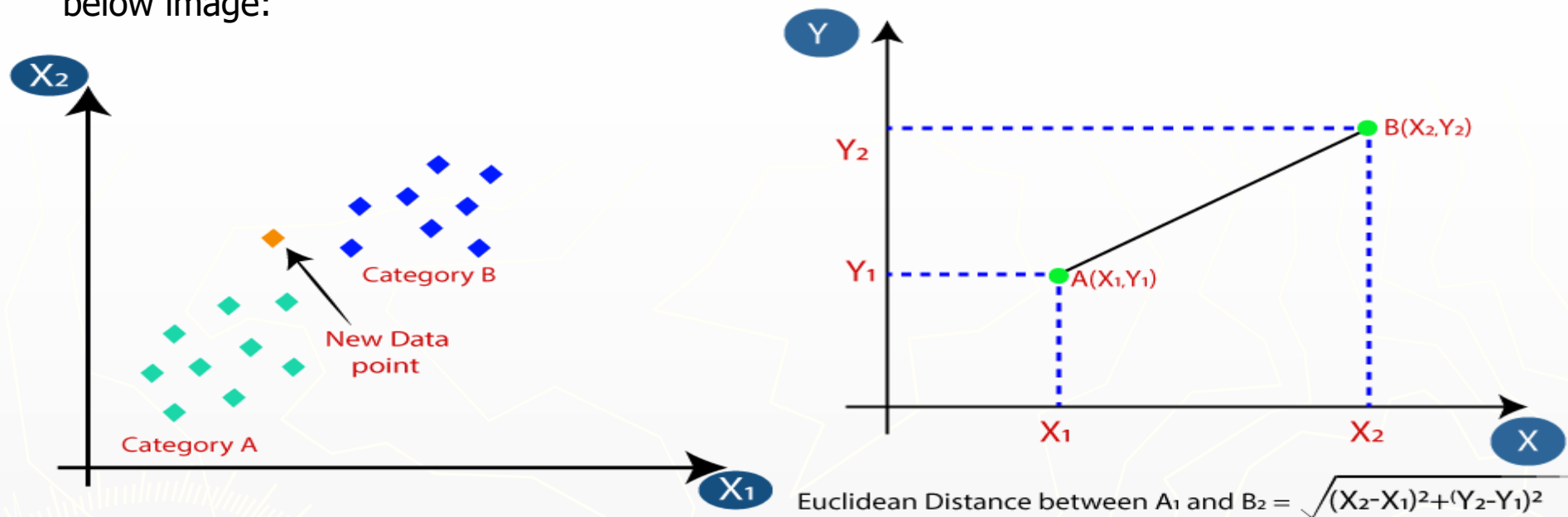
Step-4: Among these k neighbors, count the number of the data points in each category.

Step-5: Assign the new data points to that category for which the number of the neighbor is maximum.

Step-6: Our model is ready.

How does K-NN work?

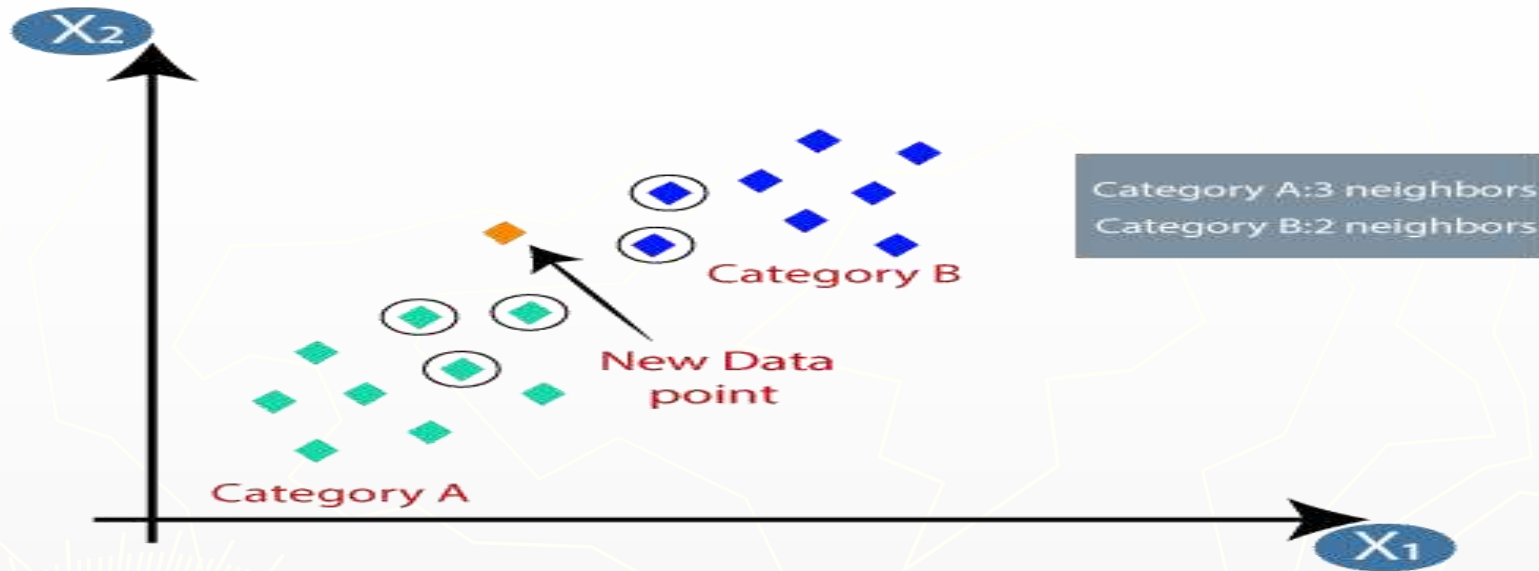
- Suppose we have a new data point and we need to put it in the required category. Consider the below image:



- Firstly, we will choose the number of neighbors, so we will choose the $k=5$.
- Next, we will calculate the **Euclidean distance** between the data points. The Euclidean distance is the distance between two points, which we have already studied in geometry. It can be calculated as by above formula
- The value of k in the KNN algorithm is related to the error rate of the model. A small value of k could lead to overfitting as well as a big value of k can lead to underfitting. **Overfitting** imply that the model is well on the training data but has poor performance when new data is coming. **Underfitting** refers to a model that is not good on the training data and also cannot be generalized to predict new data.

How does K-NN work?

- By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image:



As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.

How to select the value of K in the K-NN Algorithm?

Below are some points to remember while selecting the value of K in the K-NN algorithm:

- There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them. The most preferred value for K is 5.
- A very low value for K such as $K=1$ or $K=2$, can be noisy and lead to the effects of outliers in the model.
- Large values for K are good, but it may find some difficulties.

K-Nearest Neighbor(KNN) Algorithm

➤ Value of K will effect decision boundary in KNN.

✓ Smaller K (noisy decision boundary): With minor changes in train set, the decision boundary changes.

✓ Higher K (high bias model): With higher K, the model works consistently on Train and Test sets, but model may become underfitted.

✓ One needs to do trail and test error to find best K

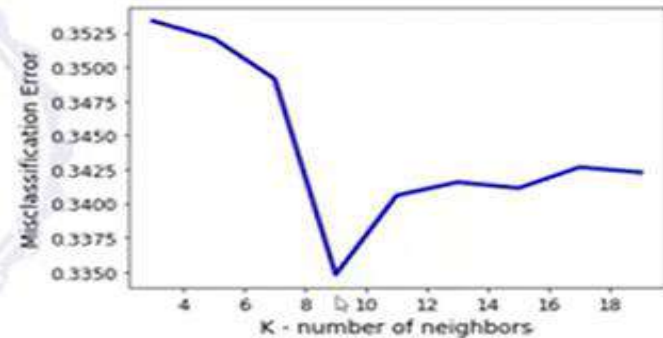
- ❖ Start with K = 3, train and Cross Validation
- ❖ K=5, train and Cross Validation
- ❖ K=7, train an Cross Validation

.....

.....

❖ Till we find best K that works best on TRAIN set and Cross Validation set.

❖ Learning Curve will help us find the best K



Cross-validation is a fundamental technique used to assess how well a machine learning model generalizes to new, unseen data and helps in detecting issues like overfitting or underfitting. It involves repeatedly partitioning a dataset into subsets for training and testing and then averaging the results to provide a more reliable estimate of the model's performance.

Advantages of KNN Algorithm:

- It is simple to implement.
- It is robust to the noisy training data
- It can be more effective if the training data is large.

Disadvantages of KNN Algorithm:

- Always needs to determine the value of K which may be complex some time.
- The computation cost is high because of calculating the distance between the data points for all the training samples.

K-Nearest Neighbor(KNN) Algorithm: Example

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Versicolor
7.3	2.9	Virginica
6.0	2.7	Versicolor
5.8	2.8	Virginica
6.3	2.3	Versicolor
5.1	2.5	Versicolor
6.3	2.5	Versicolor
5.5	2.4	Versicolor

IRIS Dataset

Sepal Length	Sepal Width	Species
5.2	3.1	?

K-Nearest Neighbor(KNN) Algorithm

Step 1: Find Distance

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(x-a)^2 + (y-b)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(5.2-5.3)^2 + (3.1-3.7)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = 0.608$$

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608

K-Nearest Neighbor(KNN) Algorithm

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608
5.1	3.8	Setosa	0.707
7.2	3.0	Virginica	2.002
5.4	3.4	Setosa	0.36
5.1	3.3	Setosa	0.22
5.4	3.9	Setosa	0.82
7.4	2.8	Virginica	2.22
6.1	2.8	Versicolor	0.94
7.3	2.9	Virginica	2.1
6.0	2.7	Versicolor	0.89
5.8	2.8	Virginica	0.67
6.3	2.3	Versicolor	1.36
5.1	2.5	Versicolor	0.60
6.3	2.5	Versicolor	1.25
5.5	2.4	Versicolor	0.75

K-Nearest Neighbor(KNN) Algorithm

Step 2: Find the rank/Sort by Distance

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Versicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Versicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Versicolor	1.36	12
5.1	2.5	Versicolor	0.60	4
6.3	2.5	Versicolor	1.25	11
5.5	2.4	Versicolor	0.75	7

K-Nearest Neighbor(KNN) Algorithm

Step 3: Find the nearest neighbors

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Versicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Versicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Versicolor	1.36	12
5.1	2.5	Versicolor	0.60	4
6.3	2.5	Versicolor	1.25	11
5.5	2.4	Versicolor	0.75	7

If k=1 , Setosa

If k=2 , Setosa

If k=3 , Setosa

If k=4 , Versicolor

If k=5 , Virginica

#here maximum ranked is **Setosa** so our new species will be Setosa

K-Nearest Neighbor(KNN) Algorithm

Step 4: Pick $K = 5$ Nearest Neighbors

If $k=1$, Setosa

If $k=2$, Setosa

If $k=3$, Setosa

If $k=4$, Versicolor

If $k=5$, Virginica

Step 5: Majority Voting

Setosa=3

Versicolor = 1

Virginica=1

#here maximum ranked is **Setosa** so our new species will be **Setosa**

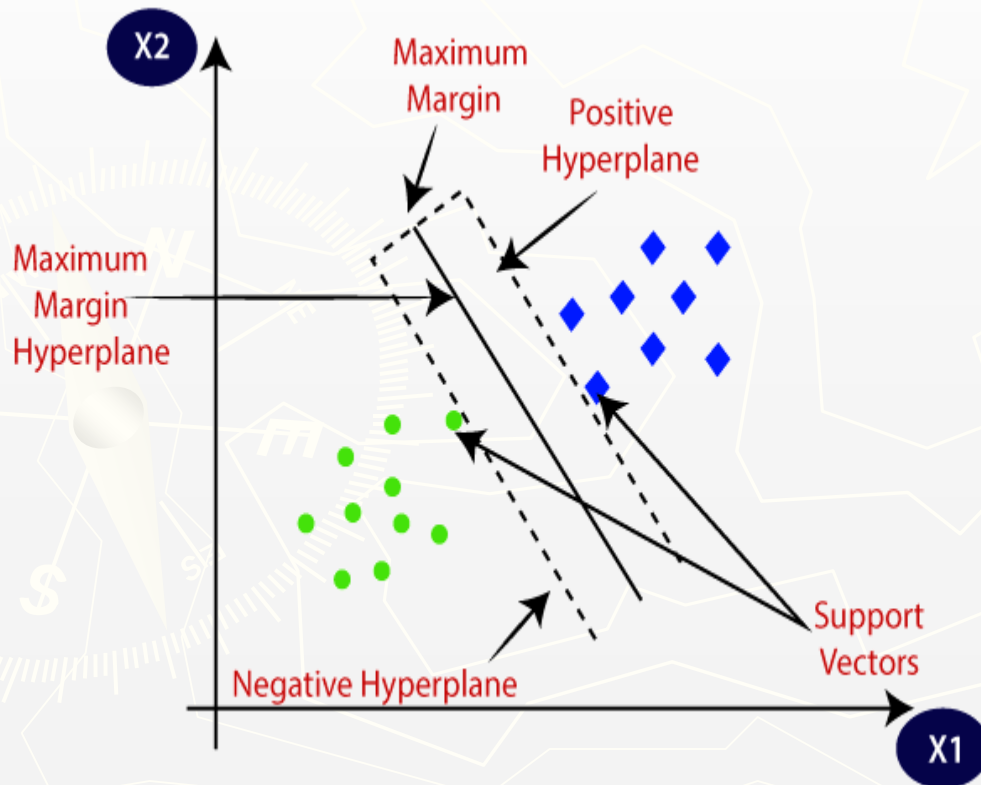
👉 **Final Prediction: Setosa**

Final Answer

👉 The new point **P(5.2,3.1)** belongs to class = **Setosa**

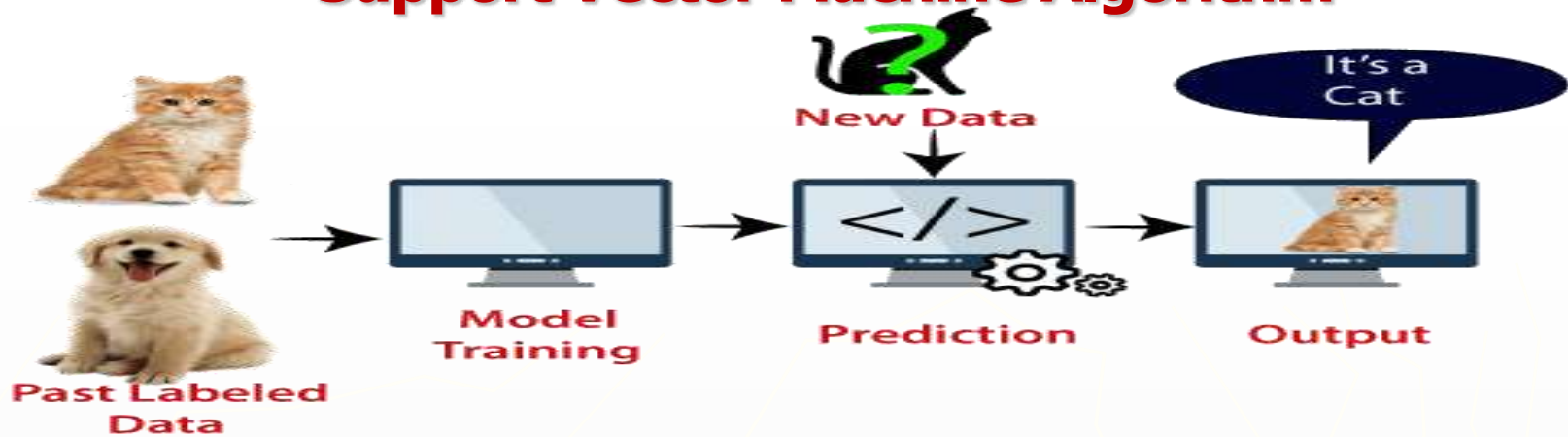
Support Vector Machine Algorithm

- Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.
- The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.
- SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Example: SVM can be understood with the example that we have used in the KNN classifier. Suppose we see a strange cat that also has some features of dogs, so if we want a model that can accurately identify whether it is a cat or dog, so such a model can be created by using the SVM algorithm. We will first train our model with lots of images of cats and dogs so that it can learn about different features of cats and dogs, and then we test it with this strange creature. So as support vector creates a decision boundary between these two data (cat and dog) and choose extreme cases (support vectors), it will see the extreme case of cat and dog. On the basis of the support vectors, it will classify it as a cat. Consider the next diagram:

Support Vector Machine Algorithm



Types of SVM

SVM can be of two types:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Hyperplane and Support Vectors in the SVM algorithm:

- **Hyperplane:** There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.

The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.

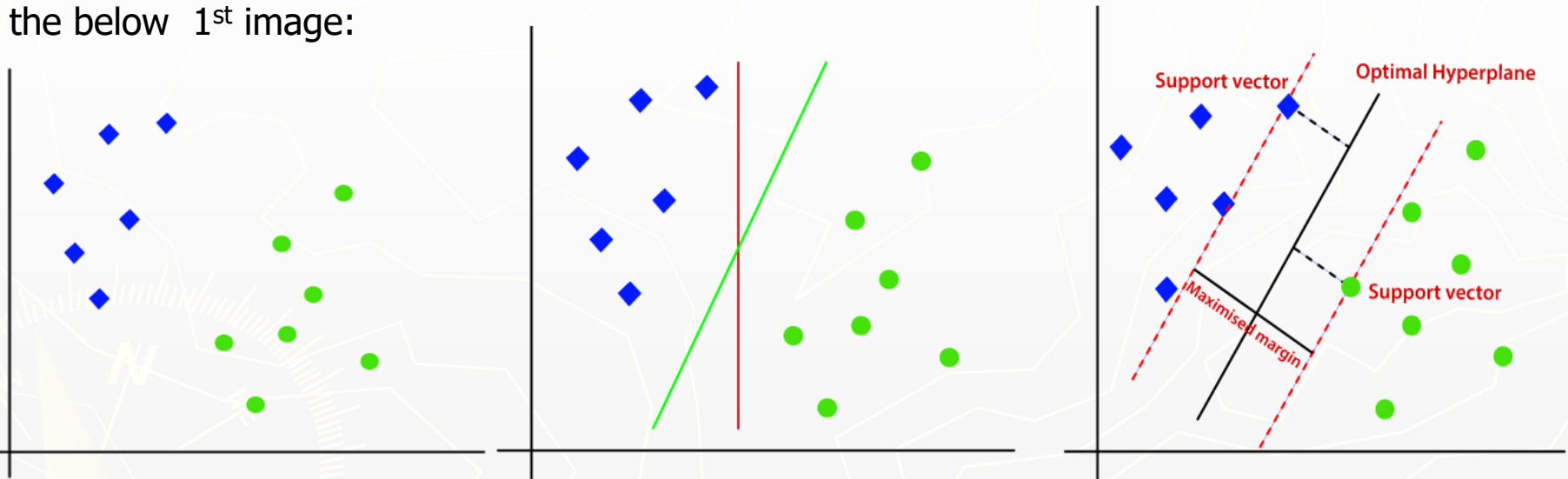
We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.

How does SVM works?

SVM :The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

Linear SVM:

The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. Consider the below 1st image:



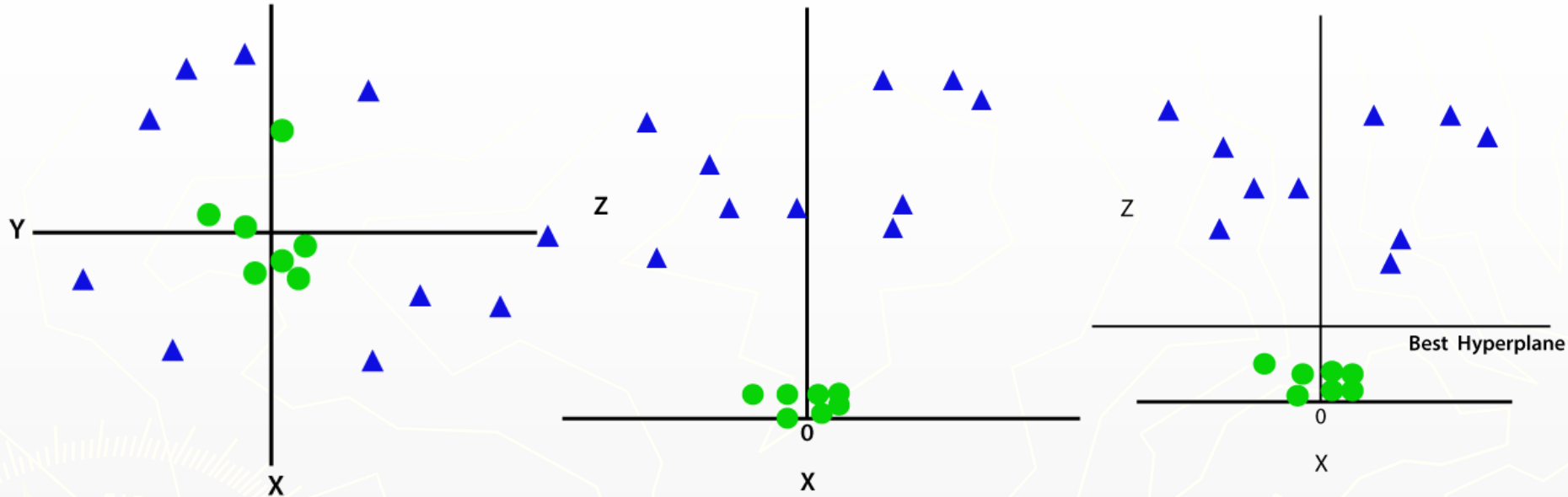
So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the 2nd image.

Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.

How does SVM works?

Non-Linear SVM:

If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image (fig. 1):



So now, SVM will divide the datasets into classes in the following way. Consider the above (fig. 2) image:

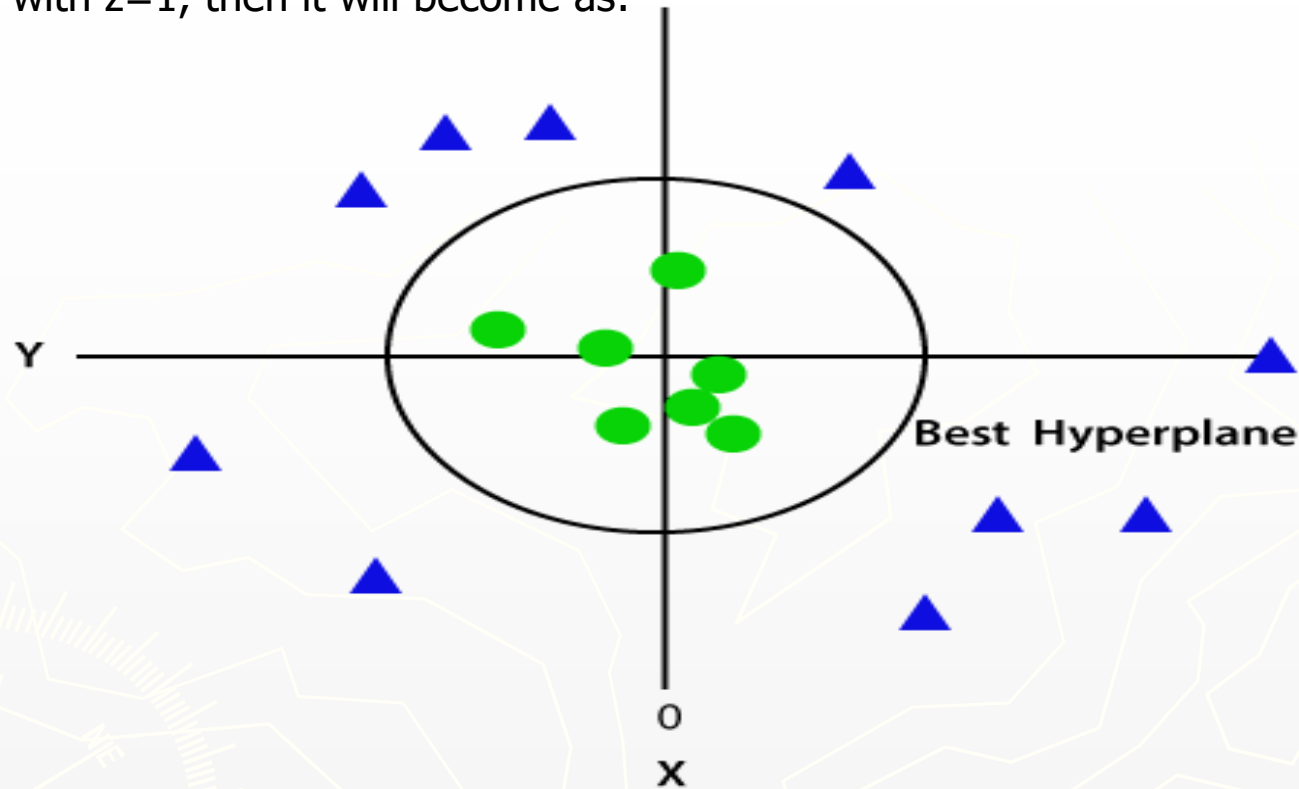
So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z. It can be calculated as: $Z = x^2 + y^2$

By adding the third dimension, the sample space will become as above image(fig. 3)

SVM algorithm can be used for **Face detection, image classification, text categorization, Bioinformatics, Generalized Predictive Control(GPC), Handwriting Recognition** etc.

How does SVM works?

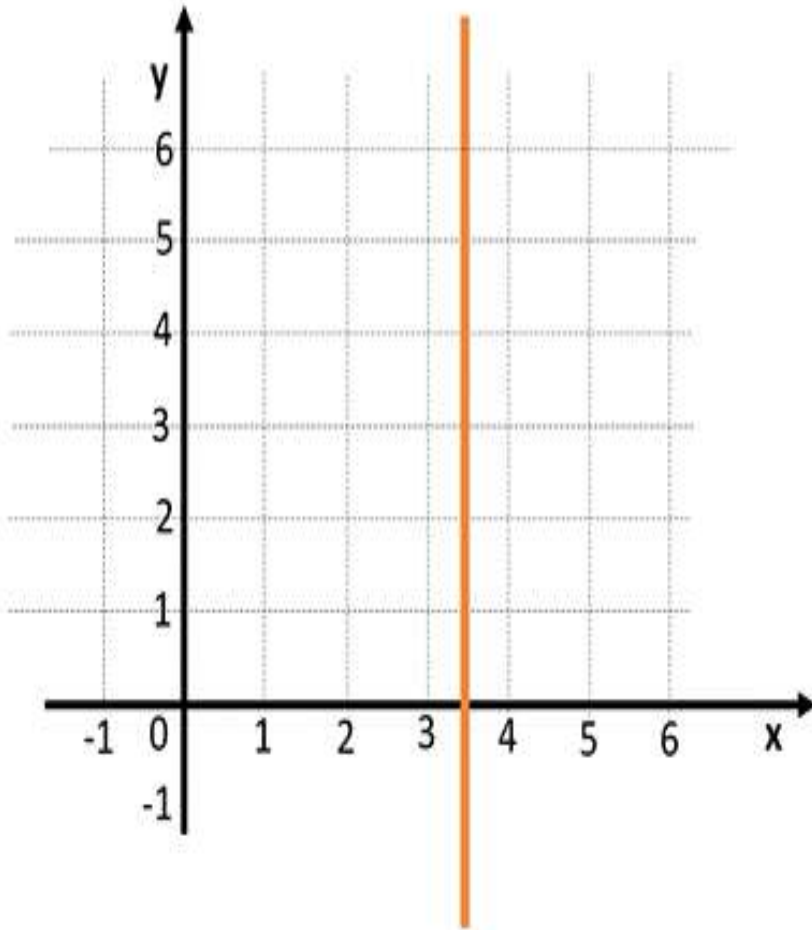
Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis. If we convert it in 2d space with $z=1$, then it will become as:



Hence we get a circumference of radius 1 in case of non-linear data.

Example: Plot hyperplane of following points in svm; (1,1) (2,1) (1,-1)(2,-1)(4,0)(5,1)(5,-1)(6,0) and yes, after solving this important svm example you will understand also following important points regarding it; Q1. What is a hyperplane in SVM and how to plot it? Q2. How do you find the support vector when other points are given ? Q3. How do you find optimal hyperplane in SVM example problem ? Q4. What is the equation of hyperplane in svm numerical example ?

The equation of a straight line

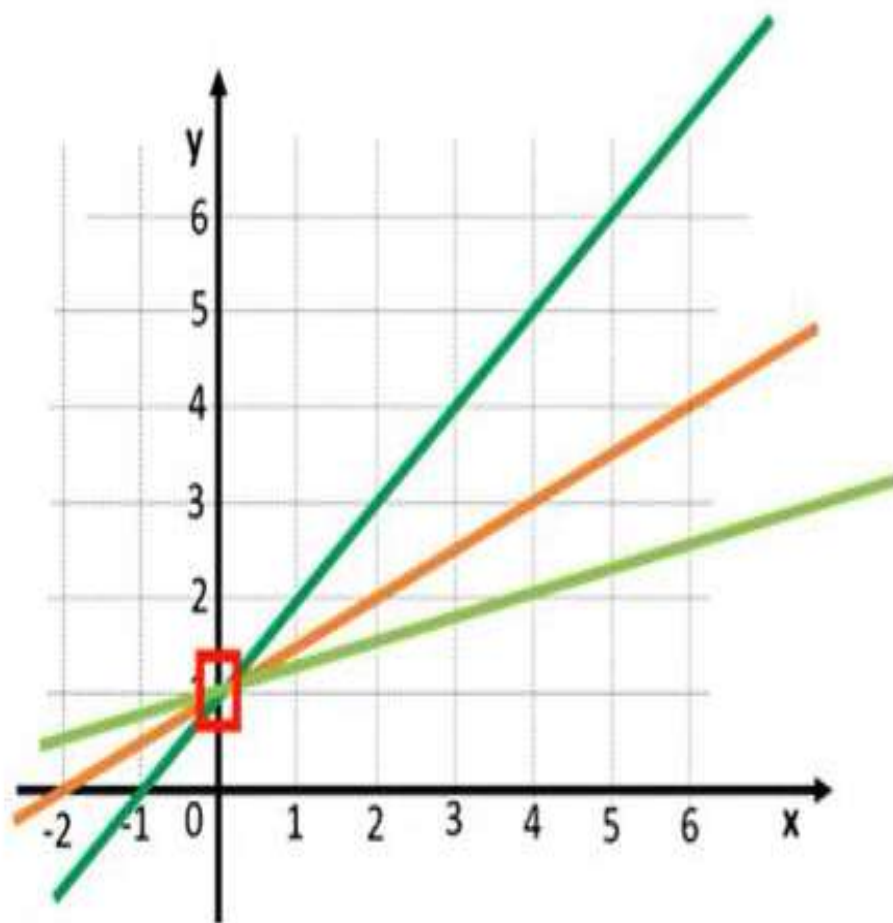


$$y = mx + b$$

$$Ax + By + C = 0$$

$$y = -\frac{A}{B}x - \frac{C}{B}$$

The equation of a straight line

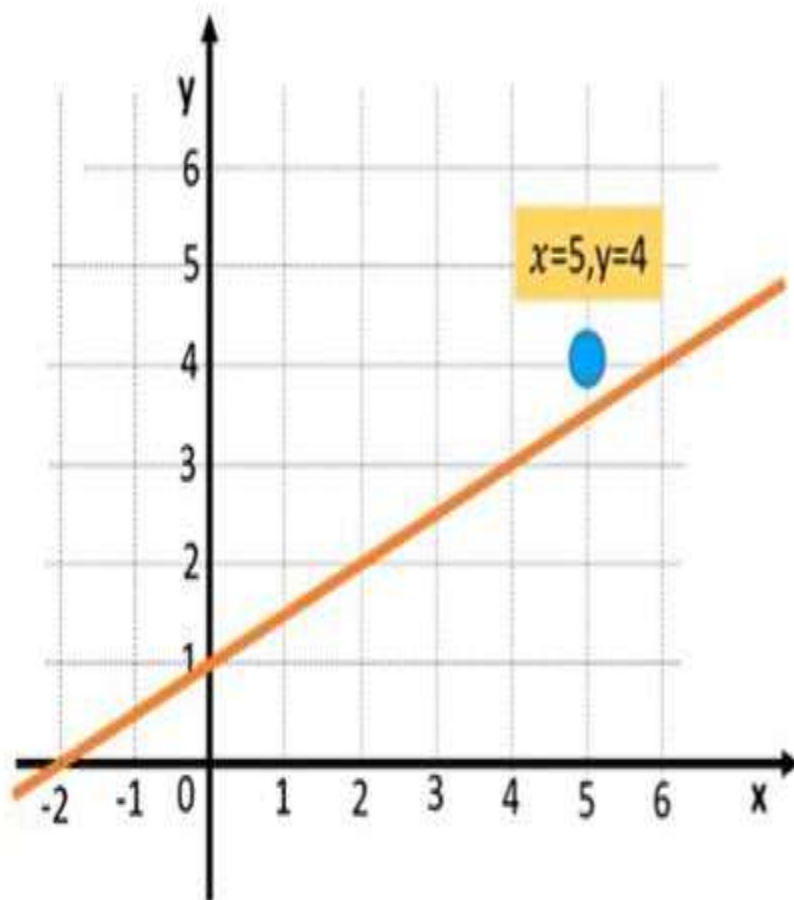


$$Ax + By + C = 0$$
$$-2x + 4y - 4 = 0$$

$$Ax + By + C = 0$$

$$y = -\frac{A}{B}x - \frac{C}{B}$$

The equation of a straight line

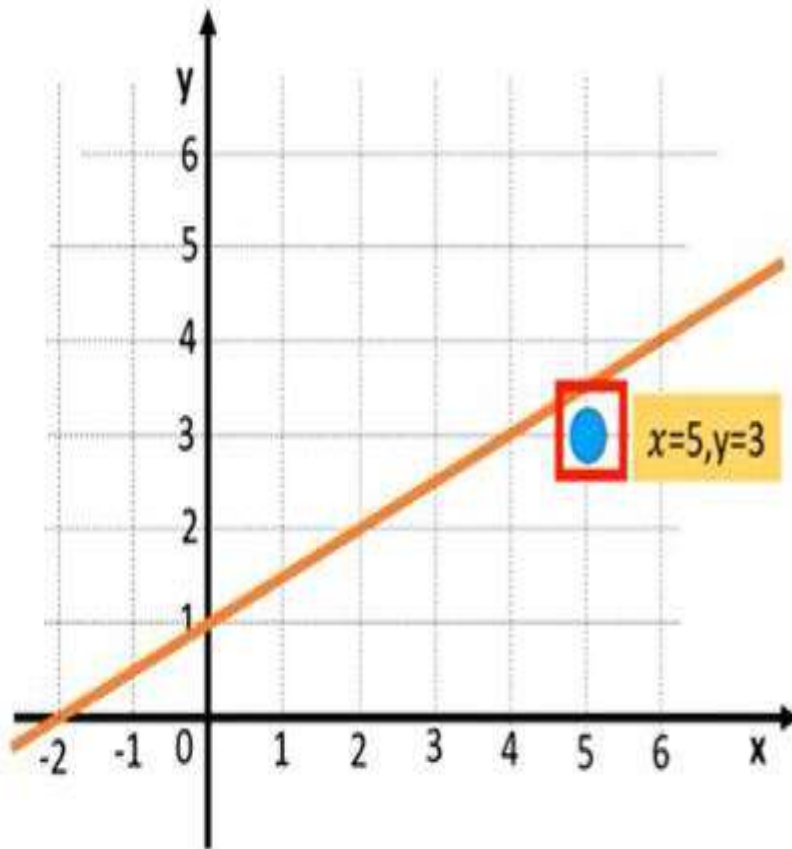


$$Ax + By + C = 0$$

$$-2x + 4y - 4 = 0$$

$$-2 \cdot 5 + 4 \cdot 4 - 4 = \boxed{2}$$

The equation of a straight line



$$Ax + By + C = 0$$

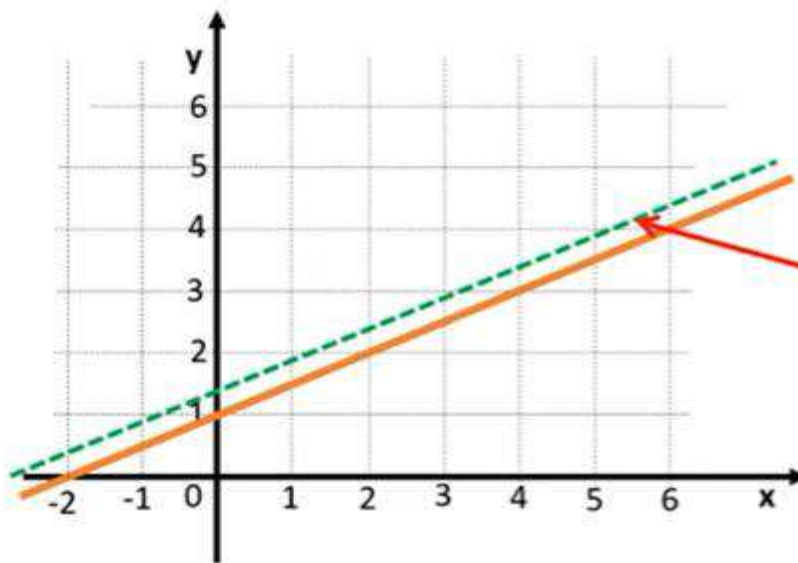
$$-2x + 4y - 4 = 0$$

$$-2 \cdot 5 + 4 \cdot 3 - 4 = -2$$

How does SVM works?

The equation of a straight line

$$y = 0.5x + 1$$



$$Ax + By + C = 0$$

$$-2x + 4y - 4 = 0$$

$$-2x + 4y - 4 = 1$$

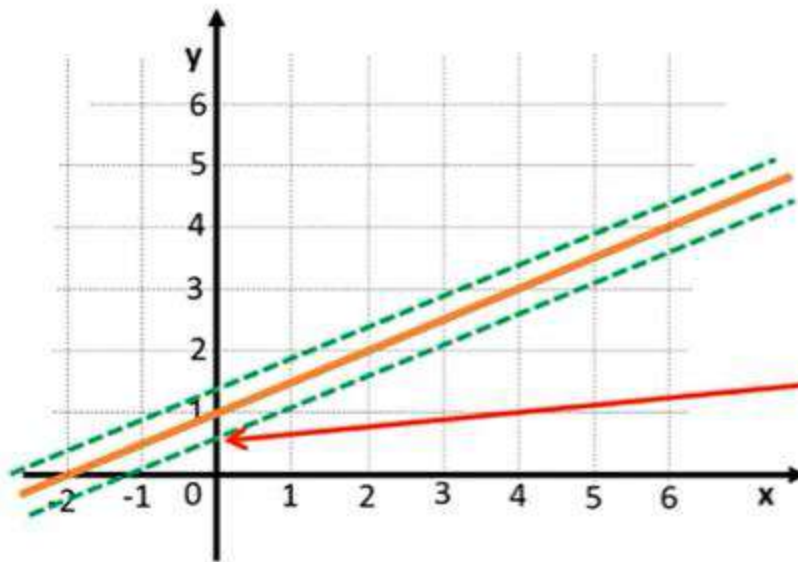
$$4y = 2x + 5$$

$$y = 0.5x + 1.25$$

How does SVM works?

The equation of a straight line

$$y = 0.5x + 1$$



$$Ax + By + C = 0$$

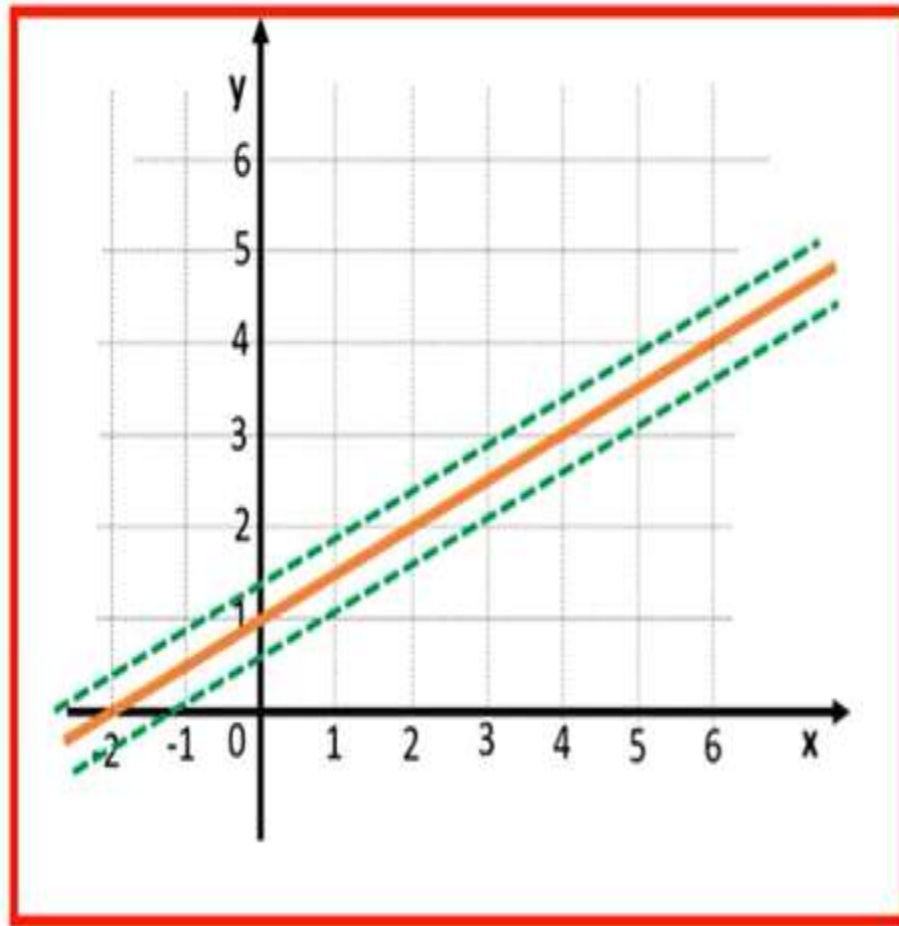
$$-2x + 4y - 4 = 0$$

$$-2x + 4y - 4 = -1$$

$$4y = 2x + 3$$

$$y = 0.5x + 0.75$$

The equation of a straight line



$$Ax + By + C = 0$$

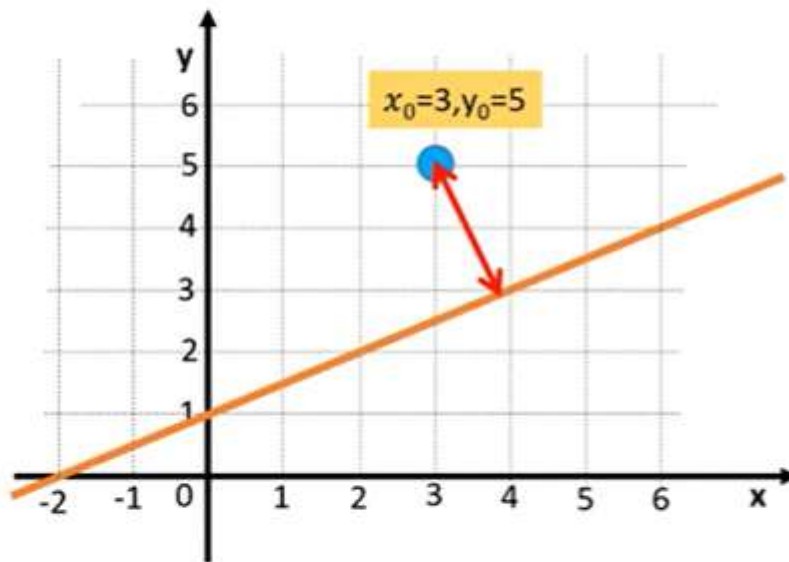
$$-2x + 4y - 4 = 1$$

$$-2x + 4y - 4 = 0$$

$$-2x + 4y - 4 = -1$$

How does SVM works?

Distance between a point and a line



$$Ax + By + C = 0$$

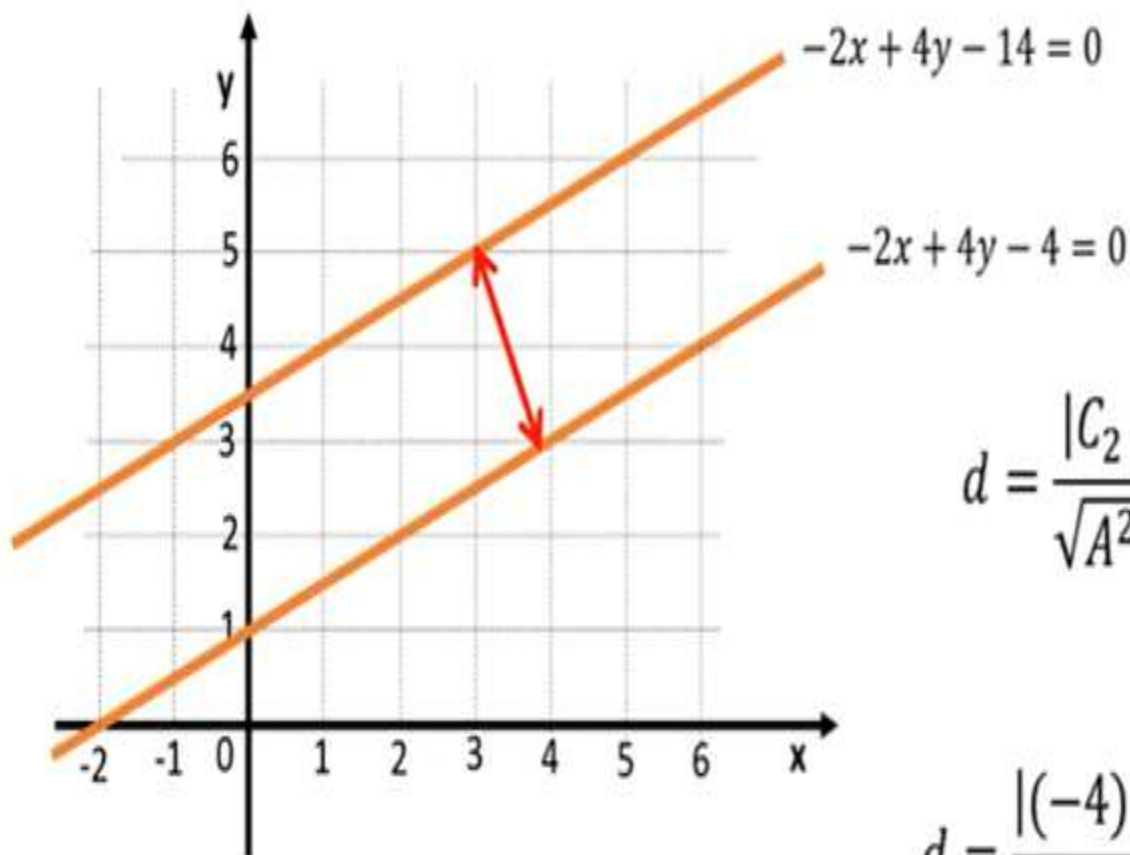
$$-2x + 4y - 4 = 0$$

$$d = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$$

$$d = \frac{|(-2) \cdot 3 + 4 \cdot 5 + (-4)|}{\sqrt{(-2)^2 + 4^2}} = \frac{10}{\sqrt{20}} = 2.236$$

Distance between two parallel lines

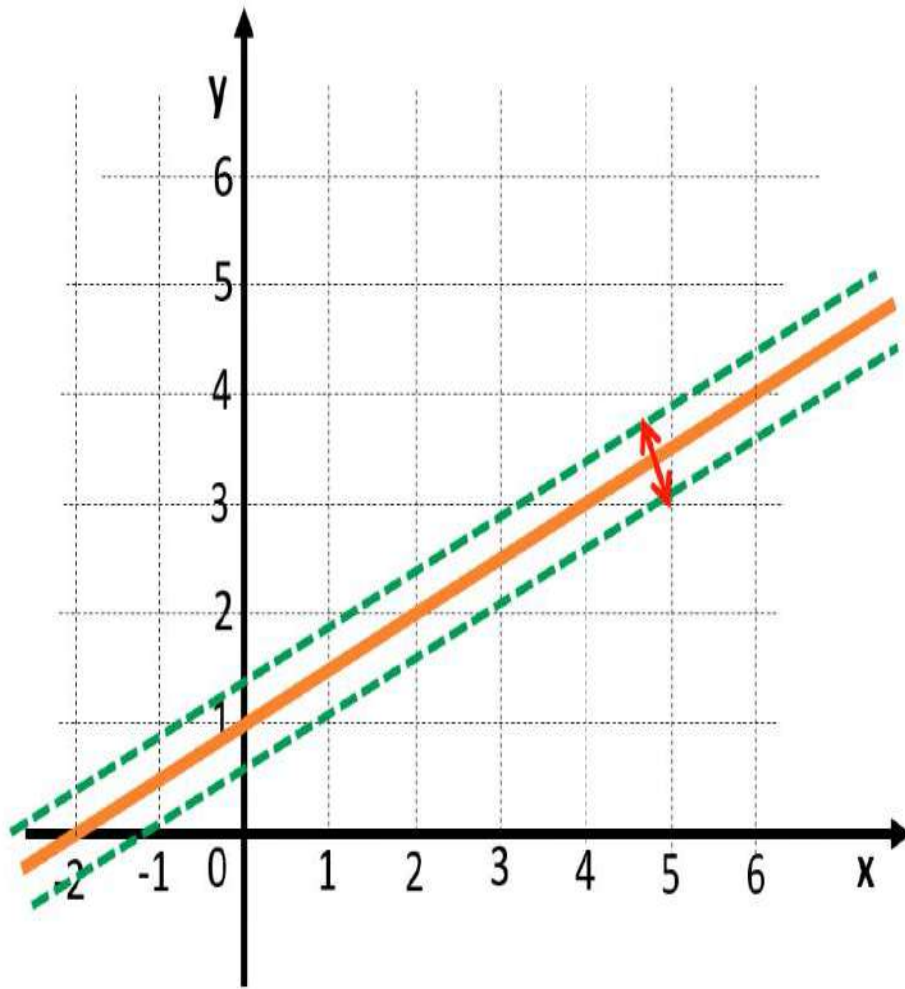
$$Ax + By + C = 0$$



$$d = \frac{|C_2 - C_1|}{\sqrt{A^2 + B^2}}$$

$$d = \frac{|(-4) - (-14)|}{\sqrt{(-2)^2 + 4^2}} = \frac{10}{\sqrt{20}} = \boxed{2.236}$$

How does SVM works?



$$Ax + By + C = 0$$

$$-2x + 4y - 4 = 1$$

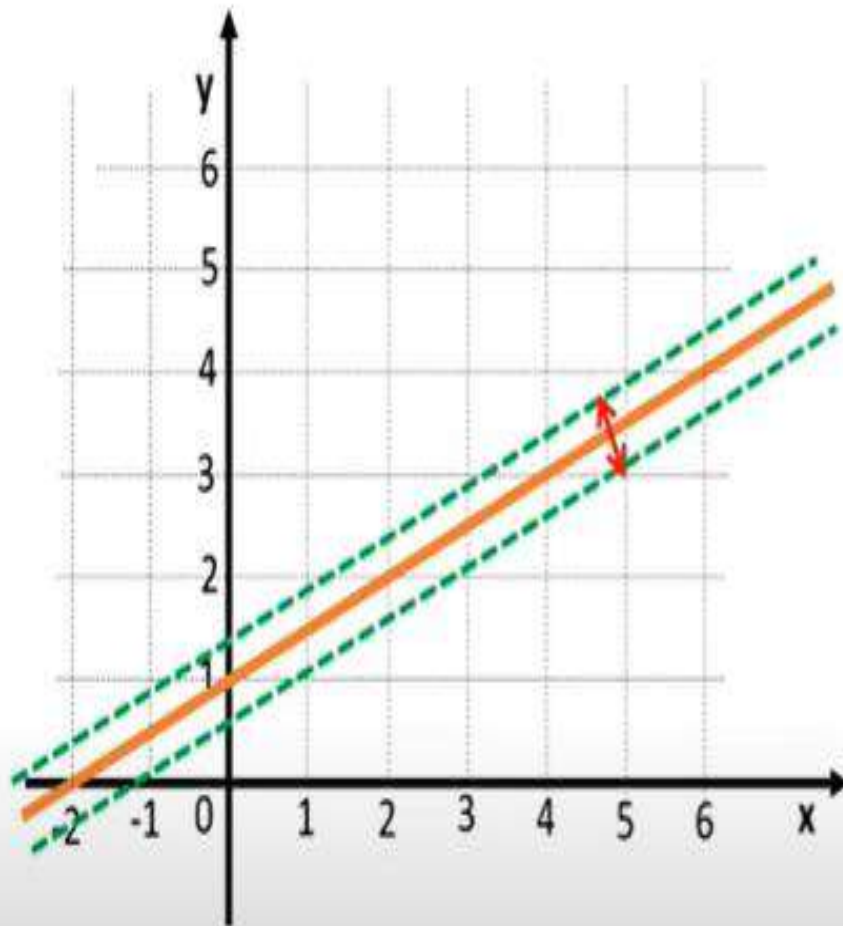
$$-2x + 4y - 4 = 0$$

$$-2x + 4y - 4 = -1$$

$$d = \frac{|C_2 - C_1|}{\sqrt{A^2 + B^2}}$$

$$d = \frac{|(-3) - (-5)|}{\sqrt{(-2)^2 + 4^2}} = \frac{2}{\sqrt{20}} = 0.447$$

Distance between two parallel lines



$$Ax + By + C = 0$$

$$-2x + 4y - 4 = 1$$

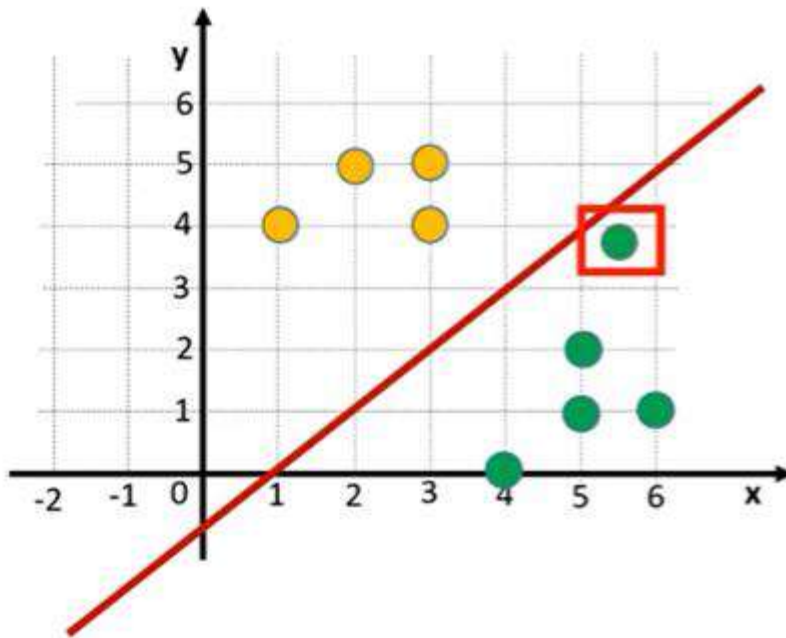
$$-2x + 4y - 4 = 0$$

$$-2x + 4y - 4 = -1$$

$$d = \frac{2}{\sqrt{A^2 + B^2}} = \frac{2}{\|w\|}$$

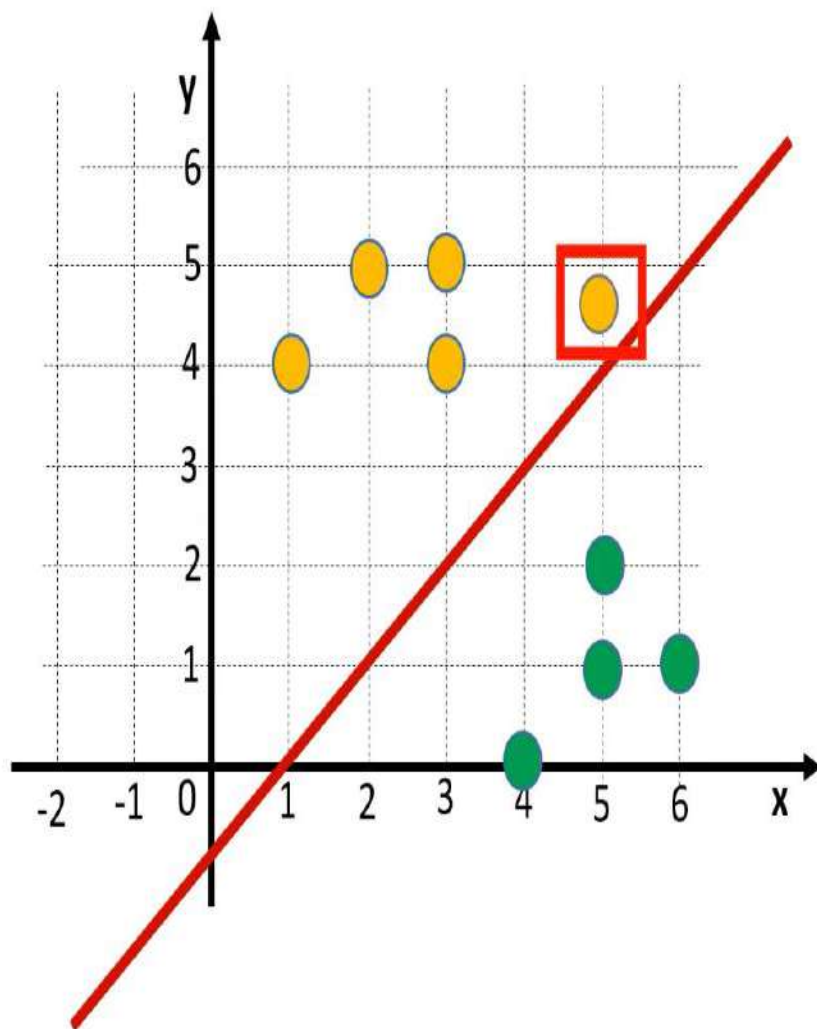
How does SVM works?

Predict



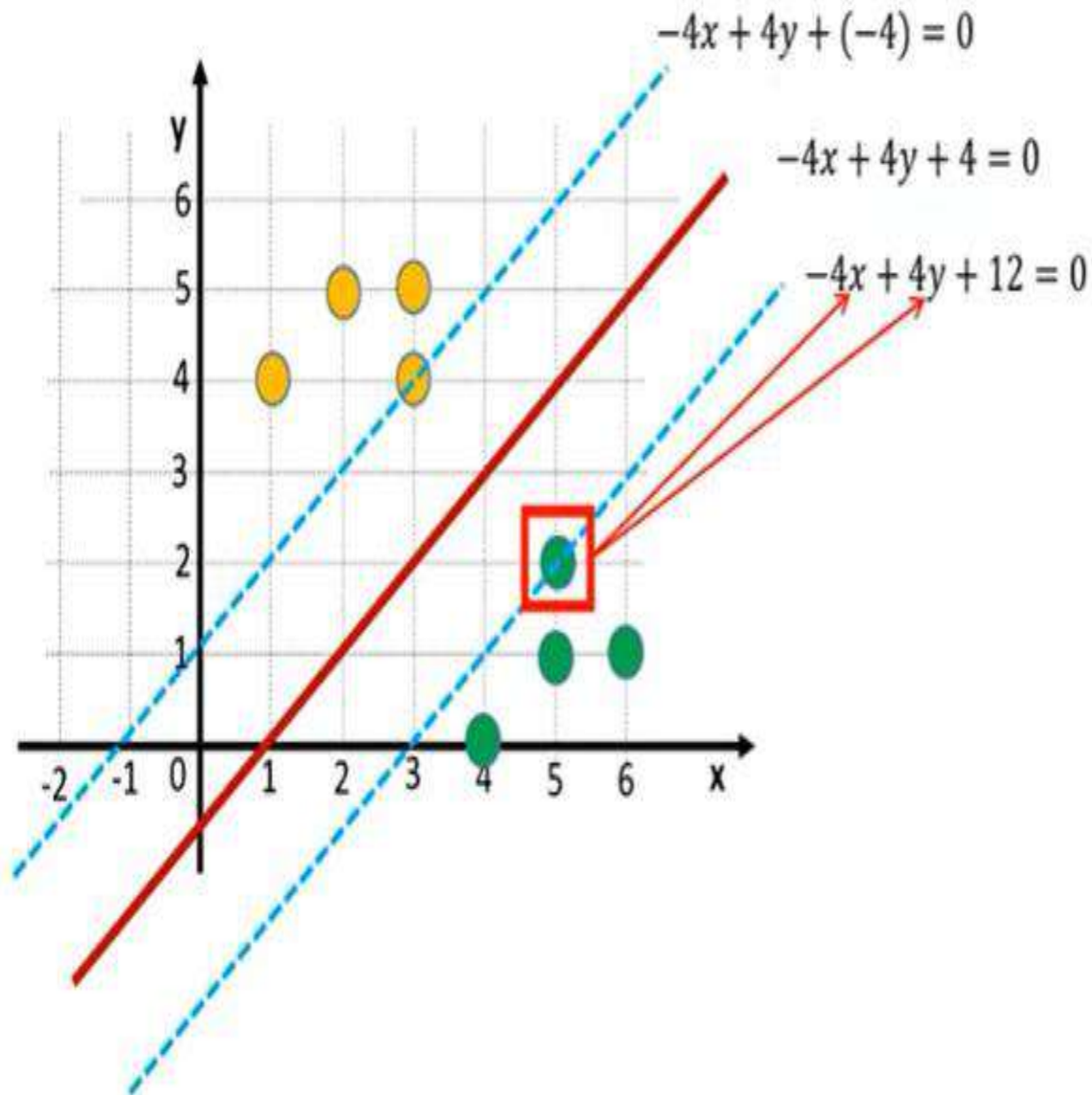
Group	x	y
A	1	4
A	2	5
A	3	5
A	3	4
B	6	1
B	4	0
B	5	2
B	5	1

Predict



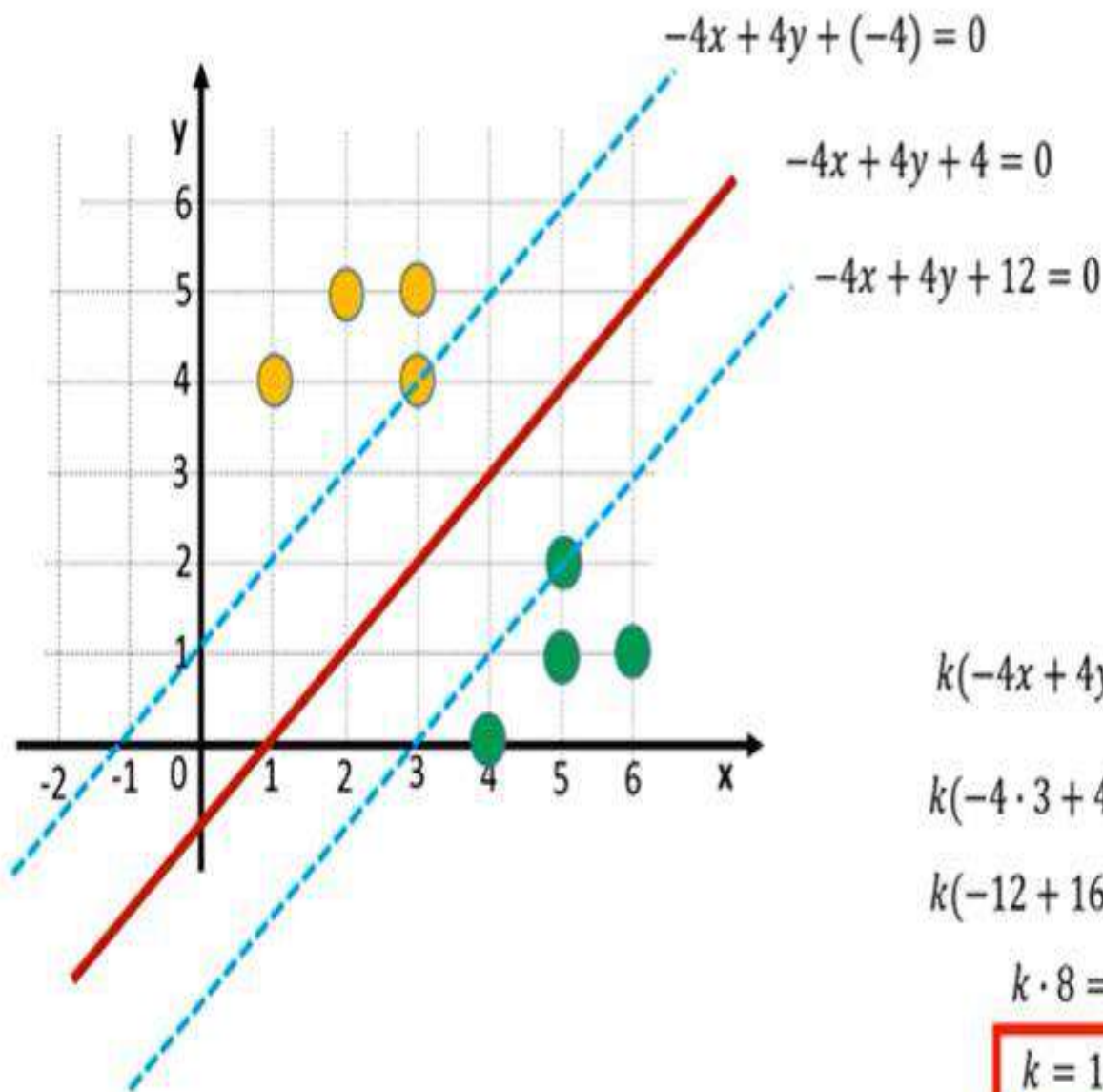
Group	x	y
A	1	4
A	2	5
A	3	5
A	3	4
B	6	1
B	4	0
B	5	2
B	5	1

SVM – the math



Group	x	y
A	1	4
A	2	5
A	3	5
A	3	4
B	6	1
B	4	0
B	5	2
B	5	1

SVM – the math



Group	x	y
A	1	4
A	2	5
A	3	5
A	3	4
B	6	1
B	4	0
B	5	2
B	5	1

$$k(-4x + 4y + 4) = 1$$

$$k(-4 \cdot 3 + 4 \cdot 4 + 4) = 1$$

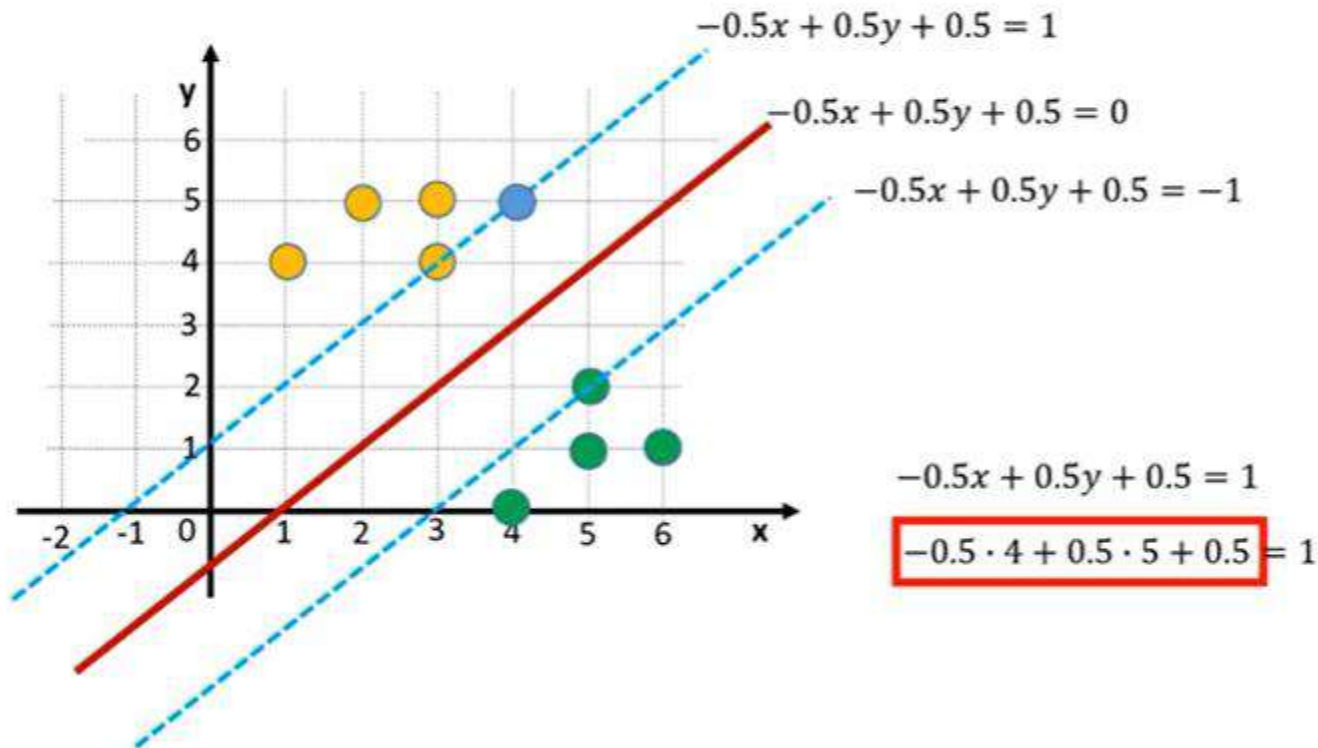
$$k(-12 + 16 + 4) = 1$$

$$k \cdot 8 = 1$$

$$k = 1/8$$

How does SVM works?

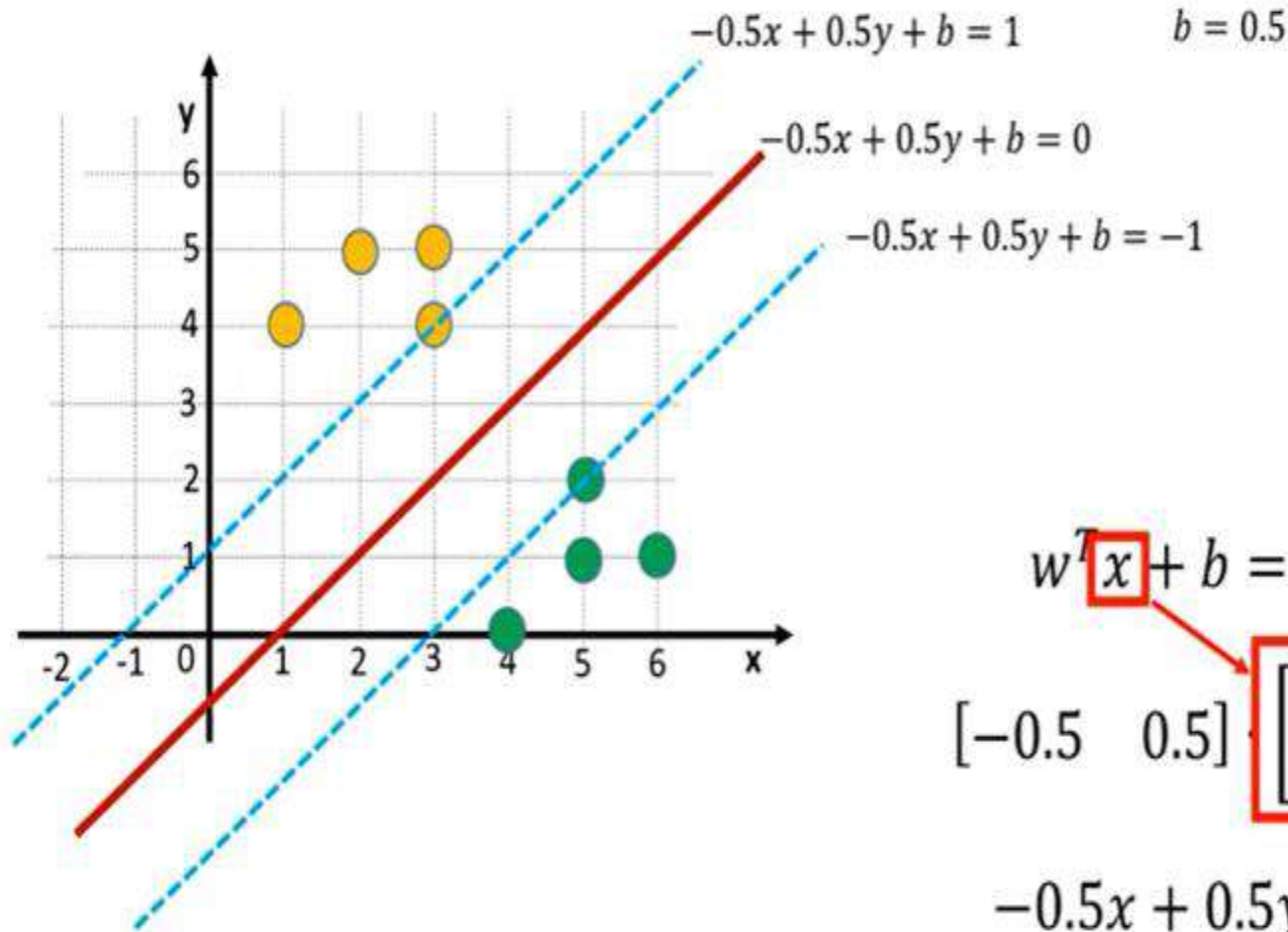
SVM – the math



Group	x	y
A	1	4
A	2	5
A	3	5
A	3	4
B	6	1
B	4	0
B	5	2
B	5	1

How does SVM works?

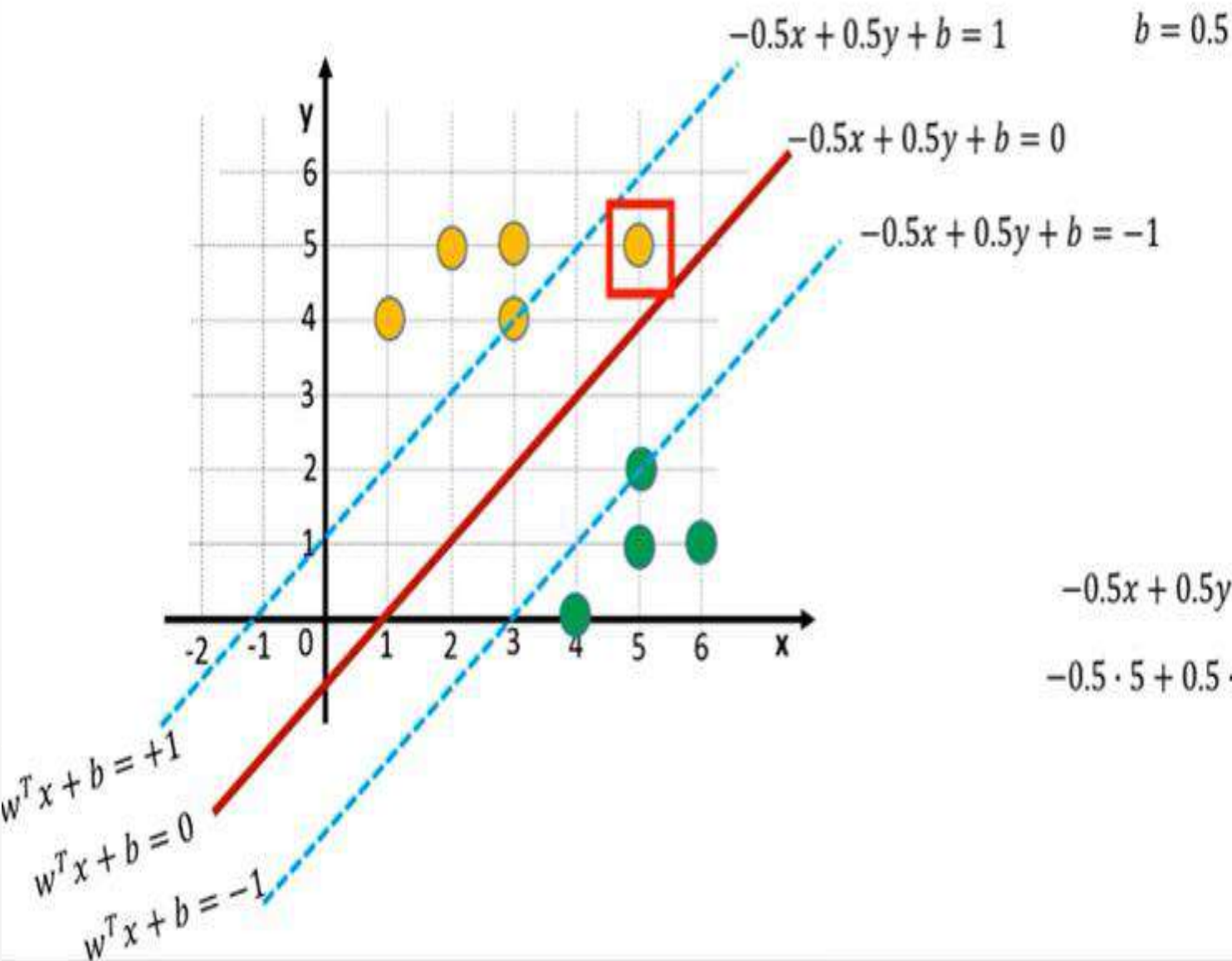
SVM – the math



Group	x	y
A	1	4
A	2	5
A	3	5
A	3	4
B	6	1
B	4	0
B	5	2
B	5	1

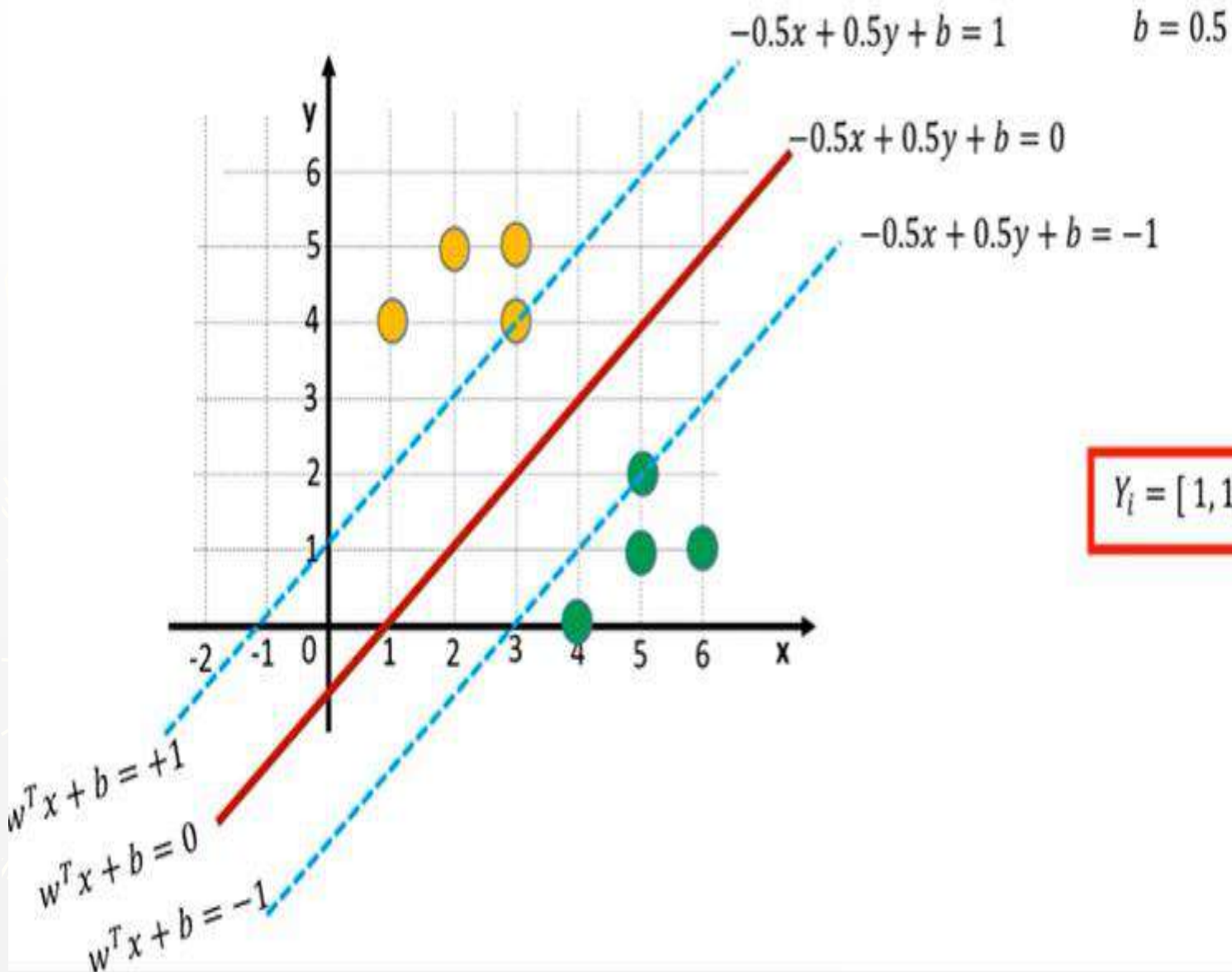
$$w^T \boxed{x} + b = 0$$
$$[-0.5 \quad 0.5] \cdot \boxed{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}} + b = 0$$
$$-0.5x + 0.5y + b = 0$$

SVM – the math



Group	x	y
A	1	4
A	2	5
A	3	5
A	3	4
B	6	1
B	4	0
B	5	2
B	5	1

SVM – the math

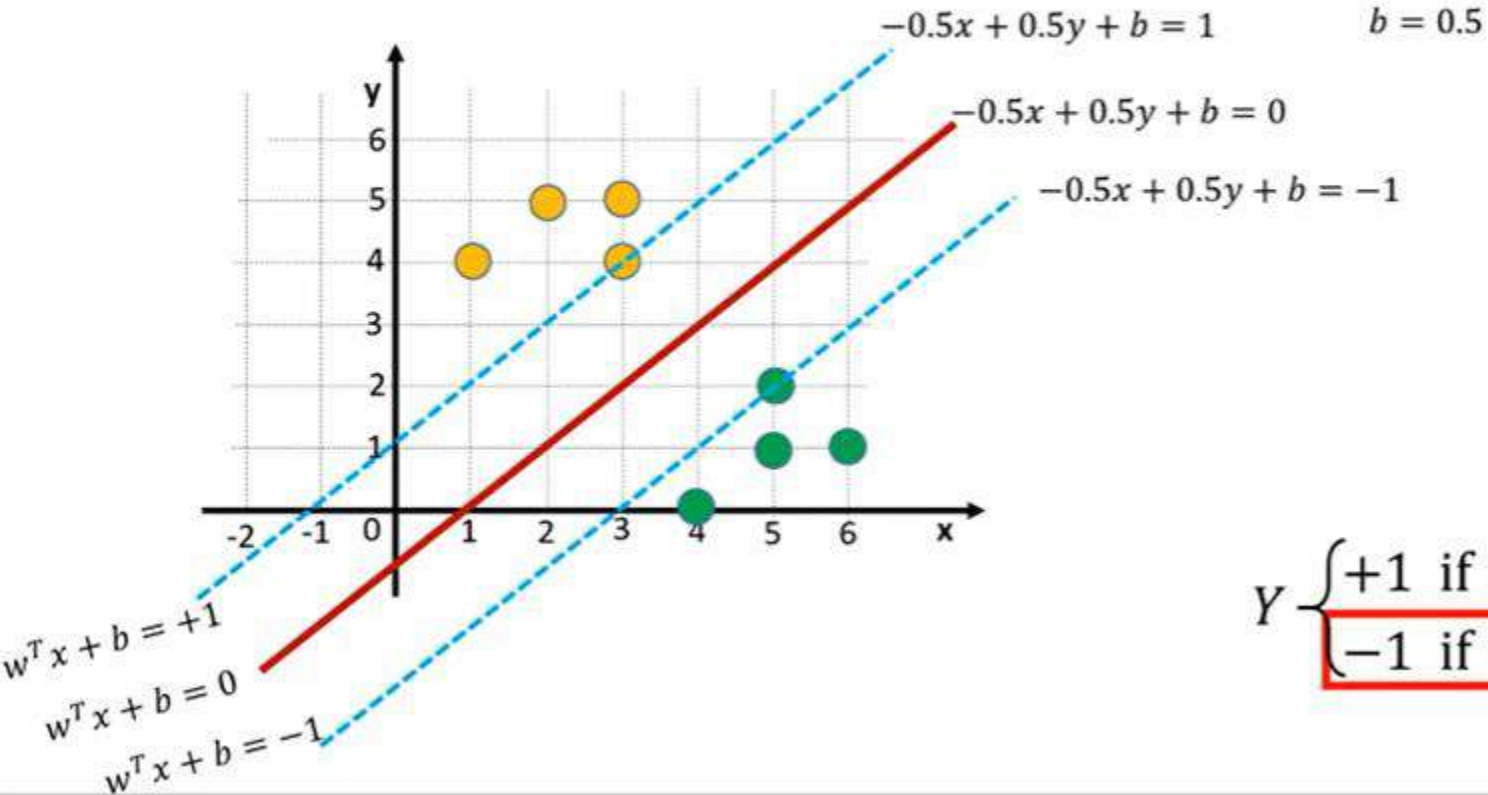


Group (Y_i)	x	y
A (+1)	1	4
A (+1)	2	5
A (+1)	3	5
A (+1)	3	4
B (-1)	6	1
B (-1)	4	0
B (-1)	5	2
B (-1)	5	1

$$Y_i = [1, 1, 1, 1, -1, -1, -1, -1]$$

How does SVM works?

SVM – the math

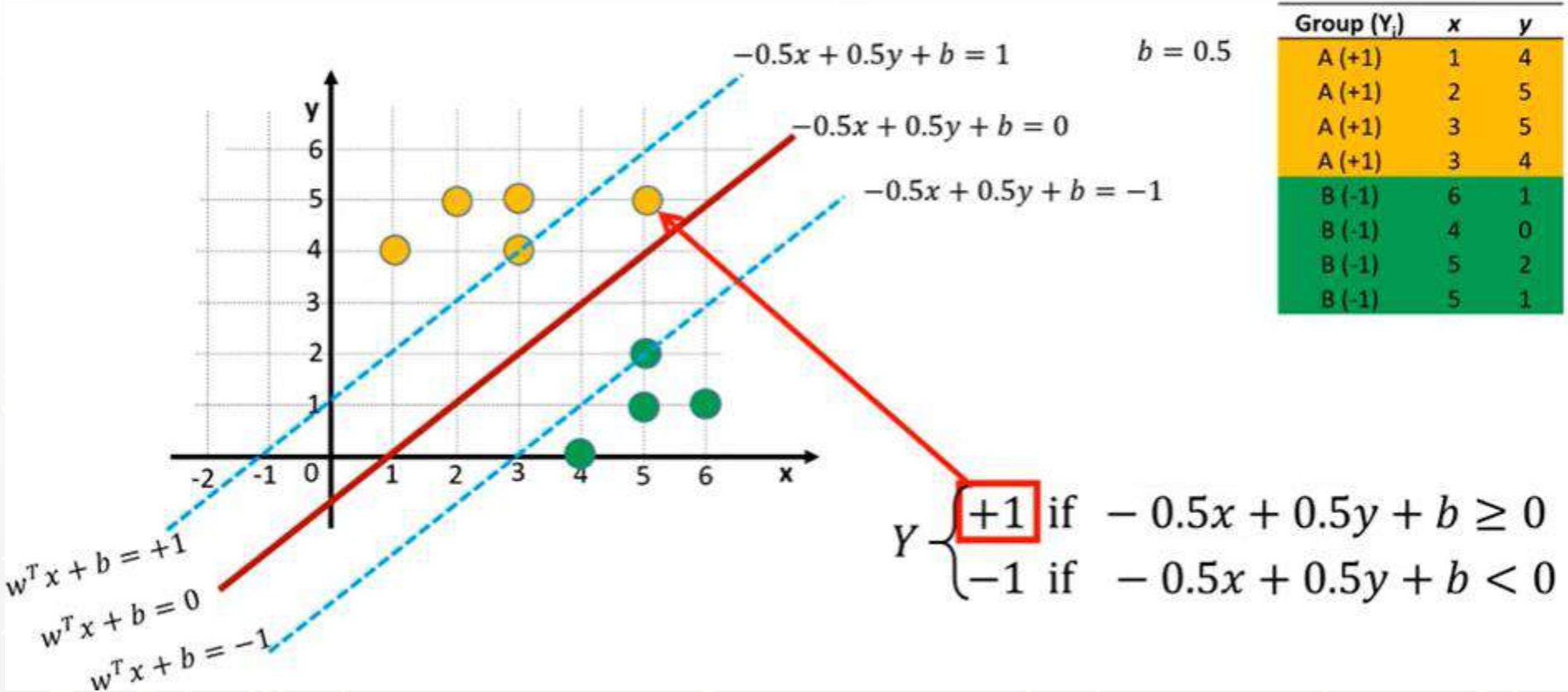


Group (Y_i)	x	y
A (+1)	1	4
A (+1)	2	5
A (+1)	3	5
A (+1)	3	4
B (-1)	6	1
B (-1)	4	0
B (-1)	5	2
B (-1)	5	1

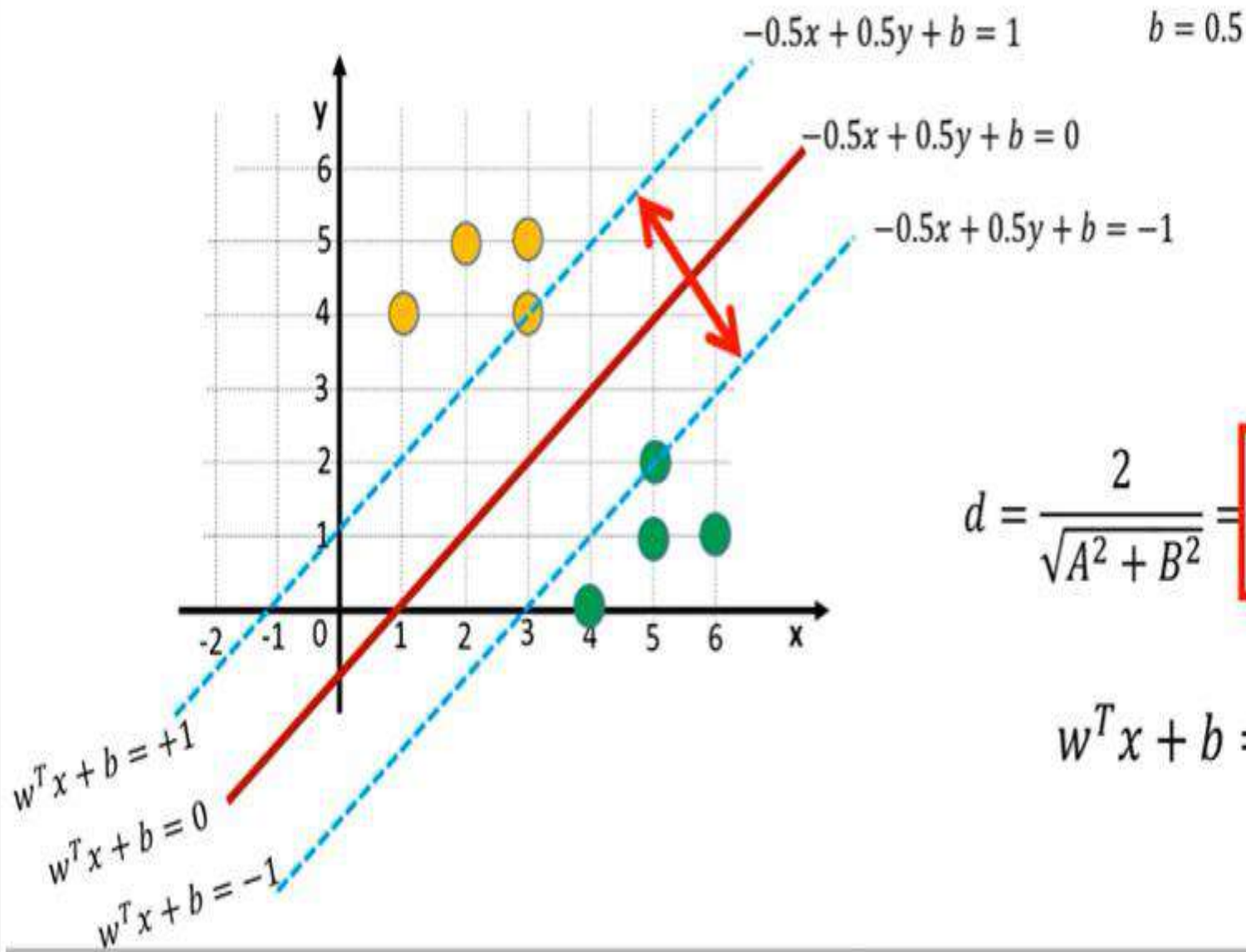
$$Y = \begin{cases} +1 & \text{if } w^T x + b \geq 0 \\ -1 & \text{if } w^T x + b < 0 \end{cases}$$

How does SVM works?

SVM – the math



SVM – the math



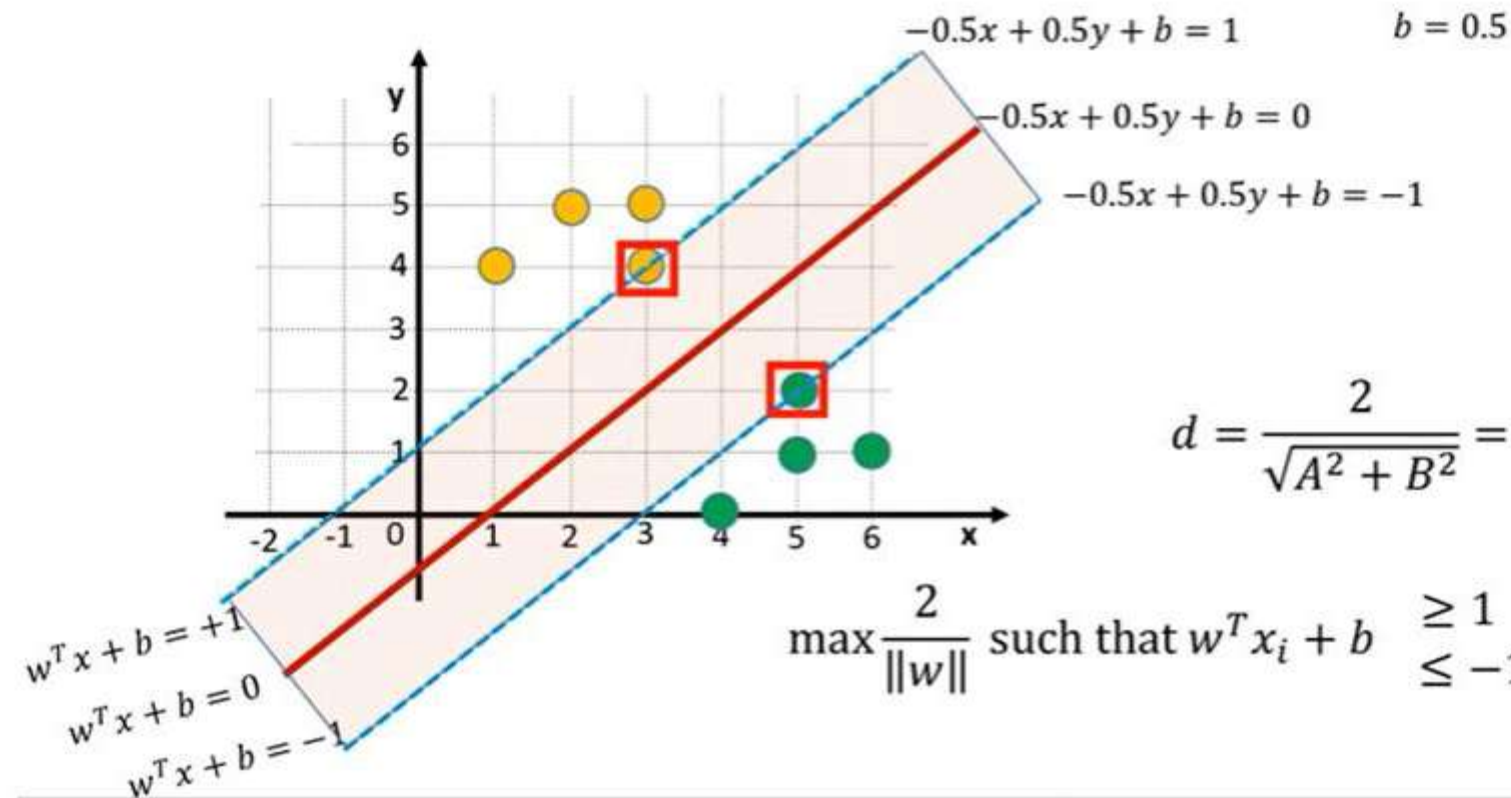
Group (Y_i)	x	y
A (+1)	1	4
A (+1)	2	5
A (+1)	3	5
A (+1)	3	4
B (-1)	6	1
B (-1)	4	0
B (-1)	5	2
B (-1)	5	1

$$d = \frac{2}{\sqrt{A^2 + B^2}} = \frac{2}{\|w\|}$$

$$w^T x + b = 0$$

How does SVM works?

SVM – the math



Group (Y_i)	x	y
A (+1)	1	4
A (+1)	2	5
A (+1)	3	5
A (+1)	3	4
B (-1)	6	1
B (-1)	4	0
B (-1)	5	2
B (-1)	5	1

Example Question

Consider a binary classification problem with the following training dataset:

Point	x_1	x_2	Class (y)
A	2	2	+1
B	4	4	+1
C	1	1	-1
D	3	1	-1

Find the optimal hyperplane equation using a linear SVM and determine the class of a new point $P(3, 3)$.



Step 1: Define the Hyperplane Equation

A linear SVM classifier has the decision boundary:

$$w_1x_1 + w_2x_2 + b = 0$$

where:

- $w = [w_1, w_2]$ is the weight vector,
- b is the bias.

Step 2: SVM Objective

We want a hyperplane: $w_1x_1 + w_2x_2 + b = 0$

such that: $y_i(w \cdot x_i + b) \geq 1$

and margin is maximized: $\text{minimize } \frac{1}{2} ||w||^2$

Step 3: Identify Support Vectors (Critical Step)

We test which points lie closest between classes.

Visually and by distance:

Closest **positive** point: **A(2,2)**

Closest **negative** point: **D(3,1)**

These will act as **support vectors**, so they satisfy:

$$y_i(w \cdot x_i + b) = 1$$

Step 4: Set up equations using support vectors

For A(2,2), $y=+1$: $2w_1 + 2w_2 + b = 1 \quad \dots(1)$

For D(3,1), $y=-1$: $3w_1 + w_2 + b = -1 \quad \dots(2)$

Step 5: Solve for w_1, w_2, b $(2w_1 + 2w_2 + b) - (3w_1 + w_2 + b) = 1 - (-1)$

Subtract (2) from (1):

$$-w_1 + w_2 = 2$$

$$w_2 = w_1 + 2 \quad \dots(3)$$

Step 6 :Use margin maximization

We minimize: $\|w\|^2 = w_1^2 + w_2^2$

Substitute (3): $\|w\|^2 = w_1^2 + (w_1 + 2)^2$

$$= 2w_1^2 + 4w_1 + 4$$

Differentiate:

$$\frac{d}{dw_1} = 4w_1 + 4 = 0$$

$$w_1 = -1$$

Then: $w_2 = w_1 + 2 = 1$

Step 7: Find b

Use equation (1):

$$2(-1) + 2(1) + b = 1$$

$$-2 + 2 + b = 1$$

$$b = 1$$

Step 8. Final Hyperplane

$$-x_1 + x_2 + 1 = 0$$

or equivalently:

$$\boxed{x_2 = x_1 - 1}$$

Step 9. Verify All Constraints

Check all points satisfy:

$$A(2,2): \quad -2 + 2 + 1 = 1 \quad \checkmark$$

$$B(4,4): \quad -4 + 4 + 1 = 1 \geq 1 \quad \checkmark$$

$$C(1,1): \quad -1 + 1 + 1 = 1 \Rightarrow y = -1 \Rightarrow -1 \cdot 1 = -1 \not\geq -1 \quad \times$$

Wait — this reveals something important.

☞ **Point C is also exactly at margin**, meaning it is **also a support vector**.

So actual support vectors are:

- A(2,2)
- C(1,1)
- D(3,1)

This confirms the solution is consistent.

Step 10: Classify new point P(3,3):

$$f(x) = -x_1 + x_2 + 1$$

$$f(3,3) = -3 + 3 + 1 = 1$$

$$y = \text{sign}(1) = +1$$

Final Answer
Optimal Hyperplane

$$-x_1 + x_2 + 1 = 0$$

or

$$x_2 = x_1 - 1$$

Class of P(3,3)

+1

Advantages and Disadvantages of Support Vector Machine Algorithm

Advantages of Support Vector Machine Algorithm

- It has a high level of accuracy
- It works very well with limited datasets
- Kernel SVM contains a non-linear transformation function to convert the complicated non-linearly separable data into linearly separable data
- It is effective on datasets that have multiple features
- It is effective when the number of features are greater than the number of data points
- It employs a subset of training points in the decision function or support vectors, making SVM memory efficient
- Apart from common kernels, it is also possible to specify custom kernels for the decision function

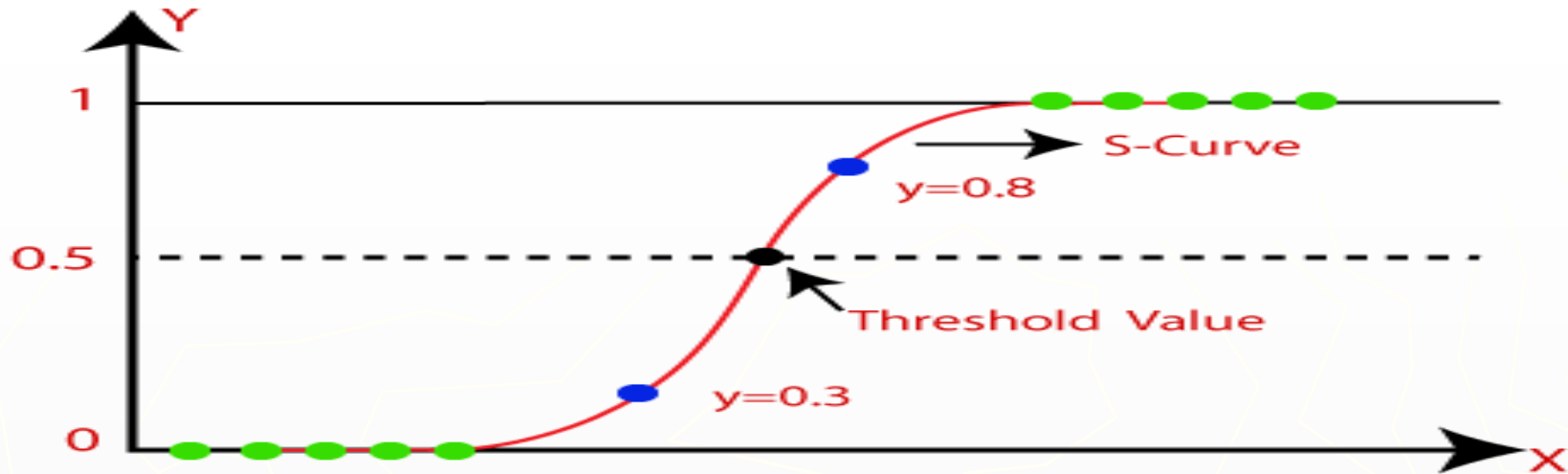
Disadvantages of Support Vector Machine Algorithm

- Does not work well with larger datasets
- Sometimes, training time with SVMs can be high
- If the number of features is significantly greater than the number of data points, it is crucial to avoid overfitting when choosing kernel functions and regularization terms
- Probability estimates are not directly provided by SVMs; rather, they are calculated by using an expensive fivefold cross-validation
- It works best on small sample sets due to its high training time

Logistic Regression Algorithm

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1.**
- Logistic Regression is much similar to the Linear Regression except that how they are used. Linear Regression is used for solving Regression problems, whereas **Logistic regression is used for solving the classification problems.**
- In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).
- The curve from the logistic function indicates the likelihood of something such as whether the cells are cancerous or not, a mouse is obese or not based on its weight, etc.
- Logistic Regression is a significant machine learning algorithm because it has the ability to provide probabilities and classify new data using continuous and discrete datasets.
- Logistic Regression can be used to classify the observations using different types of data and can easily determine the most effective variables used for the classification. The next image is showing the logistic function:

Logistic Regression Algorithm



Logistic Function (Sigmoid Function):

- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1.
- The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form. The S-form curve is called the Sigmoid function or the logistic function.
- In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Logistic Regression Algorithm

Assumptions for Logistic Regression:

- The dependent variable must be categorical in nature.
- The independent variable should not have multi-collinearity

Logistic Regression Equation:

The Logistic regression equation can be obtained from the Linear Regression equation. The mathematical steps to get Logistic Regression equations are given below:

- We know the equation of the straight line can be written as:

$$y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

In Logistic Regression y can be between 0 and 1 only, so for this let's divide the above equation by $(1-y)$:

$$\frac{y}{1-y} ; 0 \text{ for } y=0, \text{ and infinity for } y=1$$

But we need range between $-\text{[infinity]}$ to $+\text{[infinity]}$, then take logarithm of the equation it will become:

$$\log \left[\frac{y}{1-y} \right] = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

The above equation is the final equation for Logistic Regression.

Supervised Learning: **Logistic Linear Regression**

- **Independent variables:** The input characteristics or predictor factors applied to the dependent variable's predictions.
- **Dependent variable:** The target variable in a logistic regression model, which we are trying to predict.
- **Logistic function:** The formula used to represent how the independent and dependent variables relate to one another. The logistic function transforms the input variables into a probability value between 0 and 1, which represents the likelihood of the dependent variable being 1 or 0.
- **Odds:** It is the ratio of something occurring to something not occurring. It is different from probability as the probability is the ratio of something occurring to everything that could possibly occur.
- **Log-odds:** The log-odds, also known as the logit function, is the natural logarithm of the odds. In logistic regression, the log odds of the dependent variable are modeled as a linear combination of the independent variables and the intercept.
- **Coefficient:** The logistic regression model's estimated parameters, show how the independent and dependent variables relate to one another.
- **Intercept:** A constant term in the logistic regression model, which represents the log odds when all independent variables are equal to zero.
- **Maximum likelihood estimation:** The method used to estimate the coefficients of the logistic regression model, which maximizes the likelihood of observing the data given the model.

Logistic Regression Algorithm

Type of Logistic Regression:

On the basis of the categories, Logistic Regression can be classified into three types:

Binomial: In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.

Multinomial: In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep"

Ordinal: In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High".

◆ Model Representation

$$P(y = 1|x) = \frac{1}{1+e^{-(w_0+w_1x_1+\dots+w_nx_n)}}$$

Where:

- $P(y = 1|x)$ = probability that output is 1
- w_0 = bias (intercept)
- $w_1, w_2, \dots, w_n$ = weights
- $x_1, x_2, \dots, x_n$ = input features

Decision Rule

- If $P \geq 0.5 \rightarrow \text{Class} = 1$
- If $P < 0.5 \rightarrow \text{Class} = 0$

Key Idea

- It uses a **linear combination** of inputs
- Then applies a **sigmoid function**
- Output becomes a **probability (0 to 1)**

Logistic Regression Algorithm

- The dataset of pass or fail in an exam of 5 students is given in the table.
 - Use logistic regression as classifier to answer the following questions.
1. Calculate the probability of pass for the student who studied 33 hours.
 2. At least how many hours student should study that makes he will pass the course with the probability of more than 95%.

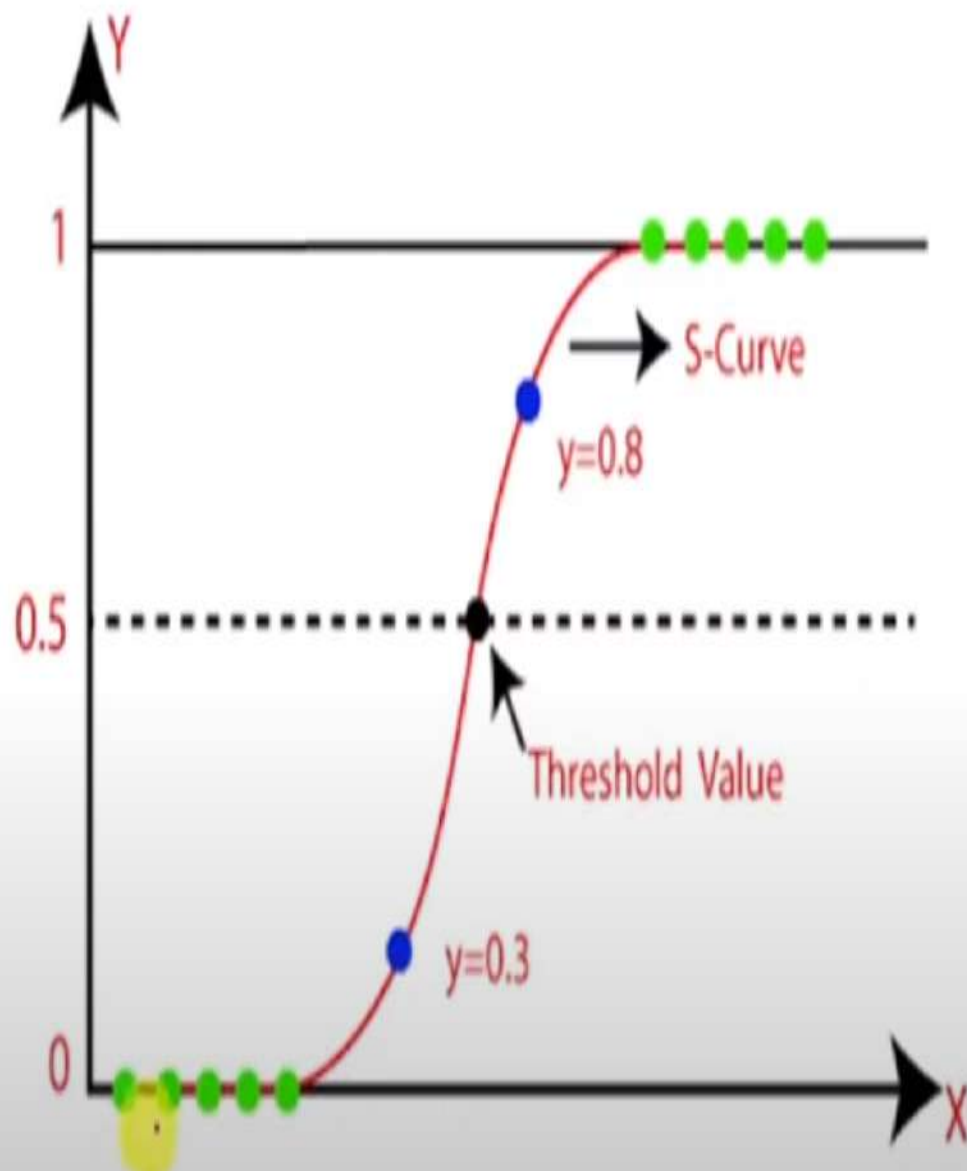
Hours Study	Pass (1) / Fail (0)
29	0
15	0
33	1
28	1
39	1

Assume the model suggested by the optimizer for odds of passing the course is,

$$\log(\text{odds}) = -64 + 2 * \text{hours}$$

- We use Sigmoid Function in logistic regression

- $$s(x) = \frac{1}{1+e^{-x}}$$



Logistic Regression Algorithm(Example)

1. Calculate the probability of pass for the student who studied 33 hours.

$$p = \frac{1}{1+e^{-z}} \quad s(x) = \frac{1}{1+e^{-x}}$$

$$z = -64 + 2 * 33 = -64 + 66 = 2$$

$$p = \frac{1}{1+e^{-2}} = 0.88$$

That is, if student studies 33 hours, then there is **88% chance** that the student will pass the exam

Hours Study	Pass (1) / Fail (0)
29	0
15	0
33	1
28	1
39	1

$$\log(\text{odds}) = z = -64 + 2 * \text{hours}$$

Logistic Regression Algorithm(Example)

2. At least how many hours student should study that makes he will pass the course with the probability of more than 95%.

Hours Study	Pass (1) / Fail (0)
29	0
15	0
33	1
28	1
39	1

$$p = \frac{1}{1+e^{-z}} = 0.95$$

$$0.95 * (1 + e^{-z}) = 1$$

$$0.95 * e^{-z} = 1 - 0.95$$

$$e^{-z} = \frac{0.05}{0.95} = 0.0526$$

$$\ln(e^{-z}) = \ln(0.0526)$$

$$\ln(e^x) = x$$

$$-z = \ln(0.0526) = -2.94$$

$$z = 2.94$$

Logistic Regression Algorithm(Example)

- $z = 2.94$
- $\log(\text{odds}) = z = -64 + 2 * \text{hours}$
- $2.94 = -64 + 2 * \text{hours}$
- $2 * \text{hours} = \underline{2.94 + 64}$
- $2 * \text{hours} = \underline{66.94}$
- $\text{hours} = \frac{66.94}{2}$
- $\text{hours} = 33.47 \text{ Hours}$

Hours Study	Pass (1) / Fail (0)
29	0
15	0
33	1
28	1
39	1

- The student should study **at least 33.47 hours**, so that he will pass the exam with more than 95% probability

Logistic Regression Algorithm(Example)

You are given the following dataset:

x1 (Hours Study)	x2 (Attendance %)	(y) (Pass=1, Fail=0)
1	50	0
2	60	0
3	65	0
4	70	1
5	80	1
6	85	1

A logistic regression model is trained as:

A logistic regression model is trained as:

$$P(y = 1) = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + w_2 x_2)}}$$

Questions:

Given:

- $w_0 = -10$
- $w_1 = 1.2$
- $w_2 = 0.1$

**a) Compute the probability for a student with:
 $x_1=3, x_2=65$**

**b) Classify the student using threshold = 0.5
Determine whether the point
($x_1=2, x_2=60$) lies above or below the decision
boundary.**

Logistic Regression Algorithm(Example)

◆ Solution

✓ (a) Probability Calculation

$$z = w_0 + w_1x_1 + w_2x_2$$

$$z = -10 + (1.2 \times 3) + (0.1 \times 65)$$

$$z = -10 + 3.6 + 6.5 = 0.1$$

$$P(y = 1) = \frac{1}{1 + e^{-0.1}} \approx 0.525$$

✓ (b) Classification

- $0.525 > 0.5$

→ Class = 1 (Pass)

✓ (c) Decision Boundary Check

Decision boundary:

For (2,60):

$$z = -10 + (1.2 \times 2) + (0.1 \times 60)$$

$$z = -10 + 2.4 + 6 = -1.6$$

Final Answers

- (a) Probability $\approx$ **0.525**
- (b) Class = **Pass (1)**
- (c) Point is **below decision boundary**

$$w_0 + w_1x_1 + w_2x_2 = 0$$

- Since $z < 0$

→ Probability < 0.5

→ Point lies below decision boundary

Supervised Learning: **Logistic Linear Regression**

Advantages

- ✓ Simple and easy to implement
- ✓ Works well for linearly separable data
- ✓ Outputs probability
- ✓ Efficient for large datasets

Disadvantages

- ✗ Cannot handle complex non-linear relationships
- ✗ Sensitive to outliers
- ✗ Assumes linear decision boundary

Applications

Spam detection

Disease prediction

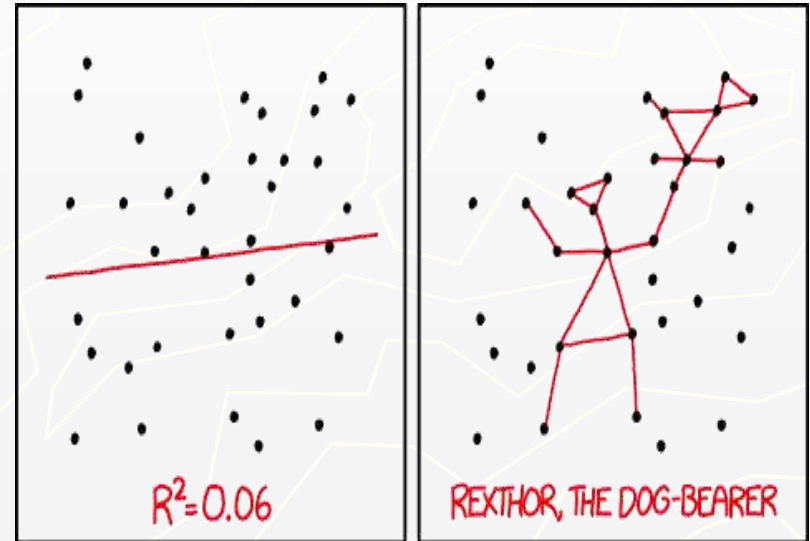
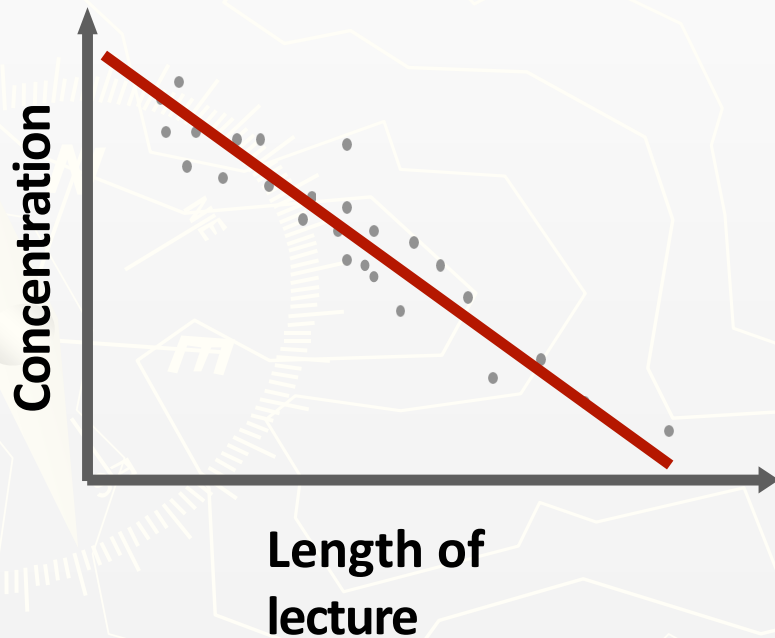
Credit risk analysis

Customer churn prediction

Supervised Learning: Regression

Regression: Regression algorithms are used to solve regression problems in which there is a linear relationship between input and output variables. These are used to predict continuous output variables, such as market trends, weather prediction, etc. Find the relationship between one dependent variable and a series of other changing variables. Some popular Regression algorithms are given below:

Simple Linear Regression Algorithm, Multiple Linear Regression Algorithm, Decision Tree Algorithm, Lasso Regression



I DON'T TRUST LINEAR REGRESSIONS WHEN IT'S HARDER TO GUESS THE DIRECTION OF THE CORRELATION FROM THE SCATTER PLOT THAN TO FIND NEW CONSTELLATIONS ON IT.

Supervised Learning: **Simple Linear Regression**

- Simple Linear Regression is a type of Regression algorithms that models the relationship between a dependent variable and a single independent variable. The relationship shown by a Simple Linear Regression model is linear or a sloped straight line, hence it is called Simple Linear Regression.
- The key point in Simple Linear Regression is that the dependent variable must be a continuous/real value. However, the independent variable can be measured on continuous or categorical values.
- Simple Linear regression algorithm has mainly three objectives:
- **Relationship Intensity:** Determining how strongly one variable influences another. Such as the relationship between Income and expenditure, etc.
- **Forecasting new observations.** Such as Weather forecasting according to temperature, Revenue of a company according to the investments in a year, etc.
- **Trend Analysis:** Analyzing how the dependent variable changes over time or across varying independent variables.
- **Simple Linear Regression Model:**

$$y = a_0 + a_1x + \epsilon$$

The Simple Linear Regression model can be represented using the above equation:

a_0 (Intercept) = It is the intercept of the Regression line (can be obtained putting $x=0$)

a_1 = It is the slope of the regression line, which tells whether the line is increasing or decreasing. **ϵ (Residuals)** = The error term. The difference between the observed value and the predicted value (For a good model it will be negligible)

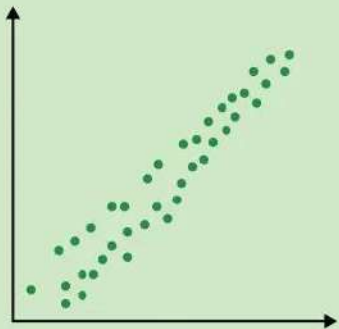
Supervised Learning: Simple Linear Regression

Assumptions

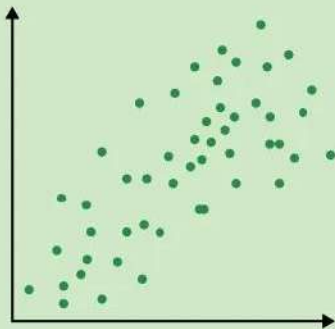
For the model to be valid, the following must be checked:

- **Linearity:** The relationship between inputs (X) and the output (Y) is a straight line
- **Independence of Errors:** The errors in predictions should not affect each other.(checked with Durbin-Watson statistic).
- **Constant Variance (Homoscedasticity):** The errors should have equal spread across all values of the input. If the spread changes (like fans out or shrinks), it's called heteroscedasticity and it's a problem for the model.
- **Normality of Errors:** The errors should follow a normal (bell-shaped) distribution.
- **No Autocorrelation:** Errors shouldn't show repeating patterns, especially in time-based data.
- **Additivity:** The total effect on Y is just the sum of effects from each X, no mixing or interaction between them.

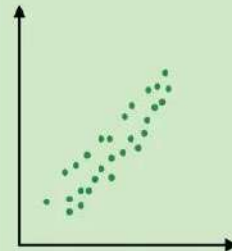
Homoscedasticity



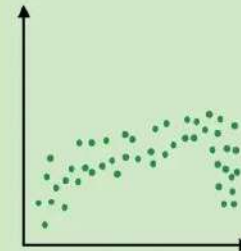
Heteroscedasticity



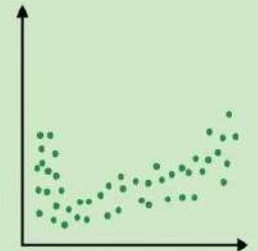
Linear



Non-Linear



Non-Linear



Supervised Learning: Simple Linear Regression

For example we want to predict a student's exam score based on how many hours they studied. We observe that as students study more hours, their scores go up. In the example of predicting exam scores based on hours studied. Here

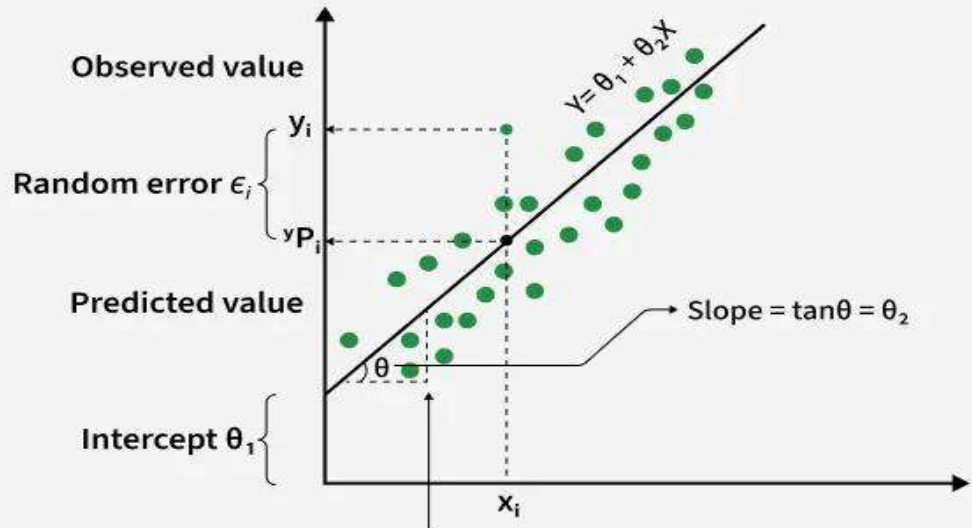
Independent variable (input): Hours studied because it's the factor we control or observe.

Dependent variable (output): Exam score because it depends on how many hours were studied.

Best Fit Line in Linear Regression

In linear regression, the best-fit line is the straight line that most accurately represents the relationship between the independent variable (input) and the dependent variable (output). It is the line that minimizes the difference between the actual data points and the predicted values from the model.

We use the independent variable to predict the dependent variable



1. Goal of the Best-Fit Line

The goal of linear regression is to find a straight line that minimizes the error (the difference) between the observed data points and the predicted values. This line helps us predict the dependent variable for new, unseen data.

Here Y is called a dependent or target variable and X is called an independent variable also known as the predictor of Y.

- θ_1 represents the intercept, which is the value of Y when $X = 0$
- θ_2 represents the slope, which shows how much Y changes for a unit change in X

Supervised Learning: Simple Linear Regression

2.Minimizing the Error: The Least Squares Method

To find the best-fit line, we use a method called [Least Squares](#). The idea behind this method is to minimize the sum of squared differences between the actual values (data points) and the predicted values from the line. These differences are called residuals.

The formula for residuals is: $Residual = y_i - \hat{y}_i$

- y_i is the actual observed value
- $\hat{y}_i$ is the predicted value from the line for that x_i

The least squares method minimizes the sum of the squared residuals:

$$\Sigma(y_i - \hat{y}_i)^2$$

This method ensures that the line best represents the data where the sum of the squared differences between the predicted values and actual values is as small as possible.

Regularization Techniques for Linear Models

- [Lasso Regression](#): Regularizes a linear regression model, it adds a penalty term to the linear regression objective function to prevent overfitting.
- [Ridge regression](#): Adds a regularization term to the standard linear objective to prevent overfitting by penalizing large coefficient in linear regression equation. It useful when the dataset has multicollinearity where predictor variables are highly correlated.
- [Elastic Net Regression](#): Hybrid regularization technique that combines the power of both L1 and L2 regularization in linear regression objective.

Supervised Learning: **Simple Linear Regression**

Sr. No	Linear Regression	Logistic Regression
1	Linear regression is used to predict the continuous dependent variable using a given set of independent variables.	Logistic Regression is used to predict the categorical dependent variable using a given set of independent variables.
2	Linear regression is used for solving Regression problem.	It is used for solving classification problems.
3	In this we predict the value of continuous variables	In this we predict values of categorical variables
4	In this we find best fit line.	In this we find S-Curve .
5	Least square estimation method is used for estimation of accuracy.	Maximum likelihood estimation method is used for Estimation of accuracy.
6	The output must be continuous value,such as price,age,etc.	Output is must be categorical value such as 0 or 1, Yes or no, etc.
7	It required linear relationship between dependent and independent variables.	It not required linear relationship.
8	There may be collinearity between the independent variables.	There should not be collinearity between independent variable.

Supervised Learning: **Simple Linear Regression**

Why Linear Regression is Important?

Here's why linear regression is important:

- It's easy to understand and interpret, making it a starting point for learning about machine learning.
- Helps predict future outcomes based on past data, making it useful in various fields like finance, healthcare and marketing.
- Many advanced algorithms, like logistic regression or neural networks, build on the concepts of linear regression.
- It's computationally efficient and works well for problems with a linear relationship.
- It's one of the most widely used techniques in both statistics and machine learning for regression tasks.
- It provides insights into relationships between variables (e.g., how much one variable influences another).

Advantages

- Simple and easy to implement with interpretable coefficients.
- Computationally efficient, suitable for large datasets and real-time applications.
- Relatively robust to outliers.
- Serves as a good baseline model.
- Widely available in ML libraries and software.

Limitations

- Assumes a linear relationship between variables.
- Sensitive to multicollinearity among features.
- Requires proper feature engineering.
- Can overfit or underfit depending on data.
- Limited for modeling complex relationships.

Supervised Learning: Simple Linear Regression

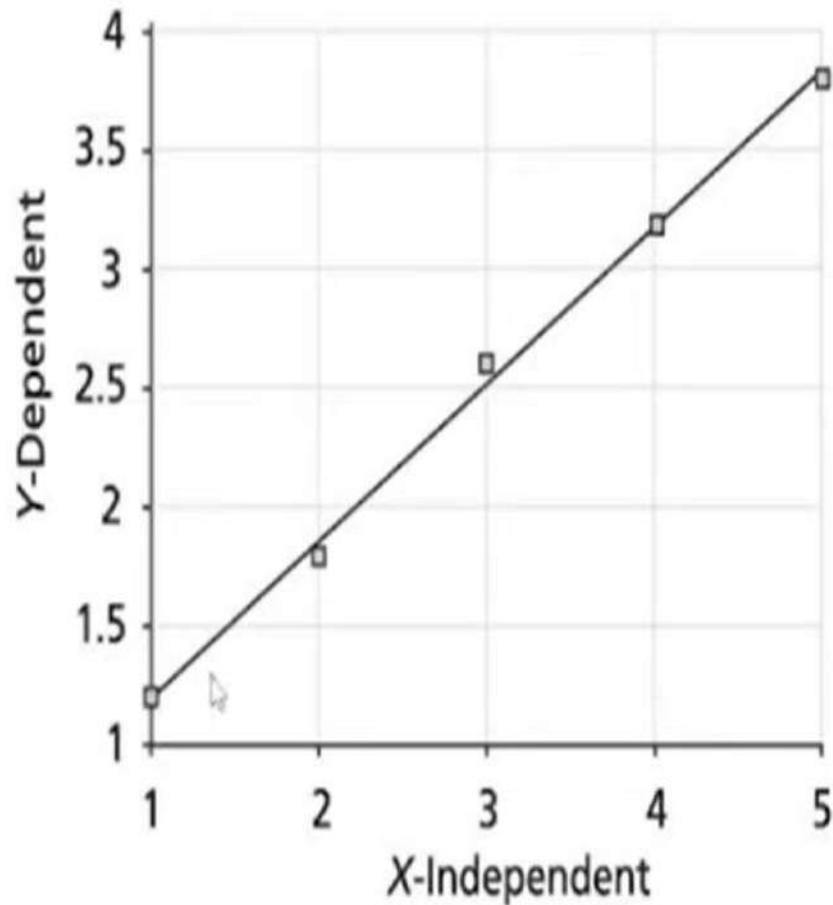
Linear Regression – Solved Example

- Let us consider an example where the five weeks' sales data (in Thousands) is given as shown in Table.
- Apply linear regression technique to predict the 7th and 12th week sales.

x_i (Week)	y_j (Sales in Thousands)
1	1.2
2	1.8
3	2.6
4	3.2
5	3.8

Supervised Learning: Simple Linear Regression

Step 1: Model Formula



x_i (Week)	y_j (Sales in Thousands)
1	1.2
2	1.8
3	2.6
4	3.2
5	3.8

Supervised Learning: Simple Linear Regression

Step 2: Calculate Slope (a1)

- Linear regression equation is given by

- $y = a_0 + a_1 * x + e$

- where

- $a_1 = \frac{(\overline{xy}) - (\bar{x})(\bar{y})}{\overline{x^2} - \bar{x}^2}$

- $a_0 = \bar{y} - a_1 * \bar{x}$

$$m = \frac{n \sum XY - (\sum X)(\sum Y)}{n \sum X^2 - (\sum X)^2}$$

x_i (Week)	y_j (Sales in Thousands)
1	1.2
2	1.8
3	2.6
4	3.2
5	3.8

Now:

- $\sum X = 15$
- $\sum Y = 12.6$
- $\sum XY = 44.4$
- $\sum X^2 = 55$
- $n = 5$

- Here, there are 5 items, i.e., $i = 1, 2, 3, 4, 5$.

	x_i (Week)	y_j (Sales in Thousands)	x_i^2	$x_i * y_j$
	1	1.2	1	1.2
	2	1.8	4	3.6
	3	2.6	9	7.8
	4	3.2	16	12.8
	5	3.8	25	19
Sum	15	12.6	55	44.4
Average	$\bar{x} = 3$	$\bar{y} = 2.52$	$\overline{x^2} = 11$	$\overline{xy} = 8.88$

Supervised Learning: Simple Linear Regression

Step 3: Calculate Intercept (a0)

$$\bullet \quad \bar{x} = 3 \quad \bar{y} = 2.52 \quad \bar{x^2} = 11 \quad \overline{xy} = 8.88$$

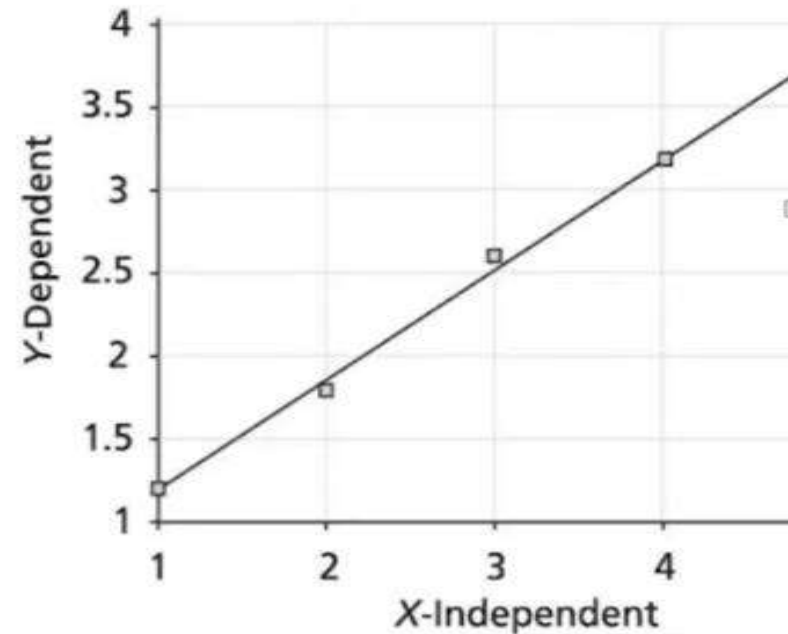
$$\bullet \quad a_1 = \frac{(\overline{xy}) - (\bar{x})(\bar{y})}{\bar{x^2} - \bar{x}^2} = \frac{8.88 - 3 \times 2.52}{11 - 3^2} = 0.66$$

$$\bullet \quad a_0 = \bar{y} - a_1 * \bar{x} = 2.52 - 0.66 * 3 = 0.54$$

• Regression equation is

$$\bullet \quad y = a_0 + a_1 * x$$

$$\bullet \quad y = 0.54 + 0.66 * x$$



$$b = \frac{n \sum XY - \sum X \sum Y}{n \sum X^2 - (\sum X)^2}$$

$$b = \frac{5 \times 44.4 - 15 \times 12.6}{5 \times 55 - 225} = \frac{222 - 189}{275 - 225} = \frac{33}{50} = 0.66$$

$$a = \frac{\sum Y - b \sum X}{n}$$

$$a = \frac{12.6 - (0.66 \times 15)}{5} = \frac{12.6 - 9.9}{5} = \frac{2.7}{5} = 0.54$$

Supervised Learning: Simple Linear Regression

Step 4: Regression Equation

- Regression equation is
- $y = a_0 + a_1 * x$
- $y = 0.54 + 0.66 * x$

Step 5: Predictions

- ✓ For 7th week:

$$y = 0.66(7) + 0.54 = 4.62 + 0.54 = 5.16$$

- ✓ For 12th week:

$$y = 0.66(12) + 0.54 = 7.92 + 0.54 = 8.46$$

◆ Final Answer

- Regression line: $y = 0.66x + 0.54$
- 7th week sales: 5.16 thousand
- 12th week sales: 8.46 thousand

Supervised Learning: **Multiple Linear Regression**

- In the previous topic, we have learned about Simple Linear Regression, where a single Independent/Predictor(X) variable is used to model the response variable (Y). But there may be various cases in which the response variable is affected by more than one predictor variable; for such cases, the Multiple Linear Regression algorithm is used.
- Moreover, Multiple Linear Regression is an extension of Simple Linear regression as it takes more than one predictor variable to predict the response variable. We can define it as:
- Multiple Linear Regression is one of the important regression algorithms which models the linear relationship between a single dependent continuous variable and more than one independent variable.
- **Example:**
- Prediction of CO₂ emission based on engine size and number of cylinders in a car.

Some key points about MLR:

- For MLR, the dependent or target variable(Y) must be the continuous/real, but the predictor or independent variable may be of continuous or categorical form.
- Each feature variable must model the linear relationship with the dependent variable.
- MLR tries to fit a regression line through a multidimensional space of data-points.

Supervised Learning: **Multiple Linear Regression**

MLR equation:

In Multiple Linear Regression, the target variable(Y) is a linear combination of multiple predictor variables $x_1, x_2, x_3, \dots, x_n$. Since it is an enhancement of Simple Linear Regression, so the same is applied for the multiple linear regression equation, the equation becomes:

Where,

$$Y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

Y = Output/Response variable

$b_0, b_1, b_2, b_3, b_n, \dots$ = Coefficients of the model.

$x_1, x_2, x_3, x_4, \dots$ = Various Independent/feature variable

Assumptions for Multiple Linear Regression:

- A linear relationship should exist between the Target and predictor variables.
- The regression residuals must be normally distributed.
- MLR assumes little or no multicollinearity (correlation between the independent variable) in data.

Supervised Learning: Multiple Linear Regression

Problem Statement

Suppose we want to predict a student's **final exam score (Y)** based on:

X1: Hours of Study

X2: Number of Classes Attended

We are given the dataset:

Hours (X ₁)	Classes (X ₂)	Score (Y)
2	10	50
4	12	60
6	14	70
8	16	80

◆ Step 1: Model Formulation

Multiple Linear Regression model:

Where:

- β_0 = intercept
- β_1, β_2 = coefficients

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

◆ Step 2: Convert to Matrix Form

$$Y = X\beta$$

$$X = \begin{bmatrix} 1 & 2 & 10 \\ 1 & 4 & 12 \\ 1 & 6 & 14 \\ 1 & 8 & 16 \end{bmatrix}, \quad Y = \begin{bmatrix} 50 \\ 60 \\ 70 \\ 80 \end{bmatrix}$$

Supervised Learning: Multiple Linear Regression

◆ Step 3: Compute X^T

$$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 4 & 6 & 8 \\ 10 & 12 & 14 & 16 \end{bmatrix}$$

◆ Step 4: Compute $X^T X$

$$X^T X = \begin{bmatrix} \sum 1 & \sum X_1 & \sum X_2 \\ \sum X_1 & \sum X_1^2 & \sum X_1 X_2 \\ \sum X_2 & \sum X_1 X_2 & \sum X_2^2 \end{bmatrix}$$

◆ Calculate each term:

- $\sum 1 = 4$
- $\sum X_1 = 2 + 4 + 6 + 8 = 20$
- $\sum X_2 = 10 + 12 + 14 + 16 = 52$
- $\sum X_1^2 = 4 + 16 + 36 + 64 = 120$
- $\sum X_2^2 = 100 + 144 + 196 + 256 = 696$
- $\sum X_1 X_2 = (2 \times 10) + (4 \times 12) + (6 \times 14) + (8 \times 16)$

$$= 20 + 48 + 84 + 128 = 280$$

$$X^T X = \begin{bmatrix} 4 & 20 & 52 \\ 20 & 120 & 280 \\ 52 & 280 & 696 \end{bmatrix}$$

Supervised Learning: **Multinle** Linear Regression

◆ **Step 5: Compute $X^T Y$** $X^T Y = \begin{bmatrix} \sum Y \\ \sum X_1 Y \\ \sum X_2 Y \end{bmatrix}$

◆ Calculate:

- $\sum Y = 50 + 60 + 70 + 80 = 260$
- $\sum X_1 Y = (2 \times 50) + (4 \times 60) + (6 \times 70) + (8 \times 80)$
 $= 100 + 240 + 420 + 640 = 1400$
- $\sum X_2 Y = (10 \times 50) + (12 \times 60) + (14 \times 70) + (16 \times 80)$
 $= 500 + 720 + 980 + 1280 = 3480$

◆ So:

$$X^T Y = \begin{bmatrix} 260 \\ 1400 \\ 3480 \end{bmatrix}$$

◆ **Step 6: Normal Equation**

$$(X^T X) \beta = X^T Y$$

$$\begin{cases} 4\beta_0 + 20\beta_1 + 52\beta_2 = 260 \\ 20\beta_0 + 120\beta_1 + 280\beta_2 = 1400 \\ 52\beta_0 + 280\beta_1 + 696\beta_2 = 3480 \end{cases}$$

Supervised Learning: **Multiple Linear Regression**

◆ Step 7: Solve the System

◆ Final Model (General Form) ◆ One Valid Solution:

$$\beta_1 + \beta_2 = 5$$

$$\beta_0 = 40 - 8\beta_2$$

Let $\beta_2=1$

Then:

- $\beta_1=4$

- $\beta_0=3$

◆ Final Equation

$$Y = 32 + 4X_1 + 1X_2$$

◆ Step 8: Verification

Predict score if:

- Hours = 5

- Classes = 13

$$Y = 20 + 5(5) + 1(13)$$

$$Y = 20 + 25 + 13 = 58$$

✓ **Predicted Score = 58**

◆ Step 9: Interpretation

- $\beta_1=5$: Each extra hour of study increases score by 5 marks

- $\beta_2=1$: Each additional class adds 1 mark

- $\beta_0=20$: Base score

Supervised Learning: **Multiple Linear Regression**

Advantages of Multiple Linear Regression

1. Simple and Easy to Understand
2. Handles Multiple Variables
3. Interpretable Model
4. 4. Computationally Efficient
5. Strong Mathematical Foundation
6. Works Well with Linear Relationships

Disadvantages of Multiple Linear Regression

1. Assumes Linearity
2. Multicollinearity Issue
3. Sensitive to Outliers
4. Requires Assumptions
5. Cannot Capture Complex Patterns
6. Overfitting (if too many variables)

Evaluation Metrics in Machine Learning

Evaluation metrics are used to measure how well a machine learning model performs. They help assess whether the model is making accurate predictions and meeting the desired goals. This is important because:

- **Model Performance** : Measures how well the model works
- **Different Tasks** : Used for classification, regression and clustering
- **Right Metric Choice** : Helps select the best way to evaluate a model
- **Better Decisions** : Ensures the model meets its objective

Evaluation metrics can vary depending on the type of problem: classification or regression.

Classification Metrics

Classification problems aim to predict discrete categories. To evaluate the performance of classification models, we use the following metrics:

1. Accuracy

Accuracy is one of the simplest metrics. It measures the percentage of correctly predicted labels out of the total predictions.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Predictions}}$$

Example: If a model correctly classifies 90 out of 100 test samples, the accuracy is 90%.

Limitations: Accuracy can be misleading when working with imbalanced datasets. For example, in a dataset where 95% of the samples belong to one class, a model predicting only the majority class will have high accuracy but poor overall performance.

Solution: Use additional metrics like **Precision**, **Recall**, or the **F1 Score** for imbalanced datasets.

Evaluation Metrics in Machine Learning

2. Precision

Precision indicates the proportion of correctly predicted positive results out of all predicted positive results. It's useful when the cost of false positives is high such as in medical diagnoses where predicting a disease when it's not present can have serious consequences. It's crucial when minimizing false positives is more important than capturing all positives, such as in spam detection or financial fraud analysis.

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

Example: If a spam filter correctly identifies 80 spam emails but incorrectly flags 20 non-spam emails as spam, the precision is $\frac{80}{80+20} = 80\%$

Use Case: In medical testing, precision is vital when confirming a diagnosis (e.g., detecting cancer) to avoid unnecessary treatments due to false positives.

Precision helps ensure that when the model predicts a positive outcome, it's likely to be correct.

3. Recall

Recall measures the model's ability to correctly identify all actual positive instances. It is important when missing a positive case (false negative) is more costly than false positives.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Where: FN = False Negatives

Example: If a model detects 90 out of 100 actual spam emails, its recall is

$$\frac{90}{90+10} = 90\%$$

Use Case: In fraud detection, a high recall ensures that as many fraudulent cases as possible are flagged, even if it means some false positives.

Evaluation Metrics in Machine Learning

4. F1 Score

The F1 Score is the harmonic mean of precision and recall. It is useful when we need a balance between precision and recall as it combines both into a single number. A high F1 score means the model performs well on both metrics. Its range is [0,1].

Lower recall and higher precision gives us great accuracy but then it misses a large number of instances. More the F1 score better will be performance. It can be expressed mathematically in this way:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\frac{2 \times 0.8 + 0.7}{0.8 + 0.7} = 74.3\%$$

Example: For a model with precision of 80% and recall of 70%, the F1 score is:

Use Case: It is effective in imbalanced datasets where Precision and Recall might not individually reflect the model's true performance.

5. Logarithmic Loss (Log Loss)

Log Loss evaluates the uncertainty of a model's predictions by considering the probability of predicted classes. Models that assign high probabilities to incorrect predictions are penalized more. If there are N samples belonging to the M class, then we calculate the Log loss in this way:

$$\text{Log Loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1-y_i) \log(1-p_i)]$$

Example: If a model predicts a probability of 0.9 for a correct class, the penalty is minimal. However, predicting 0.1 for the correct class results in a higher penalty.

Use Case: Log Loss is frequently used in multi-class classification problems to capture the model's confidence. The goal is to minimize Log Loss, as a lower Log Loss shows higher prediction accuracy.

Evaluation Metrics in Machine Learning

6. Specificity

Specificity measures the proportion of correctly identified negatives out of all actual negatives. It complements Recall in scenarios where minimizing false positives is also critical.

$$\text{Specificity} = \frac{\text{True Negatives (TN)}}{\text{True Negatives (TN)} + \text{False Positives (FP)}}$$

Use Case: In medical screening, high specificity ensures that healthy individuals are not incorrectly diagnosed with a disease.

7. Area Under the Curve (AUC) – Receiver Operating Characteristic (ROC)

AUC-ROC measures the model's ability to distinguish between positive and negative classes across various threshold levels. It plots the True Positive Rate (Recall) against the False Positive Rate (1 – Specificity). AUC ranges from 0 to 1 with higher values showing better model performance.

1. True Positive Rate (TPR) Also known as sensitivity or recall, the True Positive Rate measures how many actual positive instances were correctly identified by the model. It answers the question: "Out of all the actual positive cases, how many did the model correctly identify?"

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{TNR} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

2. True Negative Rate (TNR)

Also called specificity, the True Negative Rate measures how many actual negative instances were correctly identified by the model. It answers the question: "Out of all the actual negative cases, how many did the model correctly identify as negative?"

Evaluation Metrics in Machine Learning

3. False Positive Rate(FPR)

It measures how many actual negative instances were incorrectly classified as positive. It's a key metric when the cost of false positives is high such as in fraud detection.

$$FPR = \frac{FP}{FP + TN}$$

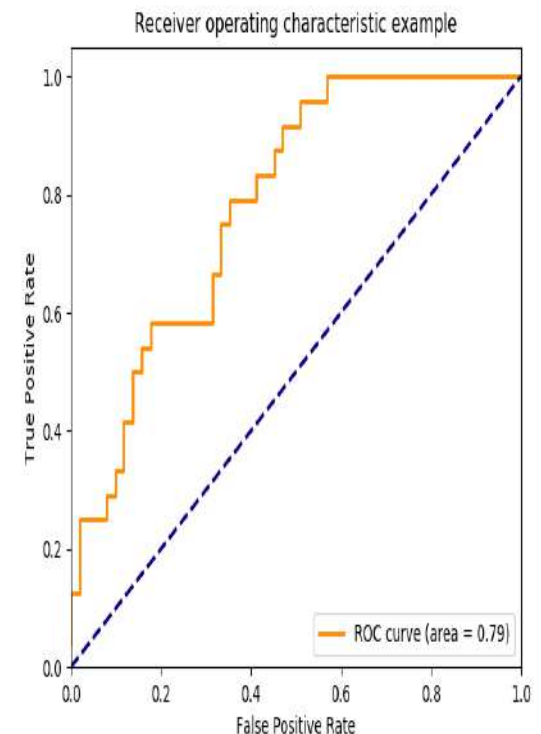
$$FNR = \frac{FN}{FN + TP}$$

4. False Negative Rate(FNR)

It measures how many actual positive instances were incorrectly classified as negative. It answers: "Out of all the actual positive cases, how many were misclassified as negative?"

It is a graphical representation of the True Positive Rate (TPR) vs the False Positive Rate (FPR) at different classification thresholds. The curve helps us visualize the trade-offs between sensitivity (TPR) and specificity (1 - FPR) across various thresholds. Area Under Curve (AUC) quantifies the overall ability of the model to distinguish between positive and negative classes.

- AUC = 1: Perfect model (always correctly classifies positives and negatives).
- AUC = 0.5: Model performs no better than random guessing.
- AUC < 0.5: Model performs worse than random guessing (showing that the model is inverted).



Evaluation Metrics in Machine Learning

8. Confusion Matrix

A confusion matrix provides a comprehensive overview of a model's predictions, breaking them down into:

- **True Positives (TP)**: Correctly predicted positives. It is the case where we predicted Yes and the real output was also Yes.
- **True Negatives (TN)**: Correctly predicted negatives. It is the case where we predicted No and the real output was also No.
- **False Positives (FP)**: Incorrectly predicted positives. It is the case where we predicted Yes but it was actually No.
- **False Negatives (FN)**: Missed positives. It is the case where we predicted No but it was actually Yes.

Confusion matrix creates a $N \times N$ matrix, where N is the number of classes or categories that are to be predicted. Here we have $N = 2$, so we get a 2×2 matrix. Suppose there is a problem with our practice which is a binary classification. Samples of that classification belong to either Yes or No. So, we build our classifier which will predict the class for the new input sample. After that, we tested our model with 165 samples and we get the following result.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

n=165	Predicted No	Predicted Yes
Actual No	50	10
Actual Yes	5	100

Evaluation Metrics in Machine Learning

9. Balanced Accuracy

Balanced Accuracy accounts for imbalanced datasets by averaging the Recall of each class.

$$\text{Balanced Accuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2}$$

Use Case: Useful when classes are highly imbalanced, such as fraud detection or rare disease diagnosis.

Regression Metrics

In the regression task, we are supposed to predict the target variable which is in the form of continuous values. To evaluate the performance of such a model below metrics are used:

1. Mean Absolute Error (MAE)

MAE measures the average magnitude of errors in a set of predictions, without considering their direction. It represents the average absolute difference between predicted values and actual values.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- y_i : Actual value.
- $\hat{y}_i$: Predicted value.
- N : Number of data points.

Example: If the actual values are [3, 5, 7] and the predicted values are [2, 5, 8], the MAE is:

$$\frac{(3-2)^2 + (5-5)^2 + (7-8)^2}{3} = \frac{1 + 0 + 1}{3} = 0.67$$

2. Mean Squared Error (MSE)

MSE calculates the average squared difference between the actual and predicted values. Unlike MAE, it gives higher weight to larger errors, making it sensitive to outliers.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Evaluation Metrics in Machine Learning

Example: Using the same values as above, the MSE is: $\sqrt{0.67} \approx 0.82$

Use Case: RMSE is preferred in applications where the magnitude of the error is critical, such as predicting demand for utilities.

4. Root Mean Squared Logarithmic Error (RMSLE)

RMSLE calculates the square root of the logarithmic differences between predicted and actual values. It penalizes under-predictions more than over-predictions, making it suitable for targets with large ranges.

$$RMSLE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\log(1 + y_i) - \log(1 + \hat{y}_i))^2}$$

Example: If $y_i = [10, 100]$ and $\hat{y}_i = [12, 110]$, RMSLE computes the error considering their relative differences.

Use Case: RMSLE is often used in scenarios like sales forecasting, where large variations in the target variable exist.

5. R² Score (Coefficient of Determination)

R² measures the proportion of variance in the dependent variable that is predictable from the independent variable(s). It ranges from 0 to 1, with higher values indicating better performance.

$$R^2 = 1 - \frac{\sum_{j=1}^n (y_j - \hat{y}_j)^2}{\sum_{j=1}^n (y_j - \bar{y})^2}$$

Where:

Example: An score of 0.85 means the model explains 85% of the variance in the target variable.

Use Case: It's widely used in linear regression and serves as a quick measure of model performance.

- y_j = Actual value
- $\hat{y}_j$ = Predicted value
- $\bar{y}$ = Mean of the actual values